Introducción a las Redes Neuronales — Curso completo

Repositorio: https://github.com/datahuba/curso-redes-neuronales
Basado en el curso, el libro y los notebooks del Dr. J. Irving Vasquez (jivg.org).

Este documento reúne, en orden, las 7 clases. Cada clase incluye:

- su **guía paso a paso**;
- sus **diapositivas**;
- las **guías de sus notebooks**.

Los notebooks (`.ipynb`) se trabajan en la computadora, desde la carpeta `notebooks/` del repositorio.

Contenido

Sección	Página
Clase 1 · Introducción e instalación	**3**
Guía de la clase	3
Diapositivas	17
Clase 2 · McCulloch y Pitts y el perceptrón	**36**
Guía de la clase	36
Diapositivas	48
Guía del notebook 1_01_mcculloch_pitts	69
Guía del notebook 1_02_perceptron_AND	71
Guía del notebook 1_03_1_perceptron_simple	73
Guía del notebook 1_03_2_regla_de_aprendizaje	75
Clase 3 · Neurona moderna y descenso por gradiente	**78**
Guía de la clase	78
Diapositivas	88
Guía del notebook 1_04_red_neuronal_simple	108
Guía del notebook 1_05_descenso_por_gradiente_1D	110
Clase 4 · Regla delta, ADALINE y varias salidas	**112**
Guía de la clase	112
Diapositivas	121
Guía del notebook 1_06_descenso_unpaso	142

Sección	Página
Guía del notebook 1_07_descenso_por_gradiente	144
Guía del notebook 1_08_adaline	147
Clase 5 · Redes multicapa y retropropagación	**149**
Guía de la clase	149
Diapositivas	156
Guía del notebook 1_09_red_multicapa_ejercicio	177
Guía del notebook 1_10_retropropagacion_base	180
Guía del notebook 1_11_retropropagacion_completo	183
Clase 6 · Optimizadores	**186**
Guía de la clase	186
Diapositivas	193
Guía del notebook 21_opt_gradiente_2d	214
Guía del notebook 22_opt_momentum	217
Guía del notebook 23_opt_lr_decay	220
Clase 7 · Redes convolucionales	**223**
Guía de la clase	223
Diapositivas	230
Guía del notebook 3_1_capa_convolucional	251
Guía del notebook 3_2_entrenar_capa_conv	254

Guía paso a paso — Clase 1: Introducción a las Redes Neuronales

Temas: presentación del curso · instalación de herramientas · inteligencia artificial y redes neuronales · aprendizaje automático
Duración: ~2 h 15 min
Material: diapositivas `Clase1_Diapositivas` (PDF y PowerPoint, 19 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales

Esta guía explica **todo** lo que aparece en las diapositivas desde cero, como si nunca hubieras visto el tema. Tiene tres partes:

- **Parte A:** preparar las computadoras **antes** de clase. Es una instalación rigurosa, con una verificación en cada paso.
- **Parte B:** qué explicar en cada diapositiva, con definiciones sencillas y ejemplos.
- **Parte C:** problemas frecuentes y cómo resolverlos.

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar qué es la inteligencia artificial y dónde encajan las redes neuronales.
2. Distinguir las tareas del aprendizaje automático (regresión, clasificación, agrupamiento…) y sus tres tipos (supervisado, no supervisado, por refuerzo).
3. Tener su computadora lista para hacer los ejercicios del curso.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación y motivación	1–5
0:20	Herramientas e instalación	6–8
0:55	Descanso	—
1:05	Inteligencia artificial y redes neuronales	9–12
1:35	El aprendizaje automático	13–17
2:05	Quiz y cierre	18–19

PARTE A — Preparar las computadoras (hazlo antes de clase)

A.0 Conceptos mínimos antes de instalar

Palabra	Qué es, en simple
Python	El lenguaje de programación que usaremos.
Paquete / librería	Código que otros ya escribieron y que instalamos para usarlo. Ejemplo: **NumPy** sirve para hacer cuentas con vectores y matrices.
Anaconda / conda	Programa que instala Python y administra "ambientes".
Ambiente	Una "caja" aislada con su propia versión de Python y sus paquetes. Así un proyecto no rompe a otro. El nuestro se llamará `nni` (*neural network introduction*).
pip	El instalador de paquetes de Python.
Terminal	Ventana donde se escriben comandos de texto. Usaremos **Anaconda PowerShell Prompt**.
Git	Programa para descargar y versionar código.
GitHub	Sitio web donde se guarda código. Allí está el repositorio del curso.
Repositorio (repo)	Carpeta de un proyecto con todos sus archivos.
Clonar	Descargar un repositorio a tu computadora con Git.
Notebook (.ipynb)	Documento que mezcla texto y celdas de código que se ejecutan una por una. Los ejercicios vienen así.
Jupyter	El programa que ejecuta los notebooks.
VS Code	Editor de código donde abriremos los notebooks.
Kernel	El "motor" de Python que ejecuta las celdas de un notebook. Debemos elegir el de nuestro ambiente `nni`.

Regla de oro: después de cada paso hay una **verificación**. No avances si la verificación falla; ve a la Parte C.

A.1 Elegir el camino

Camino	Cuándo usarlo	Qué necesita
Camino 1: Google Colab	Computadoras sin permisos para instalar, poco tiempo o plan B	Navegador y cuenta de Google
Camino 2: Instalación local	Es lo recomendado en el curso: da experiencia real de desarrollo	Instalar Anaconda, Git y VS Code (unos 40 minutos y unos 5 GB libres)

Recomendación: prepara el **Camino 2** en tu computadora y ten listo el **Camino 1** como respaldo para los alumnos que tengan problemas.

A.2 Camino 1 — Google Colab (sin instalar nada)

1. Entra a https://colab.research.google.com e inicia sesión con tu cuenta de Google.
2. Menú **Archivo** → **Abrir cuaderno** → pestaña **GitHub**.
3. Pega `datahuba/curso-redes-neuronales` y presiona Enter.
4. Elige el notebook, por ejemplo `notebooks/1_01_mcculloch_pitts.ipynb`.
5. Haz clic en la primera celda de código y presiona **Shift + Enter** para ejecutarla.
6. Para guardar tus cambios: **Archivo** → **Guardar una copia en Drive**. Si no lo haces, se pierden.

Verificación: crea una celda nueva (botón **+ Código**), escribe esto y ejecútala:

```
import numpy as np
print(np.__version__)
```

Si aparece un número de versión (por ejemplo `1.26.4`), Colab funciona.

Notebooks que usan la carpeta `nni` o archivos de datos (como `1_03_2`): en Colab necesitan el repositorio completo. Ejecuta esto en la primera celda:

```
!git clone https://github.com/datahuba/curso-redes-neuronales
%cd curso-redes-neuronales/notebooks
```

A.3 Camino 2 — Instalación local en Windows

Son **7 pasos**, en este orden:

- Pasos 1 a 3: instalar tres programas, con clics.
- Pasos 4 a 7: escribir comandos en **una sola** terminal, Anaconda PowerShell Prompt.

Paso 1. Instalar Anaconda

1. Entra a https://www.anaconda.com/download
2. Descarga el instalador para **Windows** (64 bits).
3. Ejecuta el archivo descargado (`Anaconda3-...-Windows-x86_64.exe`).
4. En el instalador:
   - **Next** → **I Agree**.
   - Elige **Just Me (recommended)**.
   - Deja la carpeta que propone. Es importante que la ruta no tenga espacios ni acentos; normalmente es `C:\Users\TuUsuario\anaconda3`.
   - En opciones avanzadas, deja marcadas las que vienen por defecto. **No** marques "Add Anaconda to my PATH".
   - **Install** y espera de 5 a 15 minutos → **Finish**.

Verificación:

1. Abre el menú Inicio, escribe `Anaconda PowerShell Prompt` y ábrelo.
2. Debe verse una línea que empieza con `(base)`, por ejemplo:

```
(base) PS C:\Users\TuUsuario>
```

3. Escribe y presiona Enter:

```
conda --version
```

Debe responder algo como `conda 24.x.x`.

> Anaconda tiene una interfaz gráfica (Anaconda Navigator), pero es lenta. Usaremos la terminal.

Paso 2. Instalar Git for Windows

1. Entra a https://git-scm.com/download/win
2. Descarga **64-bit Git for Windows Setup** y ejecútalo.
3. Acepta todas las opciones por defecto (**Next** en todo) → **Install** → **Finish**.

Verificación: cierra y vuelve a abrir Anaconda PowerShell Prompt (así reconoce el programa nuevo) y escribe:

```
git --version
```

Debe responder algo como `git version 2.4x.x.windows.1`.

Paso 3. Instalar VS Code y sus extensiones

1. Entra a https://code.visualstudio.com, descarga e instala con las opciones por defecto. Marca **"Agregar a PATH"** si aparece.
2. Abre VS Code.
3. Instala dos extensiones:
    1. Haz clic en el ícono de **Extensiones** de la barra izquierda (cuatro cuadritos) o presiona `Ctrl + Shift + X`.
    2. Busca **Python** (de Microsoft) → **Install**.
    3. Busca **Jupyter** (de Microsoft) → **Install**.

Verificación: en la barra izquierda, dentro de Extensiones, aparecen **Python** y **Jupyter** como instaladas.

Paso 4. Crear y activar el ambiente `nni`

En **Anaconda PowerShell Prompt** escribe:

```
conda create --name nni python=3.11
```

Qué significa:

- `conda create` → crea un ambiente nuevo.
- `--name nni` → le pone de nombre `nni`.
- `python=3.11` → le instala Python 3.11. En el video se usa `python=3`; 3.11 es una versión 3 concreta y estable.

Cuando pregunte `Proceed ([y]/n)?`, escribe `y` y presiona Enter. Tarda de 1 a 3 minutos.

Luego actívalo:

```
conda activate nni
```

Verificación: el inicio de la línea debe cambiar de `(base)` a `(nni)`:

```
(nni) PS C:\Users\TuUsuario>
```

⚠️ **Cada vez que abras una terminal nueva**, el ambiente vuelve a `(base)`. Debes repetir `conda activate nni`.

Paso 5. Descargar (clonar) el repositorio del curso

En la misma terminal, con `(nni)` visible:

```
cd ~
```

```
mkdir projects
```

```
cd projects
```

```
git clone https://github.com/datahuba/curso-redes-neuronales.git
```

```
cd curso-redes-neuronales
```

Qué significa:

- `cd ~` → ir a tu carpeta de usuario.
- `mkdir projects` → crear una carpeta `projects`. Si ya existía, muestra un error en rojo; no pasa nada, continúa.
- `cd projects` → entrar a esa carpeta.
- `git clone <url>` → descargar una copia del repositorio.
- `cd curso-redes-neuronales` → entrar a la carpeta del curso.

Verificación: escribe `ls`. Deben aparecer `docs`, `notebooks`, `README.md` y `requirements.txt`.

> **Alternativa sin Git:** en GitHub, botón verde **Code** → **Download ZIP**. Descomprímelo en `C:\Users\TuUsuario\projects` y entra a esa carpeta con `cd`.
>
> En el video se usa la terminal **Git Bash** para clonar; también funciona, pero así usamos una sola ventana para todo.

Paso 6. Instalar los paquetes

Dentro de la carpeta `curso-redes-neuronales`, con `(nni)` activo:

```
pip install -r requirements.txt
```

`-r requirements.txt` le dice a pip que instale **la lista de paquetes del curso**, que viene en ese archivo:

Paquete	Para qué
`jupyter`	Ejecutar notebooks
`numpy`	Vectores, matrices y cuentas
`pandas`	Tablas de datos
`matplotlib`	Gráficas
`scikit-learn`	Utilidades de aprendizaje automático (el notebook `1_03_2` lo necesita)
`pillow`	Abrir imágenes
`ipykernel`	Que VS Code reconozca el ambiente como kernel

Tarda de 2 a 5 minutos. La última línea debe decir `Successfully installed ...`.

Verificación:

```
python -c "import numpy, pandas, matplotlib, sklearn, PIL; print('TODO OK')"
```

Debe imprimir `TODO OK`. Si dice `ModuleNotFoundError`, revisa que estés en `(nni)` y repite el Paso 6.

Paso 7. Abrir los notebooks en VS Code (verificación final)

1. En VS Code: **File** → **Open Folder...** (Archivo → Abrir carpeta) → elige `C:\Users\TuUsuario\projects\curso-redes-neuronales` → **Seleccionar carpeta**. Si pregunta si confías en los autores, elige **Yes, I trust**.
2. En el panel izquierdo abre `notebooks` → `1_01_mcculloch_pitts.ipynb`.
3. **Elige el kernel (muy importante):**
    1. Arriba a la derecha, haz clic en **Select Kernel**.
    2. Elige **Python Environments...**.
    3. Elige el que diga **nni**, por ejemplo `nni (Python 3.11.x)`.
4. Crea una celda (botón **+ Code**), escribe esto y ejecútalo con **Shift + Enter**:

```
import sys, numpy as np
print(sys.executable)
print(np.__version__)
```

Verificación final:

- La primera línea debe contener `envs\nni`; eso prueba que usas el ambiente correcto.
- La segunda debe ser un número de versión.

✅ Si ves ambas cosas, **la computadora está lista**. Borra la celda de prueba.

Resumen de comandos (para copiar)

Todo en **Anaconda PowerShell Prompt**:

```
conda create --name nni python=3.11
```

```
conda activate nni
```

```
cd ~
```

```
mkdir projects
```

```
cd projects
```

```
git clone https://github.com/datahuba/curso-redes-neuronales.git
```

```
cd curso-redes-neuronales
```

```
pip install -r requirements.txt
```

Alternativa sin VS Code: con `(nni)` activo, dentro de `curso-redes-neuronales`:

```
jupyter notebook
```

Se abre el navegador con la lista de archivos. Para cerrarlo, en la terminal presiona `Ctrl + C`.

A.4 Checklist del día de la clase

- [ ] Tu computadora pasó la **verificación final** del Paso 7.
- [ ] Probaste abrir un notebook en **Colab** (plan B).
- [ ] Tienes abiertas las diapositivas.
- [ ] Llegaste 15 minutos antes para probar el proyector.
- [ ] Si los alumnos instalarán en clase, avísales que traigan laptop con **5 GB libres** y permisos de administrador. Si pueden, que descarguen Anaconda en casa: son unos 900 MB.

PARTE B — Qué explicar en cada diapositiva

Junto a cada título va el tiempo aproximado.

Diapositiva 1 — Portada (2 min)

Qué decir: "Este es un curso introductorio. Vamos a entender cómo funcionan las redes neuronales **por dentro**: su teoría y cómo programarlas desde conceptos simples. Esto es la base para después estudiar aprendizaje profundo (*deep learning*) y construir cosas más complejas."

Diapositiva 2 — Agenda (1 min)

Hoy vemos 4 temas, uno por cada video del curso:

1. Presentación del curso.
2. Instalación de herramientas.
3. Inteligencia artificial y redes neuronales.
4. Aprendizaje automático.

Diapositiva 3 — Objetivos (1 min)

Lee los tres objetivos de la clase (ver arriba). Pide que al final de la clase ellos mismos digan si los cumplieron.

Diapositiva 4 — "Ya las usas todos los días" (10 min)

Actividad (5 min): pregunta "¿Qué aplicación con inteligencia artificial usaron hoy?". Anota las respuestas en el pizarrón.

Explica cada tarjeta:

- **Vehículos:** autos que ven la calle con cámaras y deciden cómo manejar.
- **Lenguaje:** Alexa, Google, traductores y chatbots entienden y producen texto.
- **Imagen:** los filtros de TikTok detectan tu cara y la modifican.
- **Salud:** detectan enfermedades en radiografías.
- **Videojuegos:** generan gráficos realistas más rápido. Calcular la luz real (*ray tracing*) es muy costoso.
- **Refuerzo:** programas que aprenden a jugar jugando muchas veces.
- **Generativa:** DALL·E y Stable Diffusion crean imágenes a partir de un texto. Hoy ya cuesta distinguir si una imagen la hizo una persona o una IA.
- **Robótica:** robots que perciben y se mueven.

Mensaje clave: las redes neuronales ya trabajan "por detrás" en muchos lados; por eso conviene saber cómo funcionan.

Diapositiva 5 — "¿Qué veremos en el curso?" (5 min)

Recorre los módulos:

1. **Introducción** (hoy).
2. **Principios:** la neurona de McCulloch y Pitts y el perceptrón.
3. **Red neuronal simple:** funciones de activación y descenso por gradiente.
4. **Redes multicapa:** retropropagación.
5. **Preliminares (opcional):** cálculo, álgebra lineal, probabilidad y Python. Quien sienta que le falta base, que empiece por aquí.

Cómo trabajamos cada tema: leer el libro → explicación en clase → ejercicio en papel → notebook en Python.

Diapositiva 6 — "Dos formas de hacer los ejercicios" (5 min)

- Los ejercicios son **notebooks** con instrucciones: solo hay que **rellenar partes del código**.
- Usamos solo **Python y NumPy**, "a pie", a propósito. Así entendemos qué pasa por dentro, en lugar de llamar a una función mágica como `.fit()` que lo hace todo sin que veamos nada.
- **Opción 1, Colab:** no se configura nada; funciona en el navegador.
- **Opción 2, tu máquina:** más trabajo al inicio, pero es como se trabaja en proyectos reales y en equipo.

Diapositiva 7 — "Instalación (1/2): tres programas" (10 min)

Muestra dónde se descarga cada programa (Pasos 1–3 de la Parte A) y para qué sirve:

- **Anaconda:** Python + ambientes.
- **Git:** descargar el código del curso.
- **VS Code:** editor; necesita las extensiones **Python** y **Jupyter**.

Si los alumnos ya los instalaron en casa, pasa directo a la siguiente.

Diapositiva 8 — "Instalación (2/2): los comandos" (20 min)

Haz la instalación en vivo (Pasos 4–7) y que los alumnos la repitan.

Comando	En palabras
`conda create --name nni python=3.11`	"Crea una caja llamada nni con Python dentro."
`conda activate nni`	"Entra a la caja. Fíjense que ahora dice (nni)."
`cd ~`, `mkdir projects`, `cd projects`	"Voy a mi carpeta de usuario, creo la carpeta de proyectos y entro."
`git clone <url>`	"Descarga una copia del repositorio del curso."
`cd curso-redes-neuronales`	"Entra a la carpeta del curso."
`pip install -r requirements.txt`	"Mete en la caja todas las herramientas de la lista."
VS Code → Abrir carpeta → kernel `nni`	"Abro el proyecto en el editor y elijo el motor correcto."

Termina cuando todos vean la **verificación final** del Paso 7. Quien no lo logre, que use Colab hoy y lo resuelva después con la Parte C.

Diapositiva 9 — "¿Qué es la inteligencia artificial?" (8 min)

- **No existe una definición única** aceptada por todos, ni un teorema que diga "esto sí es IA y esto no".
- El libro de **Russell y Norvig** (*Inteligencia Artificial: un enfoque moderno*) agrupa las definiciones en dos enfoques:
  1. **Imitar al humano:** construir máquinas que piensen como nosotros, duplicando lo que hace el cerebro. Problema: si copiamos perfectamente al humano, también copiamos sus errores.
  2. **Actuar racionalmente:** que la máquina **perciba** su entorno, **razone** (procese) y **actúe** para resolver el problema, aunque no lo haga como el cerebro.
- **Ejemplo:** una calculadora no piensa como un humano, pero resuelve sumas perfectamente. Es "racional" para esa tarea.

Diapositiva 10 — "El test de Turing" (10 min)

El problema: como no hay una definición de inteligencia, Alan Turing propuso una **prueba**.

Cómo funciona:

1. Una persona conversa por mensajes, como en un chat, con "alguien" que no ve.
2. Del otro lado puede haber una persona o una máquina.
3. Si la persona **no logra distinguir** si habla con una máquina, la máquina **pasa** el test y se considera inteligente.

¿Por qué es "operacional"? Porque solo mide si pasa o no; no le importa cómo funciona por dentro.

Qué necesita una máquina para pasarlo (explica las flechas en orden, como un flujo):

1. **Lenguaje natural:** recibir y entender el mensaje.
2. **Representación del conocimiento:** convertir el mensaje en algo que la máquina pueda manejar internamente.
3. **Inferencia:** a partir de ese conocimiento, generar una respuesta.
4. **Síntesis:** convertir la respuesta otra vez en lenguaje para el humano.
5. **Aprendizaje:** adaptarse a cada usuario distinto.

Conexión con el curso: con el aprendizaje profundo, las redes neuronales ya cubren todas esas partes:

- extraen características a una representación interna llamada **espacio latente**;
- hacen inferencia;
- generan la salida con un **decodificador** (texto o imagen);
- y sus parámetros internos (**pesos**) se ajustan a cada tarea.

Diapositiva 11 — "¿Qué tan lejos llegan las redes?" (8 min)

Dibuja **tres ejes** en el pizarrón:

Eje	Significa	Redes neuronales
Operacional	Resolver la tarea: controlar, jugar, reconocer, generar	**Muy buenas**: reconocen personas, manejan autos, crean imágenes
Razonamiento	Representar conocimiento, inferir como humano y, al extremo, tener conciencia	Limitadas, al menos por ahora
Explicabilidad	Poder garantizar cuándo funcionan y cuándo no, y por qué	Limitadas: a veces fallan y no sabemos exactamente por qué

Conclusión: son herramientas muy efectivas para resolver problemas, pero no tienen conciencia ni son completamente explicables. Por eso en el curso nos interesa mucho **cómo entrenarlas** bien.

Diapositiva 12 — "Dónde encajan las redes neuronales" (5 min)

Explica los recuadros, uno dentro de otro:

- **Inteligencia artificial** (el más grande): todo intento de que las máquinas hagan tareas inteligentes.
    - **Aprendizaje automático:** la parte de la IA que **aprende de ejemplos**.

- **Redes neuronales:** un tipo de aprendizaje automático formado por muchas unidades simples conectadas.
  - **Aprendizaje profundo:** redes con **muchas capas**; cada capa extrae rasgos más abstractos (bordes → formas → objetos).

Diapositiva 13 — "El aprendizaje automático" (8 min)

Definición (Arthur Samuel, 1959): "dar a las computadoras la capacidad de aprender sin ser programadas explícitamente".

Ejemplo del ajedrez:

- Queremos una computadora que gane al ajedrez.
- Un programador **no puede** escribir qué hacer en cada posición posible del tablero: hay muchísimas combinaciones, sería intratable.
- Solución: que la computadora **aprenda de la experiencia**, por ejemplo jugando muchas partidas.

Tres ingredientes (están en la diapositiva):

1. **Tarea:** ganar al ajedrez.
2. **Experiencia:** partidas jugadas o datos.
3. **Desempeño:** mejora después de observar; gana más seguido.

Diapositiva 14 — "Tarea 1: Regresión" (10 min)

Qué es: encontrar una **función** que relacione una **entrada** con una **salida numérica continua**. Ejemplo: entrada = metros cuadrados de una casa; salida = precio.

Explica la gráfica:

- **Puntos naranjas = datos** (la "experiencia" o *dataset*). Pueden venir de estudios, mediciones o simulaciones.
- **Línea azul = la función real.** En la vida real **no la conocemos**; solo tenemos los puntos.
- **Líneas grises = hipótesis:** funciones candidatas que proponemos para explicar los puntos.

Conceptos:

- Una hipótesis es **consistente** si explica bien los datos que tenemos.
- Pero eso no basta: queremos que también funcione con **datos futuros** del mismo fenómeno. Eso se llama **generalización**.

Pregunta a la clase: "¿Una curva que pase exactamente por todos los puntos haciendo zigzag es buena?" → No necesariamente: puede explicar estos datos y fallar con los nuevos.

Diapositiva 15 — "Otras tareas" (10 min)

1. **Clasificación:** asignar una **etiqueta** de una lista predefinida $\{l_1, \ldots, l_n\}$. Ejemplo: le das una foto y responde "perro" o "gato" (dataset *Dogs vs Cats* de Kaggle). A diferencia de la regresión, la salida es una **categoría**, no un número continuo.
2. **Agrupamiento (clustering): no hay etiquetas**; el algoritmo agrupa solo, por parecido. Ejemplo: con medidas como qué tan puntiagudas son las orejas o qué tan largos los bigotes, encuentra dos grupos

(que resultan ser perros y gatos). Cada grupo tiene su **media** (centro) y su **varianza** (qué tan dispersos están).

3. **Reducción de dimensionalidad:** describir algo con **menos características**. Ejemplo: en 2 ejes, perros y gatos están revueltos y ninguna recta los separa; si los proyectamos a 1 solo eje nuevo, basta un **umbral** (un punto) para separarlos.
4. **Modelado generativo:** aprender la **distribución** de los datos para **crear ejemplos nuevos**. Ejemplo: aprender "cómo es un rostro" y generar rostros que no existen.

Diapositiva 16 — "Tipos de aprendizaje" (10 min)

1. **Supervisado:**
   - La experiencia es una lista de **pares (x, y)**: x = el ejemplo (una figura), y = su etiqueta (cono, cubo, esfera).
   - El algoritmo busca la función que lleva de x a y. Ese proceso se llama **entrenamiento**.
   - Después debe acertar con ejemplos **nuevos** que nunca vio: **generalización**. Ejemplo: un auto autónomo debe manejar en calles donde nunca se entrenó.
2. **No supervisado:**
   - Hay ejemplos **sin etiquetas**.
   - El algoritmo descubre qué los distingue (los cubos tienen 6 caras; las esferas no tienen vértices, aunque cambie su tamaño o color) y los agrupa.
3. **Por refuerzo:**
   - La experiencia **no está disponible** de antemano: se consigue **interactuando**.
   - Se detalla en la siguiente diapositiva.

Diapositiva 17 — "Aprendizaje por refuerzo" (8 min)

Explica el ciclo **Agente ⇄ Ambiente** del diagrama:

1. El **agente** (el programa) hace una **acción**, por ejemplo mover una pieza de ajedrez.
2. La acción cambia el **estado** del **ambiente** (el tablero).
3. El agente **observa** el nuevo estado y recibe una **recompensa**.
4. Se repite muchas veces.

Conceptos:

- **Recompensa a corto plazo:** comer una pieza.
- **Recompensa a largo plazo (horizonte):** ganar la partida. Es lo que realmente importa.
- **Objetivo:** maximizar la recompensa total.
- Al inicio el agente pierde mucho. Con la experiencia ajusta su regla de "en este estado, haz esta acción" y termina ganando.

Cierre del tema: las redes neuronales sirven para muchas de estas tareas: regresión, clasificación, reducción de dimensionalidad y generación.

Diapositiva 18 — Quiz (10 min)

Que respondan en papel durante 5 minutos; luego revisa en voz alta.

#	Pregunta	Respuesta
1	¿Por qué el test de Turing es una definición operacional?	Solo mide si pasa o no; no importa cómo funciona por dentro.
2	¿En cuál de los tres ejes destacan las redes?	El operacional: resolver problemas.
3	¿Qué es generalizar?	Funcionar bien con ejemplos que nunca vio.
4	¿Diferencia entre clasificación y agrupamiento?	La clasificación tiene etiquetas predefinidas; el agrupamiento no.
5	¿Qué maximiza un agente de refuerzo?	La recompensa acumulada (a largo plazo).

Diapositiva 19 — "Para la próxima clase" (2 min)

1. Tener el ambiente instalado y el repositorio clonado (Parte A).
2. Leer **Russell y Norvig:** capítulo 1 y secciones 18.1 y 18.2 (*Learning from examples*).
3. Leer **Goodfellow, Bengio y Courville, *Deep Learning*:** sección 5.1 (*Learning algorithms*).

La próxima clase empezaremos con la primera neurona artificial: el modelo de **McCulloch y Pitts**.

PARTE C — Problemas frecuentes y soluciones

Síntoma	Causa	Solución
`conda` no se reconoce como comando	Abriste PowerShell o CMD normal	Abre **Anaconda PowerShell Prompt** desde el menú Inicio
Sigue diciendo `(base)`	No activaste el ambiente	`conda activate nni`
`EnvironmentNameNotFound: nni`	El ambiente no se creó	Repite el Paso 4 y revisa con `conda env list`
`git` no se reconoce	Git no instalado, o terminal abierta antes de instalarlo	Instala Git (Paso 2) y **abre una terminal nueva**
`mkdir` dice que la carpeta ya existe	Ya la habías creado	No es un error grave; sigue con `cd projects`
`destination path ... already exists`	Ya clonaste antes	No pasa nada; entra con `cd curso-redes-neuronales` y actualiza con `git pull`
`Could not open requirements file`	No estás dentro de `curso-redes-neuronales`	`cd ~\projects\curso-redes-neuronales` y repite el Paso 6
`ModuleNotFoundError: No module named 'numpy'` en VS Code	El kernel elegido no es `nni`	Select Kernel → Python Environments → **nni**

Síntoma	Causa	Solución
VS Code no muestra `nni` en la lista de kernels	Falta `ipykernel` o la extensión	Con `(nni)` activo: `pip install ipykernel`; instala las extensiones Python y Jupyter; reinicia VS Code
`ModuleNotFoundError: No module named 'nni'`	El notebook no se abrió desde el repositorio	Abre en VS Code la carpeta `curso-redes-neuronales` completa, no el archivo suelto
`pip` falla con error de red o SSL	Internet o proxy de la institución	Prueba otra red, o usa Colab
La instalación de Anaconda falla por permisos	Sin permisos de administrador	Usa Colab (Camino 1)
Las tildes se ven como `�` en la terminal	Codificación de Windows	Ejecuta `chcp 65001` antes; es solo visual
Colab perdió mis cambios	No se guardó una copia	**Archivo → Guardar una copia en Drive**

Si nada funciona: usa Colab en clase y revisa la instalación después, con calma, repitiendo la Parte A paso por paso.

CLASE 1 · INTRODUCCIÓN

Introducción a las Redes Neuronales

Presentación del curso, herramientas, IA y aprendizaje automático

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

01

Presentación del curso

Motivación y temario

02

Instalar los ejercicios

Anaconda, Git, VS Code, NumPy

03

IA y redes neuronales

Definiciones, test de Turing, alcances

04

El aprendizaje automático

Definición, tareas y tipos

Al terminar la clase podrás...

1

Explicar qué es la IA

y dónde encajan las redes neuronales dentro de ella.

2

Distinguir tareas y tipos

de aprendizaje automático: regresión, clasificación, refuerzo...

3

Tener tu computadora lista

para resolver los ejercicios del curso en Python.

Ya las usas todos los días

Vehículos

Conducción autónoma y asistida

Lenguaje

Asistentes, traductores, chatbots

Imagen

Filtros de TikTok, mejora de fotos

Salud

Diagnóstico por imagen médica

Videojuegos

Renderizado acelerado

Refuerzo

Agentes que dominan juegos

Generativa

DALL·E, Stable Diffusion

Robótica

Percepción y control

Funcionan por detrás en todas partes: conviene saber cómo operan.

¿Qué veremos en el curso?

1

Introducción

IA y aprendizaje automático (hoy)

2

Principios

McCulloch y Pitts, perceptrón

3

Red neuronal simple

Activaciones, descenso por gradiente

4

Redes multicapa

Retropropagación

5

Preliminares

Cálculo, álgebra, probabilidad, Python (opcional)

CÓMO TRABAJAMOS CADA TEMA

Libro → **Explicación** → **Ejercicio en papel** → **Notebook en Python**

HERRAMIENTAS

Dos formas de hacer los ejercicios

Rellenas el código con Python y NumPy **“a pie”**, sin un simple .fit().

1 · Google Colab

- Abres el notebook en el navegador.
- **Ventaja:** no hay que instalar nada.
- Ideal como plan B.

2 · En tu computadora

- Anaconda + Git + VS Code.
- **Ventaja:** experiencia real de desarrollo.
- Es la opción recomendada.

Repositorio del curso: **github.com/datahuba/curso-redes-neuronales**

Instalación (1/2): tres programas

1

Anaconda

`anaconda.com/download`

Python y ambientes. Opción “Just Me”; no marcar PATH.

2

Git

`git-scm.com/download/win`

Descargar el código del curso. Todo “Next”.

3

VS Code

`code.visualstudio.com`

Editor. Instalar extensiones Python y Jupyter.

Unos 5 GB libres y permisos de administrador. Anaconda pesa ~900 MB: descárgalo antes.

TODO EN ANACONDA POWERSHELL PROMPT

Instalación (2/2): los comandos

```
# 4. Crear y activar el ambiente
conda create --name nni python=3.11
conda activate nni

# 5. Descargar el curso
cd ~
mkdir projects
cd projects
git clone https://github.com/datahuba/curso-redes-neuronales.git
cd curso-redes-neuronales

# 6. Instalar los paquetes
pip install -r requirements.txt
```

COMPRUEBA

- ✓ La línea empieza con **(nni)**
- ✓ **ls** muestra docs y notebooks
- ✓ Termina con **Successfully installed**

7. VS CODE

Abrir carpeta → curso-redes-neuronales

Abrir un notebook → Select Kernel → nni

¿Qué es la inteligencia artificial?

No hay una definición única aceptada. Russell y Norvig las agrupan en dos enfoques:

Imitar al humano

Máquinas con mente: duplicar lo que hace el cerebro.

Riesgo: también copiamos sus errores.

Actuar racionalmente

Percibir → **razonar** → **actuar** para resolver el problema.

No tiene que funcionar como el cerebro.

Lectura: Russell y Norvig, Inteligencia Artificial: un enfoque moderno, cap. 1

El test de Turing

Definición **operacional**: si una persona que conversa por mensajes no distingue si del otro lado hay una máquina o una persona, la máquina pasa.

QUÉ NECESITA LA MÁQUINA



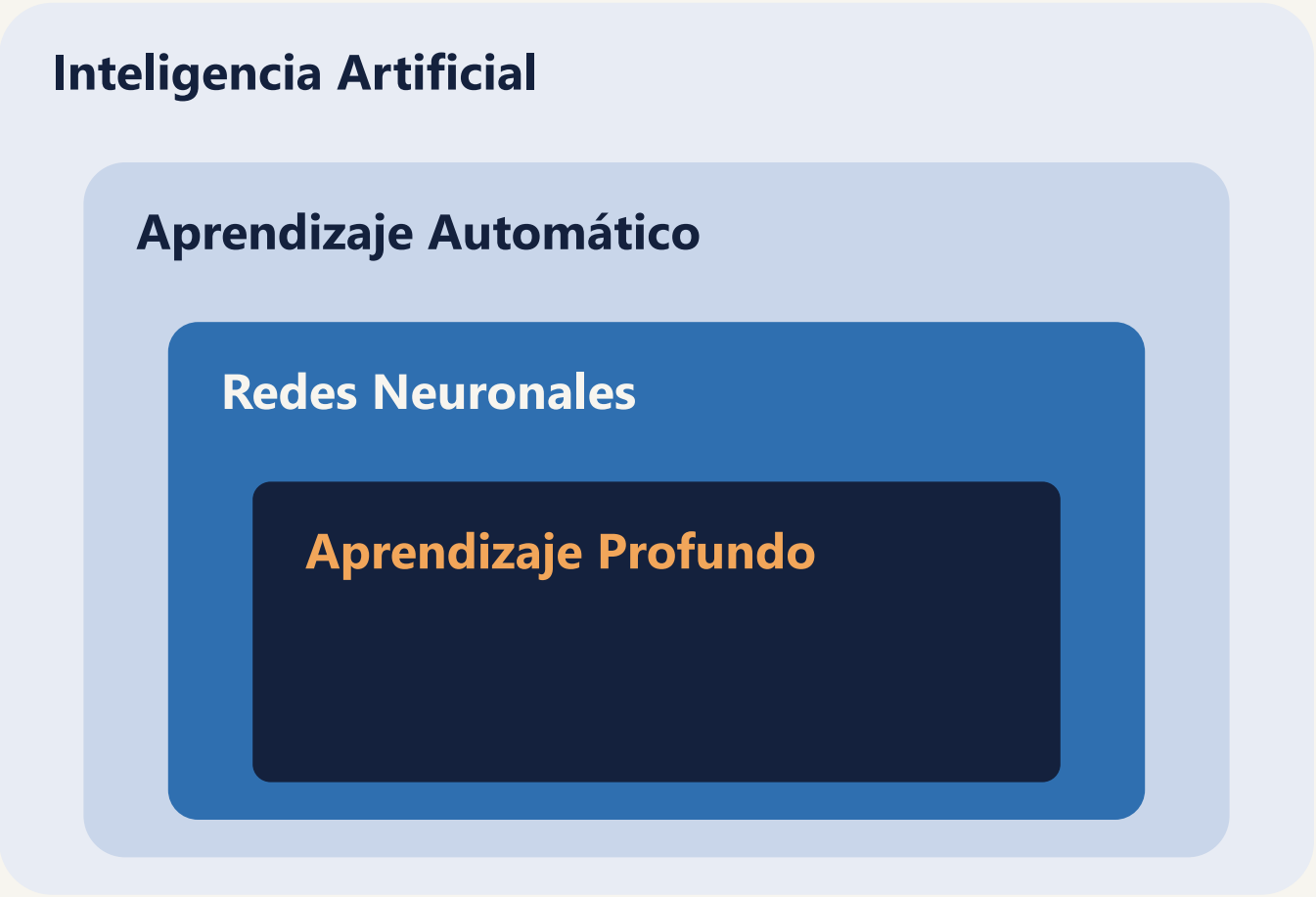
Con el aprendizaje profundo, las redes neuronales ya cubren todas estas partes.

¿Qué tan lejos llegan las redes?

Eje	Qué incluye	Redes neuronales
Operacional	Controlar, jugar, reconocer, generar	**Muy buenas**
Razonamiento	Representar conocimiento, inferir, conciencia	Limitadas
Explicabilidad	Garantizar cuándo funcionan y cuándo no	Limitadas

Herramientas muy efectivas para resolver problemas, sin conciencia ni explicación completa. Por eso nos interesa cómo entrenarlas bien.

Dónde encajan las redes neuronales



IA: tareas que asociamos con inteligencia.

Aprendizaje automático: aprende de datos sin programarse explícitamente.

Redes: unidades simples conectadas.

Deep learning: muchas capas; rasgos cada vez más abstractos.

El aprendizaje automático

> **“Dar a las computadoras la capacidad de aprender sin ser programadas explícitamente.”**
>
> Arthur Samuel, 1959

Tarea

Ganar al ajedrez

Experiencia

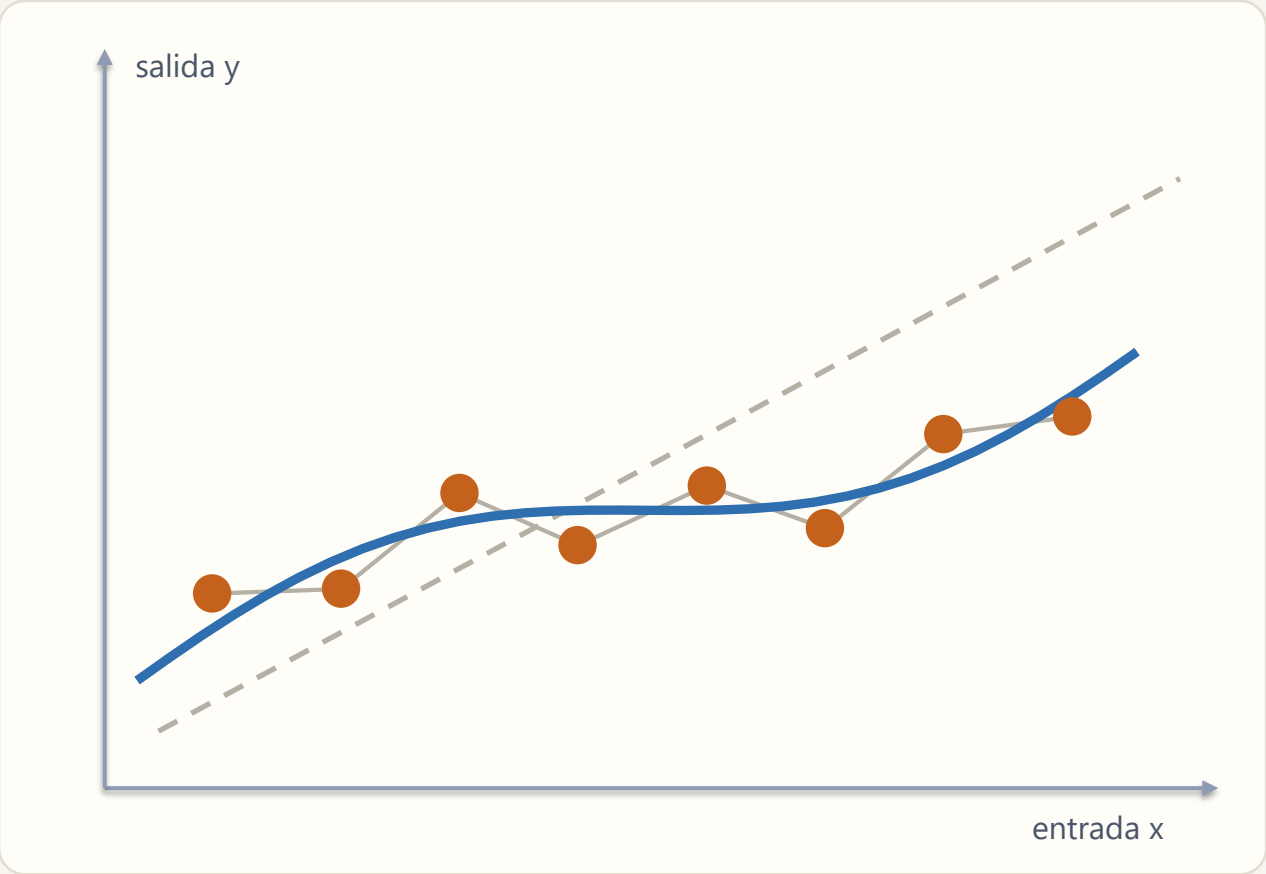
Partidas jugadas, datos

Desempeño

Mejora tras observar: gana más seguido

Es imposible programar a mano qué hacer en cada posición del tablero.

Tarea 1 · Regresión



- datos (experiencia)
- función real, desconocida
- hipótesis candidatas

Hipótesis: una función f que relaciona entrada y salida.

Consistente: explica los datos observados.

Pero queremos que también funcione con datos futuros: **generalización.**

Otras tareas

Clasificación

Asignar una etiqueta de un conjunto predefinido $\{l_1 \ldots l_n\}$. Foto → “perro” o “gato”.

Agrupamiento

Sin etiquetas: encontrar grupos (media y varianza) por orejas, bigotes…

Reducción de dimensionalidad

Menos características. De 2 ejes a 1: un umbral separa las clases.

Modelado generativo

Aprender la distribución y muestrear de ella: rostros, arte, imágenes nuevas.

Tipos de aprendizaje

Tipo	Experiencia	Qué aprende
Supervisado	Pares (x, y) etiquetados: conos, cubos, esferas	La función x → y
No supervisado	Solo ejemplos, sin etiquetas	Las distribuciones que agrupan
Por refuerzo	No está disponible: se obtiene interactuando	Qué acción tomar en cada estado

Entrenamiento con el dataset → **generalización**: acertar en ejemplos nunca vistos, como un auto autónomo en calles nuevas.

Aprendizaje por refuerzo



Corto plazo: comer una pieza.

Largo plazo (horizonte): ganar la partida.

Objetivo: maximizar la recompensa acumulada.

Ejemplo: ajedrez. Mover una ficha (acción) cambia el tablero (estado); comer una pieza o ganar da recompensa.

Quiz · 5 minutos

1. ¿Por qué el test de Turing es una definición operacional?
2. ¿En cuál de los tres ejes destacan las redes neuronales?
3. ¿Qué es generalizar?
4. ¿Diferencia entre clasificación y agrupamiento?
5. ¿Qué maximiza un agente de aprendizaje por refuerzo?

Para la próxima clase

1. Tener el ambiente nni instalado y el repositorio clonado.
2. Leer Russell y Norvig: cap. 1 y secciones 18.1 y 18.2.
3. Leer Goodfellow et al., Deep Learning: sección 5.1.

Siguiente: las redes neuronales por dentro, la neurona de McCulloch y Pitts.

Guía paso a paso — Clase 2: Principios de las redes neuronales

Temas: modelos paramétricos · neurona de McCulloch y Pitts · el perceptrón · regla de aprendizaje · práctica en Python
Duración: ~2 h 30 min
Material: diapositivas `Clase2_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: haber terminado la instalación de la Clase 1 (GUIA_PASO_A_PASO_CLASE1.md, Parte A)
Lecturas base (libro del Dr. Irving Vasquez):

- Modelo de McCulloch y Pitts
- El perceptrón
- Regla de aprendizaje del perceptrón

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Calcular a mano la salida de una neurona de McCulloch y Pitts y de un perceptrón.
2. Aplicar un paso de la regla de aprendizaje de Rosenblatt.
3. Explicar por qué un perceptrón no puede aprender XOR.
4. Programar ambas neuronas en Python.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 1 (5 min)	1–4
0:05	B1. Modelos paramétricos (10 min)	5
0:15	B2. Neurona de McCulloch y Pitts (25 min)	6–9
0:40	C1. Práctica: McCulloch y Pitts (20 min)	10
1:00	Descanso (10 min)	11
1:10	B3. El perceptrón (20 min)	12–15
1:30	B4. Regla de aprendizaje (20 min)	16–18

Tiempo	Tema	Diapositivas
1:50	C2–C5. Práctica: perceptrón (30 min)	19–20
2:20	Cierre, quiz y tarea (10 min)	21

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio

Si ya clonaste el repo en la Clase 1, descarga los cambios nuevos. En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

Verificación: debe decir `Already up to date.` o listar archivos actualizados.

Si **no** lo tenías clonado, sigue la Clase 1, Parte A, Pasos 4 a 6.

A.2 Activar el ambiente y comprobar que todo funciona

En la misma terminal:

```
conda activate nni
```

```
python docs/practica_02.py
```

Verificación: al final debe imprimir `Todas las comprobaciones pasaron.` Si las tildes salen como `�`, ejecuta primero `chcp 65001`; es solo visual.

A.3 Checklist

- [ ] `python docs/practica_02.py` funciona.
- [ ] VS Code abre `notebooks/1_01_mcculloch_pitts.ipynb` con el kernel **nni**.
- [ ] Plan B probado: el mismo notebook abre en Google Colab.
- [ ] En el pizarrón, un plano cartesiano con los 4 puntos (0,0), (0,1), (1,0), (1,1). Lo usarás tres veces.
- [ ] Diapositivas abiertas (`Clase2_Diapositivas.pdf` o `.pptx`).

PARTE B — Teoría desde cero

Repaso de la Clase 1 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Qué es aprendizaje supervisado?" → aprender de pares (entrada x, etiqueta y).
- "¿Qué es generalizar?" → acertar en ejemplos nunca vistos.

Hoy construimos la **primera pieza** de una red neuronal: la neurona artificial.

B1. Modelos paramétricos (10 min) — diapositiva 5

Idea en una frase: un modelo paramétrico es una fórmula con forma fija, cuyo comportamiento cambia según unos números llamados **parámetros**.

$$\hat{y} = \Phi_\theta(X)$$

Símbolo	Se lee	Significa
X	"equis"	La entrada (los datos que le damos)
θ	"theta"	Los parámetros (las perillas que se ajustan)
Φ	"fi"	La fórmula o modelo
ŷ	"y gorro"	La predicción del modelo
y	"y"	La respuesta correcta (etiqueta)

Ejemplo en el pizarrón: la recta ŷ = w·x + b.

- Dibuja tres rectas: (w=1, b=0), (w=2, b=1), (w=−1, b=3).
- La **forma** es siempre una recta; los **parámetros** w (inclinación) y b (desplazamiento) cambian cuál recta es.

Idea central de todo el curso:

> **Aprender = encontrar los parámetros θ que hacen que ŷ se parezca a y.**

Formalmente: |y − ŷ| < ε, con ε (épsilon) muy cercano a cero.

En redes neuronales, los parámetros se llaman **pesos (W)** y **sesgo (b)**.

B2. La neurona de McCulloch y Pitts (25 min) — diapositivas 6–9

Historia (2 min)

- **1943.** Warren McCulloch (neurocientífico) y Walter Pitts (lógico matemático).
- Fue la **primera neurona artificial**: una simplificación muy abstracta de una neurona biológica.
- Propusieron que conectando muchas de estas neuronas se puede calcular **cualquier función lógica**. Fue el primer puente entre el cerebro y la computación.

Cómo funciona (8 min)

La neurona se llama **C** y tiene tres elementos:

Elemento	Símbolo	Valores	Papel
Entradas **excitatorias**	$E = \{e_1, \ldots, e_n\}$	0 o 1	Empujan a la neurona a activarse
Entradas **inhibitorias**	$I = \{\iota_1, \ldots, \iota_m\}$	0 o 1	Si alguna vale 1, apagan la neurona
Umbral	u	0, 1, 2, …	Cuántas excitatorias encendidas hacen falta

La salida es $\hat{y} = C(E, I, u)$, y solo puede valer 0 (apagada) o 1 (activada).

Reglas (escríbelas en el pizarrón en este orden):

1. Si **alguna** entrada inhibitoria vale 1 → **$\hat{y} = 0$**.
2. Si no, suma las excitatorias: si $\Sigma\, e_i \geq u$ → **$\hat{y} = 1$**.
3. En cualquier otro caso → **$\hat{y} = 0$**.

Analogía: un portero de discoteca.

- Las inhibitorias son una "lista negra": si estás en ella, no entras, punto.
- Si no, entras solo si traes al menos *u* invitaciones (excitatorias).

Cómo se dibuja (notación de Minsky):

- Medio círculo = la neurona, con el umbral escrito dentro.
- Flecha = entrada excitatoria.
- Línea terminada en circulito = entrada inhibitoria.
- Línea de salida = el axón.

Ejemplos resueltos (10 min)

Hazlos en el pizarrón **preguntando a la clase cada fila**.

Ejemplo 1 — compuerta NOT: $C(E = \emptyset, I = \{\iota_1\}, u = 0)$. E vacío significa que no hay excitatorias, así que la suma es 0.

ι_1	$\hat{y}$	¿Por qué?
0	1	No hay inhibición y la suma $0 \geq u = 0$
1	0	La inhibitoria está activa

Resultado: la salida es lo contrario de la entrada. Es un **NOT**.

Ejemplo 2: $C(E = \{e_1\}, I = \{\iota_1\}, u = 1)$

e_1	ι_1	$\hat{y}$
0	0	0
0	1	0
1	0	**1**
1	1	0

Solo se activa con $e_1 = 1$ y $\iota_1 = 0$: implementa **"e_1 Y NO ι_1"**.

Ejemplo 3 — AND y OR con dos excitatorias:

e_1	e_2	Suma	AND (u = 2)	OR (u = 1)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	1

Regla general con n excitatorias:

Compuerta	Configuración
AND	u = n (todas encendidas)
OR	u = 1 (al menos una)
"al menos k de n"	u = k
NOT	E = ∅, una inhibitoria, u = 0

Limitaciones (5 min)

Pregunta a la clase primero: "¿Qué le falta a esta neurona?"

1. **Sin pesos:** todas las entradas valen igual. No puede decir "esta entrada importa más".
2. **Solo binaria:** no acepta valores como 3.7 metros o 25 grados.
3. **No aprende:** el umbral lo fijamos nosotros a mano. ← la más importante.

Esto motiva el perceptrón.

Ejercicios en papel (5 min)

1. ¿Qué umbral necesita una neurona con 3 excitatorias para activarse con "al menos 2 de 3"?
2. Escribe la tabla de C(E = {e_1, e_2}, I = ∅, u = 2).
3. Con C(E = {e_1, e_2}, I = {ι_1}, u = 1), ¿qué sale cuando $\iota_1 = 1$?

Respuestas:

1. u = 2.
2. 0, 0, 0, 1 (es AND).
3. Siempre 0.

B3. El perceptrón (20 min) — diapositivas 12–15

Historia (2 min)

- **1958.** Frank Rosenblatt.
- El primer perceptrón fue una **máquina física**: potenciómetros, motores y una cuadrícula de fotoceldas como "ojos". Reconocía figuras simples y **ajustaba sus pesos cuando se equivocaba**.

Analogía biológica

Neurona biológica	Perceptrón
Dendritas (reciben señales)	Entradas X
Núcleo (procesa)	Suma ponderada
Axón (envía la señal)	Salida ŷ

Qué mejora respecto a McCulloch y Pitts

	McCulloch y Pitts	Perceptrón
Entradas	Solo 0 o 1	Cualquier número real
Importancia	Todas iguales	Cada una con su **peso** w_i
Desplazamiento	Umbral u fijo	**Sesgo** b ajustable
Aprende	No	**Sí**

El modelo en dos pasos (8 min)

Paso 1 — combinación lineal: multiplicar cada entrada por su peso, sumar todo y sumar el sesgo.

$$h = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \sum_{i=1}^{n} w_i x_i + b$$

Paso 2 — función de activación escalón: decide si se activa.

$$\hat{y} = f(h) = \begin{cases} 1 & \text{si } h \geq 0 \\ 0 & \text{si } h < 0 \end{cases}$$

Analogía: decidir si salir a correr.

- x_1 = ¿hace sol? (1/0), x_2 = ¿estoy cansado? (1/0).
- Pesos: $w_1 = 2$ (el sol me anima mucho), $w_2 = -3$ (el cansancio me frena más).
- Sesgo $b = -1$ (soy algo perezoso).
- Sol y descansado: $h = 2 \cdot 1 - 3 \cdot 0 - 1 = 1 \geq 0$ → **salgo** ($\hat{y} = 1$).
- Sol y cansado: $h = 2 - 3 - 1 = -2 < 0$ → **no salgo** ($\hat{y} = 0$).

Notas:

- Una variante es la función **signo**, que devuelve −1 en lugar de 0.
- **Truco del sesgo:** agregar una entrada extra $x_0 = 1$ cuyo peso es b. Así $h = W \cdot X$ y el sesgo es "un peso más". Los notebooks lo hacen con `np.insert`.

Ejemplo resuelto del libro (5 min)

W = [0.7, −0.3, 0.4], b = 0, X = [8, 5, 6]

1. $h = 0.7 \cdot 8 + (-0.3) \cdot 5 + 0.4 \cdot 6$
2. $h = 5.6 - 1.5 + 2.4 =$ **6.5**
3. $\hat{y} = f(6.5) =$ **1**, porque $6.5 \geq 0$.

Interpretación geométrica (5 min) — usa el plano del pizarrón

Con dos entradas, la ecuación $w_1x_1 + w_2x_2 + b = 0$ es una **recta** (la **frontera de decisión**):

- de un lado, $h \geq 0 \rightarrow$ clase 1;
- del otro lado, $h < 0 \rightarrow$ clase 0.

Con n entradas, la frontera es un **hiperplano**.

AND: $w_1 = 1$, $w_2 = 1$, $b = -1.5 \rightarrow$ la recta $x_1 + x_2 = 1.5$. Solo el punto (1,1) queda arriba. Compruébalo:

x_1	x_2	$h = x_1 + x_2 - 1.5$	$\hat{y}$
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	0.5	1

OR: $w_1 = 1$, $w_2 = 1$, $b = -0.5$.

XOR — actividad (2 min): XOR vale 1 en (0,1) y (1,0), y 0 en (0,0) y (1,1). "Intenten trazar UNA recta que separe los unos de los ceros." → **Es imposible.** Decimos que XOR **no es linealmente separable**.

B4. Regla de aprendizaje de Rosenblatt (20 min) — diapositivas 16–18

El problema

¿Cómo encuentra el perceptrón **solo** los pesos correctos a partir de ejemplos (x, y)?

El algoritmo

1. **Inicializar:** pesos y sesgo con valores aleatorios pequeños, por ejemplo entre −1 y 1, o en cero.
2. **Para cada ejemplo** (X, y):
    1. Calcular la predicción $\hat{y}$.
    2. Calcular el error: $e = y - \hat{y}$.
    3. Actualizar los pesos y el sesgo:

$$W \leftarrow W + \eta\,(y - \hat{y})\,X \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

3. **Repetir** pasadas por todo el dataset hasta que no haya errores, o hasta un máximo de pasadas.

Vocabulario

Término	Significado
η (eta)	**Tasa de aprendizaje:** qué tan grande es cada corrección
$e = y - \hat{y}$	**Error:** indica la dirección de la corrección
Época	Una pasada completa por todos los ejemplos
Convergencia	Llegar a cero errores

Por qué funciona (intuición)

Caso	Error	Qué hace la regla	Efecto
Acertó	0	Nada	Si funciona, no lo toques
Dijo 0, debía decir 1	+1	Suma η·X a los pesos	h sube la próxima vez
Dijo 1, debía decir 0	−1	Resta η·X a los pesos	h baja la próxima vez

Teorema de convergencia (Novikoff, 1962): si los datos son **linealmente separables**, la regla encuentra una solución en un número **finito** de pasos. Con XOR nunca converge.

Ejemplo resuelto del libro (8 min) — hazlo paso a paso en el pizarrón

Datos: $X = [3, 5, 8]$, $y = 0$, $\eta = 0.5$, $W^0 = [0.3, -0.4, 0.5]$, $b^0 = 0.2$

1. **Predicción:** $h = 0.3 \cdot 3 + (-0.4) \cdot 5 + 0.5 \cdot 8 + 0.2 = 0.9 - 2.0 + 4.0 + 0.2 = 3.1 \rightarrow \hat{y} = 1$.
2. **Error:** $e = 0 - 1 = -1$. Se equivocó, hay que corregir.
3. **Nuevos pesos:** $W^1 = [0.3, -0.4, 0.5] + 0.5 \cdot (-1) \cdot [3, 5, 8] = [0.3 - 1.5, -0.4 - 2.5, 0.5 - 4.0] =$ **$[-1.2, -2.9, -3.5]$**.
4. **Nuevo sesgo:** $b^1 = 0.2 + 0.5 \cdot (-1) =$ **-0.3**.
5. **Comprobar:** $h = -3.6 - 14.5 - 28 - 0.3 = -46.4 < 0 \rightarrow \hat{y} = 0 = y$ ✓.

Ejercicios en papel (10 min)

A. $w_1 = 0.5$, $w_2 = -0.3$, $b = 0.4$, función escalón, $X = [1, 2]$, $y = 0$, $\eta = 0.1$.

- a) ¿Cuánto vale $\hat{y}$?
- b) ¿Cuál es el error?
- c) ¿Cuánto cambian los pesos y el sesgo?
- d) ¿Cuáles son los nuevos valores?

B. Se ejecutan 50 épocas sobre un dataset de 27 ejemplos. ¿Cuántas veces en total se presenta un ejemplo? ¿Cuántas de esas son repeticiones?

C. Encuentra a mano pesos y sesgo para NAND, que vale 0 solo en (1,1).

Respuestas:

- **A:**
  - a) $h = 0.5 - 0.6 + 0.4 = 0.3 \geq 0 \rightarrow \hat{y} = 1$.
  - b) $e = -1$.
  - c) $\Delta W = [-0.1, -0.2]$, $\Delta b = -0.1$.
  - d) $W = [0.4, -0.5]$, $b = 0.3$.
- **B:** $50 \times 27 = 1350$ presentaciones; $1350 - 27 = 1323$ repeticiones.
- **C:** $w = [-1, -1]$, $b = 1.5$ (es AND con los signos invertidos).

PARTE C — Práctica en la computadora

Preparación (todos):

1. Abre VS Code → **File** → **Open Folder** → `C:\Users\TuUsuario\projects\curso-redes-neuronales`.
2. Para cada notebook, revisa arriba a la derecha que el kernel sea **nni** (Select Kernel → Python Environments → nni).
3. En los notebooks, las partes a completar están marcadas con `# TODO`. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**.

C1. Notebook `notebooks/1_01_mcculloch_pitts.ipynb` (20 min, 3 puntos) — diapositiva 10

Qué hay que hacer:

1. **TODO 1 (2 pts):** completar la función `neuronaMyP(E, I, u)` **sin usar funciones de NumPy**, solo `for`, `if` y sumas.
2. Ejecutar la celda de prueba: con `E = [1]`, `I = [0]`, `u = 1` la salida debe ser **1**.
3. **TODO 2 (1 pt):** programa que reciba E, I y u cualesquiera y devuelva la salida.

Pistas para los alumnos (no la solución):

- Traduce las 3 reglas de B2 a código, **en el mismo orden**.
- Primero recorre `I`: si encuentras un 1, devuelve 0 de inmediato.
- Luego suma los elementos de `E` con un `for`.
- Compara la suma con `u`.

Cómo verificar (que lo prueben ellos): las tablas de los Ejemplos 1, 2 y 3 de B2 deben salir iguales.

- `neuronaMyP([], [0], 0)` → 1
- `neuronaMyP([], [1], 0)` → 0
- `neuronaMyP([1, 1], [], 2)` → 1
- `neuronaMyP([1, 0], [], 2)` → 0

Error típico: con E vacío la suma es 0, y con u = 0 la neurona **sí** se activa.

C2. Demo del docente: `docs/practica_02.py` (5 min) — diapositiva 19

Proyecta y ejecuta en Anaconda PowerShell (con `(nni)` activo, dentro del repo):

```
python docs/practica_02.py
```

Qué mostrar en la salida:

- McCulloch-Pitts reproduce AND, OR y NOT.
- El perceptrón **aprende** AND y OR en 4 épocas.
- Con XOR, `convergió=False` después de 100 épocas: la demostración en vivo de que no es linealmente separable.

Abre el archivo y señala las tres funciones: `predecir` (los pasos 1 y 2 de B3), `entrenar` (la regla de B4) y `escalon`.

C3. Notebook `notebooks/1_02_perceptron_AND.ipynb` (5 min, práctica) — diapositiva 20

1. Cambia `weight1`, `weight2` y `bias` **a mano**.
2. Ejecuta la celda de comprobación.
3. Repite hasta que las 4 filas digan `Yes`.

Pista: usa la interpretación geométrica de B3.

C4. Notebook `notebooks/1_03_1_perceptron_simple.ipynb` (10 min, 3 puntos)

1. **TODO (1 pt):** `combinacion_lineal(W, X, b)` **sin** `np.dot`: un `for` que acumule `w * x` y al final sume `b`.
2. **TODO (1 pt):** `escalon(h)`: debe devolver 1 cuando h ≥ 0 y 0 si no. Debe funcionar con un número o con un arreglo. Pista: `(h >= 0)` convertido a entero.
3. Ejecuta la pasada frontal de ejemplo: con `inputs = [0.7, -0.3]`, `weights = [0.1, 0.8]`, `bias = -0.1` → h = −0.27 → salida **0**.
4. **TODO (1 pt):** encontrar por prueba y error pesos para **OR**.

C5. Notebook `notebooks/1_03_2_regla_de_aprendizaje.ipynb` (tarea o tiempo extra, 4 puntos)

1. **Inicialización (1 pt):** pesos con `np.random.normal(loc=0, scale=1, size=(2,))` y sesgo en 0.
2. **Pérdida SAE (1 pt):** suma de los valores absolutos de `y_true - y_pred`. Mide cuántos ejemplos falla.
3. **Regla de aprendizaje (1 pt):** dentro del `for`, aplicar las fórmulas de B4 a `perceptron.W` y `perceptron.b`.
4. **Hiperparámetros (2 pts, según el notebook):** con 3 épocas suele quedar incompleto. Probar `n_epocas` entre 20 y 50 y `tasa_aprendizaje` entre 0.01 y 0.5. Observar:
   - la gráfica de pérdida, que debe bajar;
   - la frontera de decisión, que debe separar la mayoría de puntos rojos y azules.

> Las soluciones de los `TODO` están en la clave del docente, que queda solo en la computadora del profesor y no está en el repositorio.

Cierre (10 min) — diapositiva 21

Resumen en tres líneas:

1. Neurona artificial = suma ponderada + función de activación.
2. El perceptrón aprende corrigiendo sus pesos con el error: $W \leftarrow W + \eta(y - \hat{y})X$.
3. Solo puede separar clases con una recta: problemas **linealmente separables**.

Cliffhanger: "XOR no se puede con una sola neurona. Minsky y Papert lo demostraron en 1969 en su libro *Perceptrons*, y la investigación casi se detuvo por años. ¿La solución? Conectar neuronas en **capas**. Lo veremos en las próximas clases."

Quiz de salida (en papel, 3 min):

Pregunta	Respuesta
¿Qué tiene el perceptrón que no tiene McCulloch-Pitts?	Pesos, entradas reales y aprendizaje
¿Qué hace una entrada inhibitoria activa?	Fuerza la salida a 0
Si ŷ = y, ¿cuánto cambian los pesos?	Nada (el error es 0)
¿Qué es una época?	Una pasada completa por el dataset
¿Por qué el perceptrón no aprende XOR?	No es linealmente separable

Tarea:

1. Terminar los notebooks `1_01`, `1_03_1` y `1_03_2` (10 puntos en total).
2. En `docs/practica_02.py`:
   - cambia `eta` a 0.01 y a 1: ¿cambia el número de épocas para AND?
   - dibuja en papel la recta aprendida para OR junto a los 4 puntos.
3. Leer del libro: Neurona artificial moderna, preparación de la Clase 3.

Rúbrica:

Notebook	Puntos
1_01 McCulloch-Pitts	3
1_03_1 Perceptrón simple	3
1_03_2 Regla de aprendizaje	4
1_02 Perceptrón AND	Práctica, sin puntos

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`NameError: name 'np' is not defined`	No se ejecutó la celda de `import`	Ejecuta las celdas desde la primera, en orden
`ModuleNotFoundError: No module named 'nni'` (en `1_03_2`)	El notebook no se abrió desde la carpeta del repo	Abre en VS Code la carpeta `curso-redes-neuronales` completa; en Colab, clona el repo y haz `%cd curso-redes-neuronales/notebooks`
`ModuleNotFoundError: No module named 'sklearn'`	Falta scikit-learn	Con `(nni)` activo: `pip install scikit-learn`
La función devuelve `None`	Falta el `return`	Revisa que todas las ramas terminen en `return`
El perceptrón no mejora	Muy pocas épocas o η muy pequeña	Sube `n_epocas` a 20–50

Síntoma	Causa	Solución
La pérdida sube y baja sin parar	Los datos no son separables, o η es muy grande	Baja η; acepta "mayoría bien clasificada"
`ValueError: shapes not aligned`	W y X de distinto tamaño	Imprime `W.shape` y `X.shape`; deben coincidir
El kernel no aparece como `nni`	Falta `ipykernel`	`pip install ipykernel` y reinicia VS Code
Tildes como `?` en la terminal	Codificación de Windows	`chcp 65001`

CLASE 2 · MÓDULO 2

Principios de las redes neuronales

De la neurona de McCulloch y Pitts al perceptrón

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 1
0:05	Modelos paramétricos
0:15	Neurona de McCulloch y Pitts
0:40	Práctica: McCulloch y Pitts
1:00	Descanso
1:10	El perceptrón
1:30	Regla de aprendizaje
1:50	Práctica: perceptrón en Python
2:20	Cierre, quiz y tarea

Al terminar la clase podrás…

1

Calcular a mano

la salida de una neurona de McCulloch y Pitts y de un perceptrón.

2

Aplicar la regla

de aprendizaje de Rosenblatt, paso a paso.

3

Explicar XOR

por qué un perceptrón no puede aprenderlo.

4

Programar

ambas neuronas en Python.

Repaso de la Clase 1

¿Aprendizaje supervisado?

Aprender de pares (entrada x, etiqueta y).

¿Generalizar?

Acertar en ejemplos que nunca vio.

Hoy: la primera pieza de una red neuronal, la neurona artificial.

Modelos paramétricos

$$\hat{y} = \phi_\theta(X)$$

Símbolo	Significa
X	La entrada (los datos)
θ	Los parámetros: las perillas
$\hat{y}$	La predicción del modelo
y	La respuesta correcta

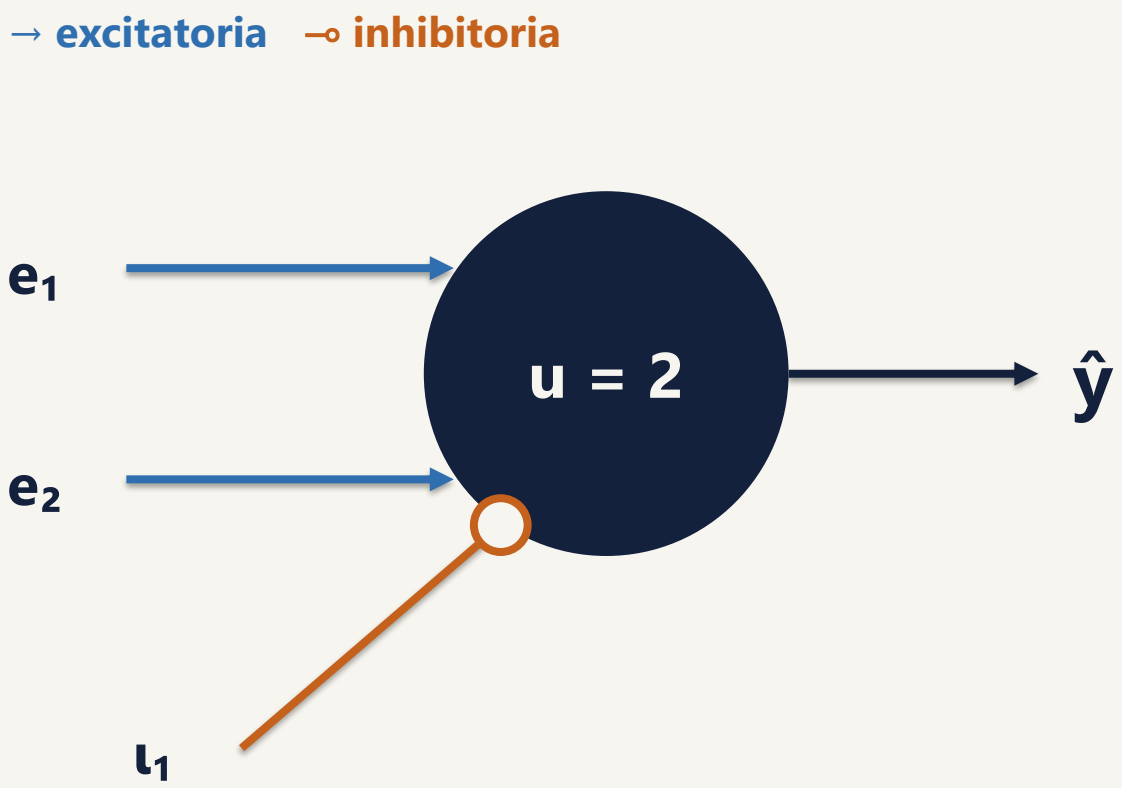
IDEA CENTRAL DEL CURSO

Aprender = encontrar θ tal que $\hat{y} \approx y$

Ejemplo: la recta $\hat{y} = w \cdot x + b$

En redes: θ = pesos W y sesgo b

Neurona de McCulloch y Pitts (1943)



Elemento	Valores
E: excitatorias	0 o 1
I: inhibitorias	0 o 1
u: umbral	0, 1, 2, …
ŷ: salida	0 o 1

La primera neurona artificial: una simplificación de la neurona biológica.

Cómo decide la neurona

1. **Alguna inhibitoria vale 1** → $\hat{y} = 0$
2. **Si no, $\Sigma e_i \geq u$** → $\hat{y} = 1$
3. **En cualquier otro caso** → $\hat{y} = 0$

Las reglas se aplican en este orden.

ANALOGÍA

Un portero de discoteca

Inhibitorias = lista negra: si estás, no entras.

Si no, entras con al menos u invitaciones (excitatorias).

Compuertas lógicas con McCulloch y Pitts

e_1	e_2	Σ	AND (u=2)	OR (u=1)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	1

Compuerta	Configuración
AND	u = n
OR	u = 1
≥ k de n	u = k
NOT	E = ∅, 1 inhibitoria, u = 0

NOT: sin excitatorias la suma es 0 ≥ u = 0, así que se activa… salvo que la inhibitoria valga 1.

Limitaciones y ejercicios

Le falta…

- **Pesos:** todas las entradas valen igual.
- **Valores reales:** solo acepta 0 o 1.
- **Aprender:** el umbral se fija a mano.

En papel · 5 min

1. Umbral para “al menos 2 de 3”.
2. Tabla de $C(E=\{e_1,e_2\}, I=\emptyset, u=2)$.
3. $C(E=\{e_1,e_2\}, I=\{\iota_1\}, u=1)$ con $\iota_1 = 1$.

La limitación más importante: no aprende. Eso lo resuelve el perceptrón →

20 MIN · 3 PUNTOS

Práctica · notebooks/1_01_mcculloch_pitts

TODO 1 · 2 pts neuronaMyP(E, I, u) sin NumPy

TODO 2 · 1 pt leer E, I y u arbitrarios

Traduce las 3 reglas al código, en el mismo orden.

```
# comprobaciones
neuronaMyP([], [0], 0)     # 1
neuronaMyP([], [1], 0)     # 0
neuronaMyP([1, 1], [], 2)  # 1
neuronaMyP([1, 0], [], 2)  # 0
neuronaMyP([1], [0], 1)    # 1
```

Sin Python: abre el notebook en Google Colab.

Descanso · 10 min

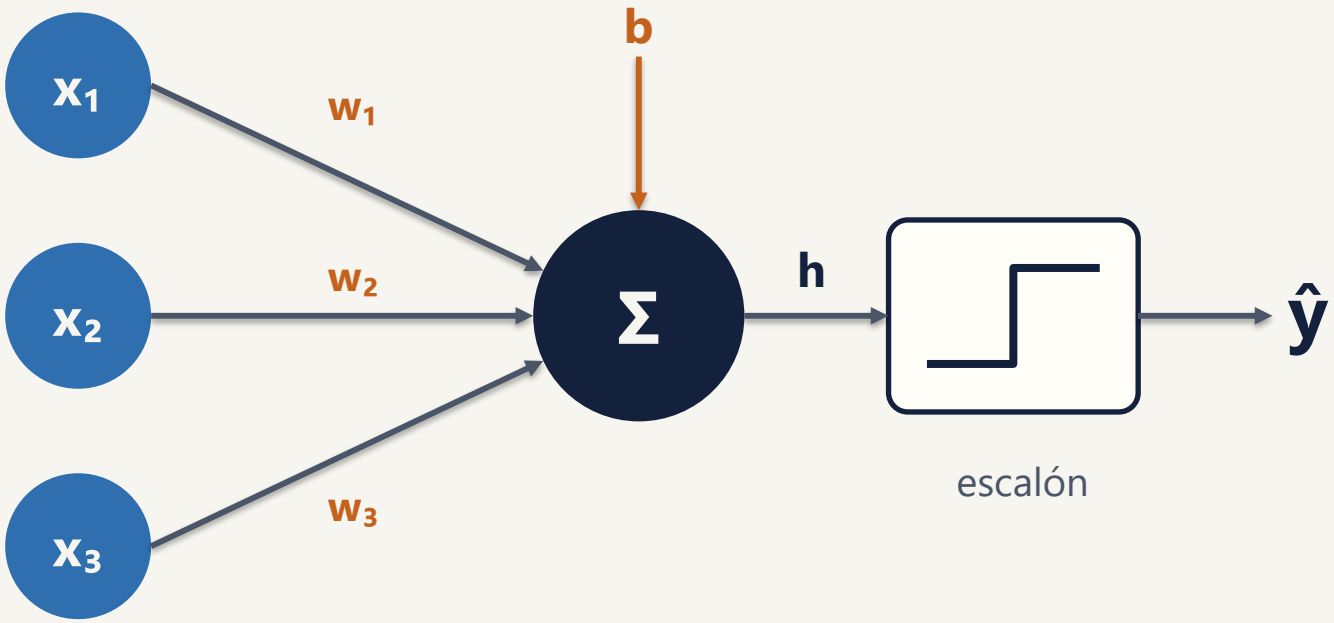
Después: el perceptrón

El perceptrón (Rosenblatt, 1958)

	McCulloch y Pitts	Perceptrón
Entradas	Solo 0 o 1	Números reales
Importancia	Todas iguales	Un peso w_i por entrada
Desplazamiento	Umbral u fijo	Sesgo b ajustable
¿Aprende?	No	**Sí**

El primero fue una máquina física con fotoceldas como “ojos” que ajustaba sus pesos al equivocarse.

El modelo en dos pasos



Truco: una entrada extra $x_0 = 1$ con peso b; así el sesgo es "un peso más".

1 · Combinación lineal

$$h = \Sigma\, w_i \cdot x_i + b$$

2 · Activación escalón

$$\hat{y} = 1 \text{ si } h \geq 0$$

$$\hat{y} = 0 \text{ si } h < 0$$

Ejemplos: calcular la salida

Del libro

$W = [0.7, -0.3, 0.4] \cdot b = 0$

$X = [8, 5, 6]$

$h = 0.7 \cdot 8 - 0.3 \cdot 5 + 0.4 \cdot 6$

$= 5.6 - 1.5 + 2.4 = 6.5$

$\hat{y} = f(6.5) = 1$

¿Salgo a correr?

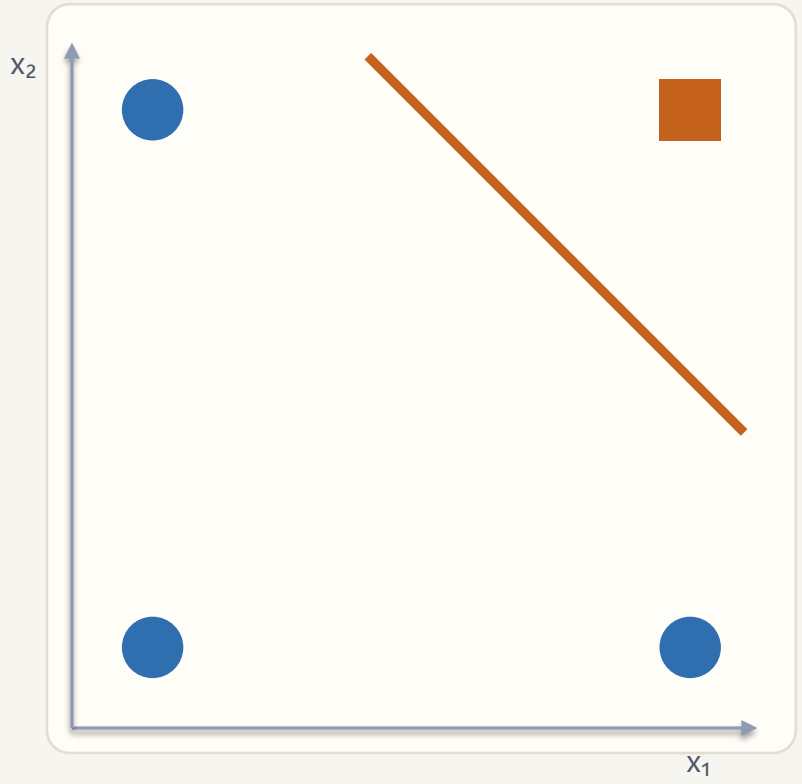
x_1 = ¿hace sol? · x_2 = ¿estoy cansado?

$w = [2, -3] \cdot b = -1$

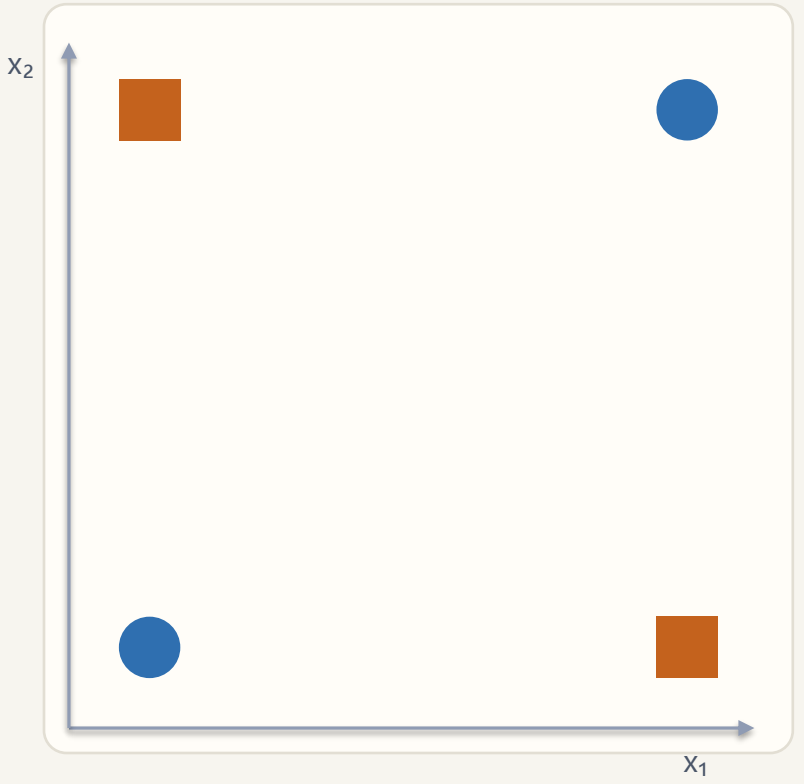
Sol y descansado: $h = 1 \rightarrow$ **salgo**

Sol y cansado: $h = -2 \rightarrow$ **no salgo**

El perceptrón traza una recta



AND: separable



XOR: ¿una recta?

■ clase 1 ● clase 0

Frontera de decisión

$w_1x_1 + w_2x_2 + b = 0$

AND: w = [1, 1], b = −1.5

OR: w = [1, 1], b = −0.5

XOR: ninguna recta separa los cuadrados de los círculos.

→ no es linealmente separable.

Regla de aprendizaje de Rosenblatt

$$W \leftarrow W + \eta\,(y - \hat{y})\,X$$
$$b \leftarrow b + \eta\,(y - \hat{y})$$

Caso	Error $y - \hat{y}$	Efecto
Acertó	0	No cambia nada
Dijo 0, debía decir 1	+1	Suma η·X: h sube
Dijo 1, debía decir 0	−1	Resta η·X: h baja

η = tasa de aprendizaje · **época** = una pasada por todo el dataset · converge si los datos son linealmente separables

Ejemplo: un paso de aprendizaje

$X = [3, 5, 8] \cdot y = 0 \cdot \eta = 0.5 \cdot W^0 = [0.3, -0.4, 0.5] \cdot b^0 = 0.2$

#	Cálculo
1	$h = 0.9 - 2.0 + 4.0 + 0.2 = 3.1 \rightarrow \hat{y} = 1$
2	error $= 0 - 1 = -1$ (se equivocó)
3	$W^1 = W^0 - 0.5 \cdot [3, 5, 8] = [-1.2, -2.9, -3.5]$
4	$b^1 = 0.2 - 0.5 = -0.3$
5	Comprobar: $h = -46.4 < 0 \rightarrow \hat{y} = 0 = y$ ✓

Ejercicios en papel · 10 min

A

$w = [0.5, -0.3]$

$b = 0.4 \cdot \eta = 0.1$

$X = [1, 2] \cdot y = 0$

¿ŷ? ¿error? ¿nuevos pesos?

B

50 épocas sobre 27 ejemplos.

¿Cuántas presentaciones son repeticiones?

C

NAND vale 0 solo en (1, 1).

Encuentra pesos y sesgo.

DOCS/PRACTICA_02.PY

El perceptrón en Python

```
def predecir(x, w, b):
    h = sum(wi * xi for wi, xi in zip(w, x)) + b
    return 1 if h >= 0 else 0

def entrenar(X, y, eta=0.1, epocas=100):
    w, b = [0.0] * len(X[0]), 0.0
    for _ in range(epocas):
        for xi, yi in zip(X, y):
            error = yi - predecir(xi, w, b)
            w = [wj + eta*error*xj for wj, xj in zip(w, xi)]
            b += eta * error
    return w, b
```

EJECUTA

python docs/practica_02.py

AND, OR: aprende en 4 épocas

XOR: convergió=False tras 100 épocas

30 MIN

Práctica · notebooks del perceptrón

Notebook	Qué hacer	Pts
1_02_perceptron_AND	Ajustar pesos a mano para AND	—
1_03_1_perceptron_simple	Combinación lineal sin np.dot, escalón, pesos de OR	3
1_03_2_regla_de_aprendizaje	Inicializar, pérdida SAE, regla, hiperparámetros	4

Comprueba 1_03_1: inputs [0.7, −0.3], weights [0.1, 0.8], bias −0.1 → h = −0.27 → salida 0

En 1_03_2: sube las épocas a 20–50 y mira cómo baja la pérdida.

Para llevar

1. Neurona = suma ponderada + activación.
2. El perceptrón aprende corrigiendo pesos con el error.
3. Solo separa clases con una recta.

¿Y XOR? Conectando neuronas en capas. Próximas clases.

Tarea: notebooks 1_01, 1_03_1 y 1_03_2 (10 pts)

Leer: libro, “Neurona artificial moderna”

Guía del notebook `1_01_mcculloch_pitts`: la neurona de McCulloch y Pitts

Puntos: 3 · **Tiempo:** 20–30 min · **Clase:** 2 · **Archivo:** `notebooks/1_01_mcculloch_pitts.ipynb`
Teoría completa: Guía de la Clase 2, sección B2.

¿Qué vas a aprender?

- Cómo decide la primera neurona artificial de la historia (1943).
- A traducir unas reglas escritas en palabras a una función de Python con `for` e `if`.

Antes de empezar

1. Abre **Anaconda PowerShell Prompt** y escribe `conda activate nni`.
2. Abre VS Code → **File** → **Open Folder** → `curso-redes-neuronales` → carpeta `notebooks` → `1_01_mcculloch_pitts.ipynb`.
3. Arriba a la derecha, elige el kernel **nni**.
4. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**. Las partes que tú completas dicen `TODO`.

> **Sin instalar nada:** en Google Colab, **Archivo** → **Abrir cuaderno** → **GitHub** → `datahuba/curso-redes-neuronales` → este notebook.

La idea en 2 minutos

La neurona se escribe **C(E, I, u)**:

- **E**: lista de entradas **excitatorias** (cada una vale 0 o 1). Empujan a la neurona a encenderse.
- **I**: lista de entradas **inhibitorias** (0 o 1). Si alguna vale 1, la neurona se apaga.
- **u**: el **umbral**: cuántas excitatorias encendidas hacen falta.

Reglas, **en este orden**:

1. Si **alguna** inhibitoria vale 1 → la salida es **0**.
2. Si no, suma las excitatorias: si la suma es **mayor o igual que u** → la salida es **1**.
3. En otro caso → **0**.

Analogía: un portero de discoteca. Las inhibitorias son la "lista negra": si estás ahí, no entras. Si no, entras solo si traes al menos *u* invitaciones (excitatorias).

Paso a paso

Celda "importamos paquetes"

Solo carga NumPy. Ejecútala.

Celda `TODO (2 puntos)`: la función `neuronaMyP(E, I, u)`

Reemplaza `pass` por tu código. **No uses funciones de NumPy**: solo `for`, `if` y sumas.

Pistas:

1. Recorre la lista `I` con un `for`. Si encuentras un 1, haz `return 0` de inmediato.
2. Crea una variable `suma = 0` y súmale cada elemento de `E` con otro `for`.
3. Al final, compara: si `suma >= u` devuelve 1; si no, 0.

Celda "suponga"

Ya trae `E = [1]`, `I = [0]` y `u = 1`. Agrega una línea que imprima `neuronaMyP(E, I, u)`.
✅ **Comprobación:** debe imprimir **1**: no hay inhibición y la suma (1) alcanza el umbral (1).

Celda `TODO (1 punto)`: un programa para cualquier E, I y u

Escribe código que pruebe la neurona con varios casos. Por ejemplo, una función `evaluar(E, I, u)` que imprima la entrada y la salida, y llámala con distintos valores.

✅ **Comprobación** (todos deben coincidir):

Llamada	Qué representa	Salida
`neuronaMyP([], [0], 0)`	NOT con entrada 0	1
`neuronaMyP([], [1], 0)`	NOT con entrada 1	0
`neuronaMyP([1, 1], [], 2)`	AND de (1, 1)	1
`neuronaMyP([1, 0], [], 2)`	AND de (1, 0)	0
`neuronaMyP([0, 1], [], 1)`	OR de (0, 1)	1
`neuronaMyP([1, 1, 0], [], 2)`	"al menos 2 de 3"	1

Errores frecuentes

Síntoma	Causa	Solución
Imprime `None`	Falta un `return` en algún camino	Revisa que la función siempre devuelva 0 o 1
`neuronaMyP([], [0], 0)` da 0	Olvidaste que con E vacío la suma es 0, y 0 ≥ 0	Con `>=` sale 1; revisa que no uses `>`
Una inhibitoria activa no apaga la neurona	Revisas I después de sumar, o no haces `return`	Revisa I **primero** y sal con `return 0`
`NameError: neuronaMyP is not defined`	No ejecutaste la celda de la función	Ejecútala antes de las de prueba

Guía del notebook `1_02_perceptron_AND`: pesos a mano para AND

Puntos: práctica, sin puntos · **Tiempo:** 5–10 min · **Clase:** 2 · **Archivo:** `notebooks/1_02_perceptron_AND.ipynb`
Teoría completa: Guía de la Clase 2, sección B3.

¿Qué vas a aprender?

- Que un perceptrón es una **recta** que separa dos grupos de puntos.
- A elegir pesos y sesgo **a mano** para que el perceptrón calcule la compuerta AND.

Antes de empezar

1. `conda activate nni` en Anaconda PowerShell Prompt.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_02_perceptron_AND.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El perceptrón calcula:

1. **Combinación lineal:** h = peso1·x_1 + peso2·x_2 + sesgo
2. **Activación escalón:** salida = 1 si h ≥ 0, y 0 si h < 0.

AND vale 1 **solo** cuando las dos entradas son 1:

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Necesitas que h sea **negativa** en los tres primeros casos y **positiva o cero** en el último.

Paso a paso

Celda `TODO`: `weight1`, `weight2` y `bias`

Cambia los tres valores (que empiezan en 0).

Pistas para razonar:

- Con los dos pesos iguales y positivos, el caso (1, 1) da la suma más grande.

- El sesgo debe ser negativo: lo bastante para "apagar" los casos con un solo 1, pero no tanto como para apagar el caso (1, 1).
- Prueba pesos de 1 y busca el sesgo entre −1 y −2.

Celda "ESTO DEBE QUEDAR IGUAL"

No la modifiques. Prueba tus pesos con las 4 entradas y te dice cuántas acertaste.

✅ **Comprobación:** debe decir `Bien! Acertaste todas.` y la columna `Is Correct` debe decir `Yes` en las cuatro filas. Por ejemplo, una solución válida da combinaciones lineales −1.5, −0.5, −0.5 y 0.5.

> Hay **muchas** soluciones correctas: cualquier recta que deje solo el punto (1, 1) del lado positivo.

Para pensar

- ¿Qué cambiarías para que calcule **OR** (1 si al menos una entrada es 1)? → Solo el sesgo, menos negativo.
- ¿Y NAND (lo contrario de AND)? → Todos los signos al revés.

Errores frecuentes

Síntoma	Causa	Solución
`Obtuvieste 1 mal` en la fila (1, 1)	El sesgo es demasiado negativo	Súbelo un poco (hazlo menos negativo)
Fallan las filas (0, 1) y (1, 0)	El sesgo no es lo bastante negativo	Bájalo
`NameError: weight1`	No ejecutaste la celda de los pesos	Ejecútala antes que la de prueba

Guía del notebook `1_03_1_perceptron_simple`: el perceptrón en Python

Puntos: 3 · **Tiempo:** 15–20 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_1_perceptron_simple.ipynb`
Teoría completa: Guía de la Clase 2, sección B3.

¿Qué vas a aprender?

- A programar las dos piezas del perceptrón: la **combinación lineal** y la **función escalón**.
- A unirlas para hacer una predicción ("pase frontal").

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_1_perceptron_simple.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Combinación lineal:** $h = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$. Multiplica cada entrada por su peso, suma todo y suma el sesgo.
- **Escalón:** devuelve 1 si $h \geq 0$, y 0 si $h < 0$.
- **Perceptrón:** salida = escalón(combinación lineal).

Paso a paso

Celda "Cargamos paquetes"

Carga NumPy. Ejecútala.

Celda `TODO (1 punto)` : `combinacion_lineal(W, X, b)`

Sin usar `np.dot`. Pistas:

1. Empieza con `suma = 0.0`.
2. Recorre los pares (peso, entrada) a la vez con `for w, x in zip(W, X):` y acumula `w * x`.
3. Al final devuelve la suma más `b`.

Celda `TODO (1 punto)` : `escalon(h)`

Debe devolver 1 si $h \geq 0$ y 0 si no, y funcionar tanto con un número como con un arreglo de NumPy.

Pista: la comparación `h >= 0` da `True` / `False`. Conviértela a entero, por ejemplo con `np.asarray(h >= 0).astype(int)`.

Celda "Perceptrón simple"

Ya está hecha: une tus dos funciones. Ejecútala.

Celda "Definamos unos pesos y sesgo"

Usa entradas [0.7, −0.3], pesos [0.1, 0.8] y sesgo −0.1.

✅ **Comprobación:** imprime `Output:` y luego **0**. La cuenta es h = 0.07 − 0.24 − 0.1 = −0.27, que es menor que 0.

Celda `TODO (1 punto)`: pesos para OR

Busca por prueba y error pesos y sesgo que den **OR**: 0 en (0, 0) y 1 en los otros tres casos. Pruébalos con las 4 entradas usando tu función `perceptron`.

✅ **Comprobación:** una solución válida (pesos [1, 1], sesgo −0.5) imprime:

```
(0, 0) -> 0
(0, 1) -> 1
(1, 0) -> 1
(1, 1) -> 1
```

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported operand ... NoneType`	`combinacion_lineal` todavía devuelve `None`	Completa la función y vuelve a ejecutar esa celda
La salida es `True` / `False` en vez de 1/0	No convertiste a entero	Usa `.astype(int)`
Da 1 en lugar de 0 en el ejemplo	Olvidaste sumar el sesgo, o usaste `>` al revés	Revisa `+ b` y la condición `>= 0`

Guía del notebook `1_03_2_regla_de_aprendizaje`: el perceptrón aprende

Puntos: 4 · **Tiempo:** 25–35 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_2_regla_de_aprendizaje.ipynb`
Teoría completa: Guía de la Clase 2, sección B4.

¿Qué vas a aprender?

- Que el perceptrón puede **aprender solo** sus pesos a partir de ejemplos.
- A medir qué tanto se equivoca (la **pérdida**) y a ver cómo baja con el entrenamiento.
- A dibujar la **frontera de decisión** (la recta que separa las clases).

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_2_regla_de_aprendizaje.ipynb` → kernel **nni**.
   Usa la carpeta del repo, porque el notebook importa el paquete `nni` que está ahí.
3. Ejecuta las celdas en orden con **Shift + Enter**.

> **En Colab:** primero ejecuta en una celda nueva `!git clone https://github.com/datahuba/curso-redes-neuronales` y `%cd curso-redes-neuronales/notebooks`.

La idea en 2 minutos

Regla de Rosenblatt, aplicada a cada ejemplo:

$$W \leftarrow W + \eta \cdot (y - \hat{y}) \cdot X \quad y \quad b \leftarrow b + \eta \cdot (y - \hat{y})$$

- **error = $y - \hat{y}$**: 0 si acertó, +1 o −1 si se equivocó.
- **η** (tasa de aprendizaje): el tamaño de la corrección.
- **Época:** una pasada por todos los ejemplos.
- **SAE** (suma de errores absolutos): cuántos ejemplos se equivoca en total. Con etiquetas 0/1, es exactamente el número de errores.

Paso a paso

Celdas 1–2: datos sintéticos

Cargan las librerías y generan **400 puntos** de dos clases (rojos = 0, azules = 1), que se dibujan.

Celdas 3–6: la clase `Perceptron`

Ya está hecha. Empieza con pesos en cero. Ejecútalas.

Celda `TODO`: inicialización (1 punto)

Asigna pesos aleatorios a `clasificador.W` (dos números) y un sesgo a `clasificador.b`.

- Pista: `np.random.normal(loc=0, scale=1, size=(2,))` genera dos números al azar alrededor de 0.
- El sesgo puede empezar en 0.0.
- Para obtener siempre los mismos números (y poder compararlos), escribe antes `np.random.seed(0)`.

✅ **Comprobación:** con `np.random.seed(0)` los pesos son ≈ **[1.764, 0.400]**.

Celda `TODO: perdidaSAE` (1 punto)

Suma de los valores absolutos de (y_true − y_pred).

- Pistas: `np.abs` y `np.sum`.

✅ **Comprobación:** con la semilla 0, la pérdida inicial es **93** (se equivoca en 93 de 400 puntos). Con otra semilla sale otro número; lo importante es que sea un entero entre 0 y 400.

Celda: gráfica del clasificador sin entrenar

Dibuja la frontera con pesos al azar. Normalmente separa mal los colores.

> ⚠️ Con scikit-learn 1.9 o más nuevo, esta gráfica exige el argumento `n_classes=2`. La copia del repositorio ya lo trae. Si usas una versión vieja y sale `unexpected keyword argument 'n_classes'`, bórralo.

Celda: `ReglaDeAprendizaje` (1 punto)

Completa las dos líneas `None` con la regla de arriba:

- los pesos nuevos son los pesos actuales más tasa · error · `xi`;
- el sesgo nuevo es el sesgo actual más tasa · error.

Celdas de entrenamiento

`n_epocas = 3` suele quedarse corto. La gráfica muestra la pérdida por época.

Celda `TODO (2 puntos)`: ajustar hiperparámetros

Sube `n_epocas` (por ejemplo a 20–50) y prueba `tasa_aprendizaje` entre 0.01 y 0.5. Después vuelve a ejecutar las celdas de entrenamiento y de la gráfica final.

✅ **Comprobación** (semilla 0, tasa 0.1, 20 épocas):

- la pérdida baja de **93** a unos **37**;
- tras la última actualización, el notebook queda en **34 errores de 400**;
- la frontera separa la mayoría de los puntos rojos y azules.

No tiene que ser perfecto: los datos se mezclan un poco y una recta no puede separarlos al 100 %.

Para pensar

- ¿Por qué la pérdida a veces **sube** de una época a otra? → La regla corrige ejemplo por ejemplo; arreglar uno puede descomponer otro.
- ¿Llegará alguna vez a 0? → Solo si los datos son linealmente separables.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... missing ... 'n_classes'`	scikit-learn nuevo	Agrega `n_classes=2,` dentro de `DecisionBoundaryDisplay(...)`
`ModuleNotFoundError: No module named 'nni'`	Notebook abierto fuera del repo	Abre la carpeta `curso-redes-neuronales` completa
La pérdida no cambia	`perceptron.W` sigue en `None`, o no ejecutaste la celda de la regla	Completa la regla y ejecútala de nuevo
Resultados distintos cada vez	Pesos iniciales al azar	Usa `np.random.seed(0)` antes de inicializar

Guía paso a paso — Clase 3: Red neuronal simple (parte 1)

Temas: neurona artificial moderna · qué es una derivada · funciones de activación · descenso por gradiente en una variable · práctica en Python
Duración: ~2 h 30 min
Material: diapositivas `Clase3_Diapositivas` (PDF y PowerPoint, 20 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clases 1 y 2 (ambiente `nni` instalado y repositorio clonado)
Lecturas base (libro del Dr. Irving Vasquez):

- Neurona artificial moderna
- Descenso por gradiente (preliminares)

> El módulo 3 del curso ("Red neuronal simple") es largo, así que se divide en dos clases:
>
> - **Clase 3 (esta):** la neurona moderna, las funciones de activación y el descenso por gradiente con una sola variable.
> - **Clase 4:** usar el descenso por gradiente para entrenar los pesos de una neurona.

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar en qué se diferencia la neurona moderna del perceptrón: su activación es **continua y derivable**.
2. Calcular a mano las funciones de activación lineal, sigmoide, tanh y ReLU.
3. Explicar con sus palabras qué es una derivada y para qué sirve.
4. Aplicar a mano varias iteraciones del descenso por gradiente y reconocer el efecto de la tasa de aprendizaje.
5. Programar una neurona con sigmoide y el descenso por gradiente en Python.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 2 (10 min)	1–4
0:10	B1. Del escalón a la neurona moderna (15 min)	5–6

Tiempo	Tema	Diapositivas
0:25	B2. ¿Qué es una derivada? (10 min)	7
0:35	B3. Funciones de activación (20 min)	8–10
0:55	B4. Ejemplo a mano (5 min)	11
1:00	C1. Práctica: neurona con sigmoide (20 min)	12
1:20	Descanso (10 min)	13
1:30	B5. Descenso por gradiente (25 min)	14–17
1:55	Ejercicios en papel (10 min)	18
2:05	C2. Práctica: descenso por gradiente (20 min)	19
2:25	Cierre, quiz y tarea (5 min)	20

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

Verificación: debe decir `Already up to date.` o listar archivos nuevos, entre ellos `docs/practica_03.py`.

> Si `git pull` se queja de "cambios locales" en un notebook, es porque lo modificaste en clase. Guarda una copia con otro nombre (por ejemplo `1_01_mio.ipynb`) y ejecuta `git checkout -- notebooks` antes de repetir `git pull`.

A.2 Activar el ambiente y probar la práctica

```
conda activate nni
```

```
python docs/practica_03.py
```

Verificación: imprime una tabla de funciones de activación, varias filas de descenso por gradiente y al final `Todas las comprobaciones pasaron.`

Opcional, para ver las gráficas:

```
python docs/practica_03.py --graficar
```

Se abren dos ventanas: las cuatro funciones de activación y los pasos del descenso sobre una parábola. Ciérralas para terminar.

A.3 Checklist

- [ ] `python docs/practica_03.py` funciona.
- [ ] VS Code abre `notebooks/1_04_red_neuronal_simple.ipynb` con el kernel **nni**.
- [ ] Existe la carpeta `notebooks/output` (la usa el notebook `1_05` para guardar una gráfica; viene en el repositorio).
- [ ] En el pizarrón: una parábola grande $y = x^2$ con los puntos $x = 5, 4, 3.2$. La usarás para el descenso por gradiente.
- [ ] Una calculadora a mano (o la del celular) con la función e^x, para el ejemplo de la sigmoide.

PARTE B — Teoría desde cero

Repaso de la Clase 2 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Cómo calcula su salida un perceptrón?" → suma ponderada $h = \Sigma\, w_i x_i + b$, y luego el escalón: 1 si $h \geq 0$, 0 si no.
- "¿Cómo aprende?" → corrige los pesos con el error: $W \leftarrow W + \eta(y - \hat{y})X$.
- "¿Qué no pudo aprender?" → XOR, porque no es linealmente separable.

Hoy damos el siguiente paso: cambiar el escalón por algo "suave" y aprender una herramienta nueva para entrenar, el **descenso por gradiente**.

B1. Del escalón a la neurona moderna (15 min) — diapositivas 5–6

¿Qué tiene de malo el escalón?

Dibuja el escalón en el pizarrón: vale 0 a la izquierda de $h = 0$ y salta a 1 a la derecha.

1. **Es "todo o nada".** Si $h = -0.01$ o $h = -100$, la salida es la misma (0). La neurona no nos dice si estuvo **cerca** o **lejos** de acertar.
2. **Es plano en casi todas partes.** Si movemos un poquito un peso, la salida casi nunca cambia. No hay una "pista" de hacia dónde mover los pesos.
3. **Salta de golpe en $h = 0$.** En ese punto no hay una pendiente definida.

Analogía: buscar las llaves a oscuras con alguien que solo dice "sí las encontraste" o "no las encontraste". Es mucho más fácil si te dice "frío… tibio… caliente": eso es lo que da una función suave.

La neurona artificial moderna

- **1960.** Bernard Widrow y Marcian Hoff proponen **ADALINE** (*ADAptive LInear NEuron*).
- Los **mismos dos pasos** del perceptrón:
    1. combinación lineal: $h = \Sigma\, w_i x_i + b$;

2. activación: $\hat{y} = f(h)$.

- **La diferencia:** f es **continua y derivable**. No salta; cambia poco a poco.
- Se entrena con la **regla delta**, que viene del **descenso por gradiente** (lo vemos hoy y la próxima clase).

En los diagramas modernos, la suma y la activación se dibujan juntas en un solo círculo.

B2. ¿Qué es una derivada? (10 min) — diapositiva 7

No hace falta saber cálculo para seguir el curso; basta esta idea:

> **La derivada de una función en un punto es la pendiente de la curva en ese punto:** cuánto sube (o baja) la función si avanzas un poquito a la derecha.

Dibuja la parábola $f(x) = x^2$ y una recta que solo **toca** la curva en un punto (la **recta tangente**):

- En $x = 3$ la curva sube fuerte hacia la derecha → la pendiente es **positiva y grande**.
- En $x = -3$ la curva baja hacia la derecha → la pendiente es **negativa**.
- En $x = 0$ (el fondo) la curva está plana → la pendiente es **0**.

Para $f(x) = x^2$ la derivada es **$f'(x) = 2x$**:

x	−3	0	1	3
$f'(x) = 2x$	−6	0	2	6

Las dos ideas que usaremos:

1. El **signo** de la derivada dice hacia dónde sube la función.
2. Su **tamaño** dice qué tan empinada está.

Notación: $f'(x)$ se lee "f prima de x". También se escribe df/dx.

B3. Funciones de activación (20 min) — diapositivas 8–10

Una **función de activación** transforma la combinación lineal h en la salida $\hat{y}$. Hoy vemos cuatro:

Función	Fórmula	Rango de salida	En palabras
Lineal	$f(h) = h$	cualquier número	Deja pasar h tal cual. Sirve para **regresión** (predecir números).
Sigmoide	$f(h) = 1 / (1 + e^{-h})$	entre 0 y 1	Un escalón "suave". Sirve para **probabilidades**.
Tangente hiperbólica (tanh)	$f(h) = (e^{h} - e^{-h}) / (e^{h} + e^{-h})$	entre −1 y 1	Como la sigmoide, pero centrada en 0.
ReLU	$f(h) = \max(0, h)$	0 o positivo	Corta los negativos a 0. Es la más usada en redes profundas.

> **e** es un número especial ≈ 2.718 (como π ≈ 3.14). e^{-h} significa "e elevado a menos h".

Tabla de valores (diapositiva 9)

Calcula 2 o 3 casillas en el pizarrón con calculadora, y muestra el resto:

h	lineal	sigmoide	tanh	ReLU
−3	−3	0.0474	−0.9951	0
−1	−1	0.2689	−0.7616	0
0	0	0.5	0	0
1	1	0.7311	0.7616	1
3	3	0.9526	0.9951	3

Que noten:

- la sigmoide **nunca** llega a 0 ni a 1;
- vale exactamente **0.5 en h = 0**;
- tanh es como la sigmoide, pero "estirada" entre −1 y 1.

La sigmoide en detalle (diapositiva 10)

- **Saturación:** para h muy grande o muy negativo, la curva se aplana. Ahí la pendiente es casi 0 y la neurona aprende muy lento. Es un problema conocido.
- **Su derivada tiene una forma muy cómoda:**

$$\sigma'(h) = \sigma(h)\,\big(1 - \sigma(h)\big)$$

h	−3	−1	0	1	3
σ′(h)	0.0452	0.1966	**0.25**	0.1966	0.0452

La pendiente máxima es 0.25, en h = 0, y casi se anula en los extremos (saturación). La usaremos en la Clase 4.

- **Una nota importante:** si todas las neuronas fueran **lineales**, por muchas que pongamos en cadena, el resultado seguiría siendo una recta. Las funciones **no lineales** (sigmoide, tanh, ReLU) son las que permiten aprender patrones curvos.

B4. Ejemplo a mano (5 min) — diapositiva 11

Es el mismo ejemplo que verificarán en el notebook `1_04`.

Datos: entradas X = [0.7, −0.3], pesos W = [0.1, 0.8], sesgo b = −0.1.

1. **Combinación lineal:** h = 0.1·0.7 + 0.8·(−0.3) − 0.1 = 0.07 − 0.24 − 0.1 = **−0.27**
2. **Sigmoide:** $e^{0.27}$ ≈ 1.3100, así que σ(−0.27) = 1 / (1 + 1.3100) = 1 / 2.3100 ≈ **0.4329**
3. **Con tanh** en lugar de sigmoide: tanh(−0.27) ≈ **−0.2636**

Compara con la Clase 2: con el escalón, la salida sería simplemente 0 (porque h < 0). La sigmoide dice algo más rico: "0.43, casi en la frontera".

B5. Descenso por gradiente (25 min) — diapositivas 14–17

El problema (diapositiva 14)

Queremos encontrar el valor de x que hace **lo más pequeña posible** una función f(x). En redes neuronales, f será **el error de la red**, y x serán **los pesos**.

Analogía de la montaña a oscuras: estás en una montaña, de noche, sin linterna. Quieres bajar al valle. No ves nada, pero **sientes con los pies hacia dónde baja el terreno**. Das un paso en esa dirección, vuelves a sentir y repites. Eso es el descenso por gradiente: la "inclinación que sientes" es la **derivada**.

La regla (diapositiva 15)

$$x_{nuevo} = x - \eta\, f'(x)$$

Símbolo	Significa
x	El valor actual (dónde estás parado)
f′(x)	La pendiente en ese punto (hacia dónde sube)
η (eta)	La **tasa de aprendizaje**: el tamaño del paso

¿Por qué el signo menos? La derivada apunta hacia donde la función **sube**. Queremos **bajar**, así que vamos en la dirección **contraria**:

- si la pendiente es positiva (sube a la derecha), restamos → vamos a la izquierda;
- si es negativa (sube a la izquierda), restar un negativo suma → vamos a la derecha.

El algoritmo:

1. Empieza en un valor inicial x_0 (puede ser al azar).
2. Calcula la pendiente f′(x).
3. Actualiza: x ← x − η·f′(x).
4. Repite hasta que x casi no cambie, o un número fijo de veces (**iteraciones**).

Ejemplo resuelto (diapositiva 16)

f(x) = x^2, f′(x) = 2x, η = 0.1.

Un paso desde x = −2:
x_nuevo = −2 − 0.1·(2·(−2)) = −2 − 0.1·(−4) = −2 + 0.4 = **−1.6**.
Comprobamos que mejoró: f(−2) = 4 y f(−1.6) = 2.56 ✓.

Cinco pasos desde x = 5 (hazlos en el pizarrón):

Iteración	x	f(x) = x^2	f′(x) = 2x	x − 0.1·f′(x)
1	5	25	10	**4**
2	4	16	8	**3.2**
3	3.2	10.24	6.4	**2.56**
4	2.56	6.55	5.12	**2.048**

Iteración	x	$f(x) = x^2$	$f'(x) = 2x$	$x - 0.1 \cdot f'(x)$
5	2.048	4.19	4.096	**1.6384**

Los pasos se hacen **más cortos** al acercarse al fondo, porque la pendiente es más pequeña ahí. Nadie le dice al algoritmo que frene: lo hace solo.

La tasa de aprendizaje (diapositiva 17)

Mismo problema ($f(x) = x^2$, desde $x = 5$), distintos valores de η, 5 pasos:

η	Valores de x	Qué pasa
0.01	5 → 4.9 → 4.80 → 4.71 → 4.61 → 4.52	**Muy lento**: pasos diminutos
0.1	5 → 4 → 3.2 → 2.56 → 2.05 → 1.64	**Bien**: baja de forma estable
0.5	5 → 0 → 0 → …	Llega de un salto (solo pasa en este problema tan simple)
0.9	5 → −4 → 3.2 → −2.56 → 2.05 → …	**Oscila**: salta de un lado al otro del valle
1.1	5 → −6 → 7.2 → −8.64 → 10.37 → …	**Diverge**: cada salto lo aleja más

Mensaje clave: η es un **hiperparámetro**: lo elegimos nosotros, no lo aprende la red. Si es muy pequeño, el entrenamiento tarda; si es muy grande, se descontrola.

Ejercicios en papel (10 min) — diapositiva 18

1. Con $f(x) = x^2$, $x_0 = 3$ y $\eta = 0.25$, calcula dos pasos del descenso.
2. Del libro: minimiza $f(x) = 2x + x^2$, empezando en $x_0 = 10$, con $\eta = 0.2$. Calcula 6 valores. (Pista: $f'(x) = 2 + 2x$.)
3. ¿Cuánto vale la sigmoide en $h = 0$? ¿Y su derivada?

Respuestas:

1. $f'(3) = 6 \rightarrow x_1 = 3 - 0.25 \cdot 6 =$ **1.5**; $f'(1.5) = 3 \rightarrow x_2 = 1.5 - 0.75 =$ **0.75**.
2. 10 → **5.6** → **2.96** → **1.376** → **0.4256** → **−0.1446** → **−0.4868**. Se acerca a $x = -1$, que es donde $f'(x) = 2 + 2x$ vale 0 (el mínimo).
3. $\sigma(0) =$ **0.5** y $\sigma'(0) = 0.5 \cdot (1 - 0.5) =$ **0.25**.

PARTE C — Práctica en la computadora

Preparación (todos):

1. Abre VS Code → **File** → **Open Folder** → `C:\Users\TuUsuario\projects\curso-redes-neuronales`.
2. En cada notebook, revisa que el kernel sea **nni** (arriba a la derecha).
3. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**. Las partes a completar dicen `# TODO`.

C1. Notebook `notebooks/1_04_red_neuronal_simple.ipynb` (20 min, 3 puntos) — diapositiva 12

Qué hay que hacer:

1. **TODO (1 pt):** completar `sigmoide(h)`.
   - Pista: la fórmula es $1 / (1 + e^{-h})$. En NumPy, e^x se escribe `np.exp(x)`.
2. **TODO (1 pt):** completar la clase `Neurona`:
   - `combinacion_lineal(X)` debe devolver $h = W \cdot X + b$. Pista: `np.dot(self.W, X)` hace la suma de productos.
   - `forward(X)` debe calcular h con el método anterior y luego aplicar `self.activacion(h)`.
3. Ejecuta la celda de prueba.
   Verificación: debe imprimir `Output: 0.4329` (el ejemplo de B4).
4. **TODO (1 pt):** escribir una función `tanh(h)` con la fórmula $(e^h - e^{-h})/(e^h + e^{-h})$, crear una neurona con ella y comparar con la sigmoide.
   Verificación: con los mismos datos, la tanh da ≈ **−0.2636**. Compara también con `np.tanh(-0.27)`.

Errores típicos:

- Escribir `np.exp(h)` en vez de `np.exp(-h)` (falta el signo menos). Da 0.5671 en lugar de 0.4329.
- Olvidar `self.` dentro de la clase (`W` en lugar de `self.W`).

C2. Notebook `notebooks/1_05_descenso_por_gradiente_1D.ipynb` (15 min, 2 puntos) — diapositiva 19

Qué hay que hacer:

1. **TODO (1 pt):** completar `gradient_descent_update(theta, learning_rate)`.
   - Pista: es exactamente la regla de B5, y la derivada ya está programada como `df(x)`.
2. Ejecuta la celda de prueba.
   Verificación: el historial debe ser `[5, 4.0, 3.2, 2.56, 2.048, 1.6384]` (puede mostrar más decimales).
3. Ejecuta la celda de la gráfica: debe dibujar la parábola con los puntos rojos bajando.
4. **TODO (1 pt):** reescribe el ciclo para que se detenga solo cuando el error sea menor que 0.001 (`1e-3`), y cuenta las iteraciones.
   - Pista: usa `while` en lugar de `for`, y un contador.
   - **Verificación:** depende del criterio que elijan, y ambos son válidos si lo explican:
     - si paran cuando el **cambio** $|x_{nuevo} - x| < 0.001$, salen **32** iteraciones;
     - si paran cuando **f(x)** < 0.001, salen **23**.

C3. Demo del docente: `docs/practica_03.py` (5 min)

Proyecta y ejecuta (con `(nni)` activo, dentro del repo):

```
python docs/practica_03.py --graficar
```

Muestra:

- la tabla de activaciones (la misma de B3);
- las cinco tasas de aprendizaje de B5: lento, bien, de un salto, oscila, diverge;
- la gráfica de las cuatro activaciones.

Pregunta a la clase: "¿Qué η usarían?"

Cierre (5 min) — diapositiva 20

Resumen en tres líneas:

1. La neurona moderna cambia el escalón por una función **suave y derivable** (sigmoide, tanh, ReLU…).
2. La derivada es la **pendiente**: dice hacia dónde sube una función.
3. El descenso por gradiente baja paso a paso: $\mathbf{x \leftarrow x - \eta \cdot f'(x)}$, con η ni muy grande ni muy pequeña.

Avance: "La próxima clase usaremos esto mismo, pero en vez de bajar por x^2, bajaremos por **el error de la red**, moviendo **los pesos**."

Quiz de salida (en papel, 3 min):

Pregunta	Respuesta
¿Qué cambia la neurona moderna respecto al perceptrón?	La activación es continua y derivable
¿Entre qué valores está la salida de la sigmoide? ¿Y la de tanh?	0 y 1; −1 y 1
¿Qué representa la derivada en un punto?	La pendiente de la curva ahí
¿Por qué la regla del descenso lleva un signo menos?	Para ir en contra de la pendiente, hacia abajo
¿Qué pasa si la tasa de aprendizaje es muy grande?	Oscila o diverge

Tarea:

1. Terminar los notebooks `1_04` y `1_05` (5 puntos en total).
2. Ejecutar `python docs/practica_03.py --graficar` y cambiar en el archivo el x_0 inicial a −8: ¿llega igual al mínimo?
3. Leer del libro: Descenso por gradiente (aprendizaje), preparación de la Clase 4.

Rúbrica:

Notebook	Puntos
1_04 Red neuronal simple	3
1_05 Descenso por gradiente 1D	2

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`NameError: name 'np' is not defined`	No se ejecutó la celda de `import`	Ejecuta las celdas desde la primera, en orden
`TypeError: unsupported format string passed to NoneType`	La función todavía devuelve `None`	Completa el `TODO` y vuelve a ejecutar esa celda **y** la de prueba
La sigmoide da 0.5671 en vez de 0.4329	Falta el signo menos: `np.exp(h)`	Debe ser `np.exp(-h)`
`FileNotFoundError: ... output/gradient_descent_1D.png`	No existe la carpeta `output`	Haz `git pull` (ya viene en el repo) o créala dentro de `notebooks`
`RuntimeWarning: overflow in exp`	h muy grande en `np.exp`	Es solo un aviso; con valores normales no aparece
El ciclo `while` no termina	El criterio de parada nunca se cumple, o η ≥ 1 hace que diverja	Revisa la condición y usa η = 0.1; detén la celda con el botón ■
Las gráficas de `practica_03.py` no aparecen	Falta `--graficar`	`python docs/practica_03.py --graficar`
`git pull` dice que hay cambios locales	Modificaste notebooks del repo	Guarda tu copia con otro nombre y ejecuta `git checkout -- notebooks`

CLASE 3 · MÓDULO 3 (PARTE 1)

Red neuronal simple: activación y gradiente

Neurona moderna, funciones de activación y descenso por gradiente

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 2
0:10	Del escalón a la neurona moderna
0:25	¿Qué es una derivada?
0:35	Funciones de activación
0:55	Ejemplo a mano
1:00	Práctica: neurona con sigmoide
1:20	Descanso
1:30	Descenso por gradiente
1:55	Ejercicios en papel
2:05	Práctica: descenso por gradiente
2:25	Cierre y tarea

Al terminar la clase podrás…

1

Distinguir

la neurona moderna del perceptrón: activación continua.

2

Calcular

las activaciones lineal, sigmoide, tanh y ReLU.

3

Explicar

qué es una derivada: la pendiente.

4

Aplicar

el descenso por gradiente y elegir η.

5

Programar

una neurona y el descenso en Python.

Repaso de la Clase 2

¿Cómo calcula?

$h = \Sigma\, w_i x_i + b$ y luego el escalón: 1 si $h \geq 0$.

¿Cómo aprende?

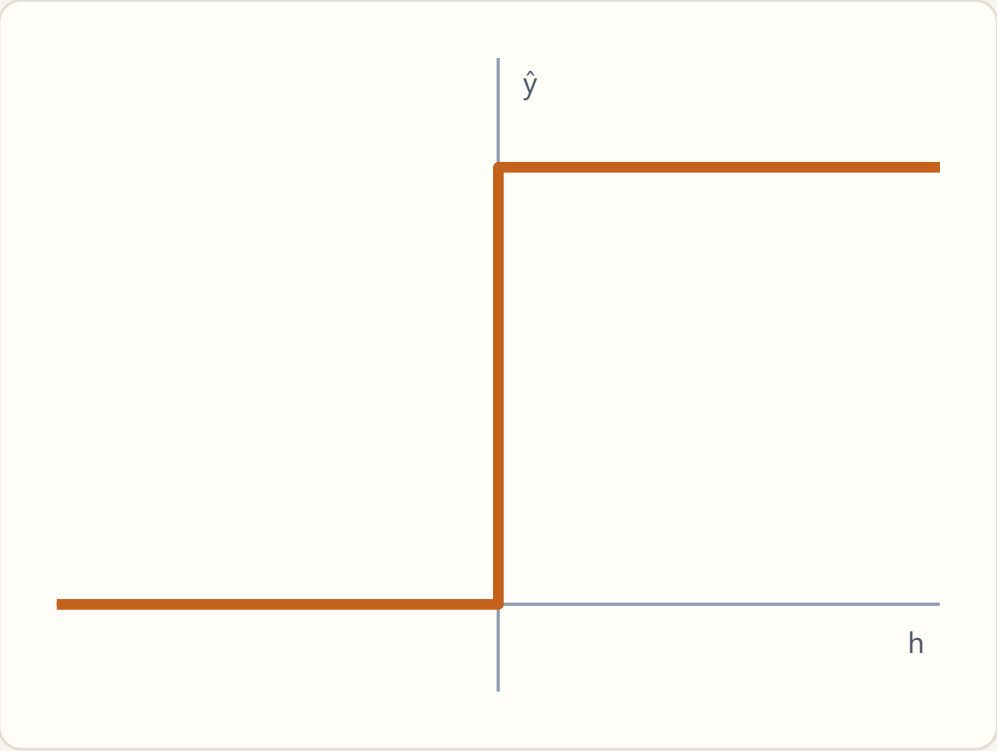
$W \leftarrow W + \eta(y - \hat{y})X$: corrige con el error.

¿Qué no pudo?

XOR: no es linealmente separable.

Hoy: cambiar el escalón por algo suave y aprender a bajar montañas.

¿Qué tiene de malo el escalón?



Escalón: salta de 0 a 1 en h = 0

- **Todo o nada:** h = −0.01 y h = −100 dan lo mismo.
- **Plano:** mover un poco un peso casi nunca cambia la salida.
- **Salto:** en h = 0 no hay pendiente definida.

Buscar llaves con “sí / no” es difícil; con “frío… tibio… caliente” es fácil.

La neurona artificial moderna

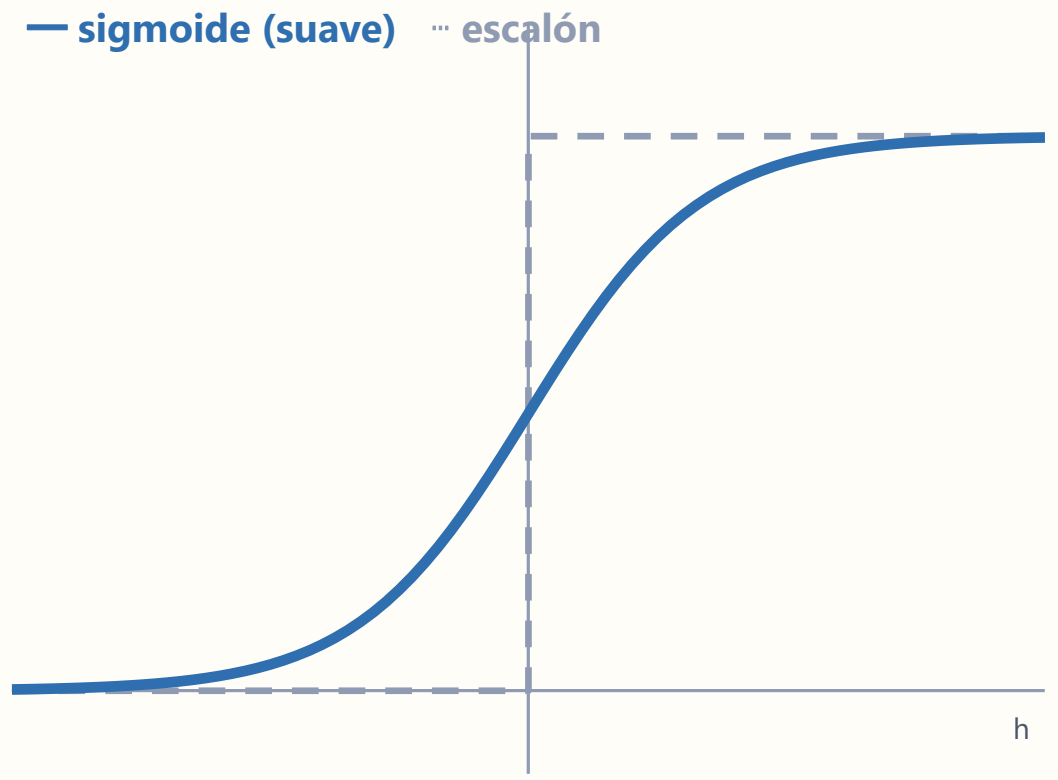
Widrow y Hoff, 1960

ADALINE: ADAptive LInear NEuron.

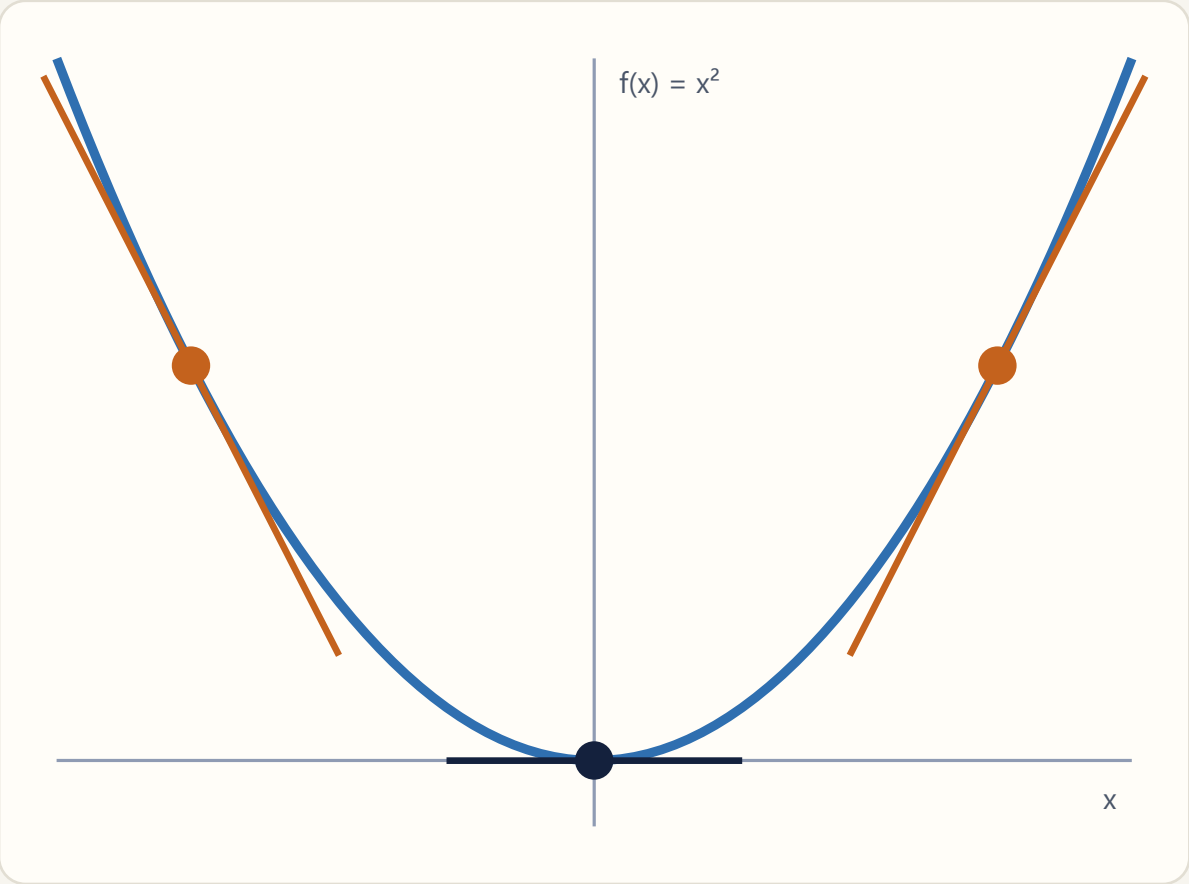
Los mismos dos pasos

$$h = \Sigma\, w_i x_i + b$$

$$\hat{y} = f(h) \quad \text{con } f \textbf{ continua y derivable}$$



¿Qué es una derivada?



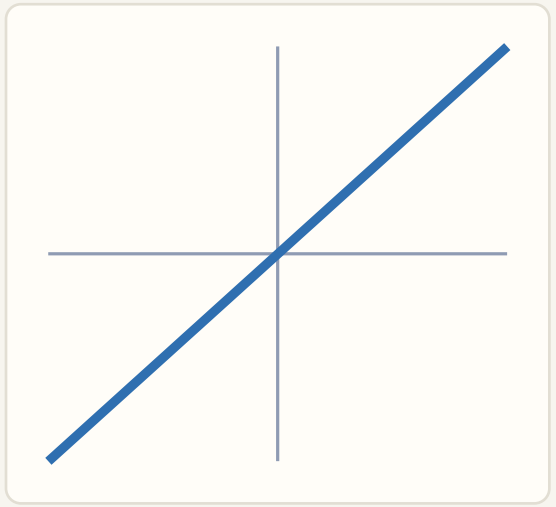
Derivada = pendiente de la curva en un punto: cuánto sube si avanzas un poquito a la derecha.

x	−3	0	1	3
f′(x) = 2x	−6	0	2	6

Signo: hacia dónde sube.

Tamaño: qué tan empinada está.

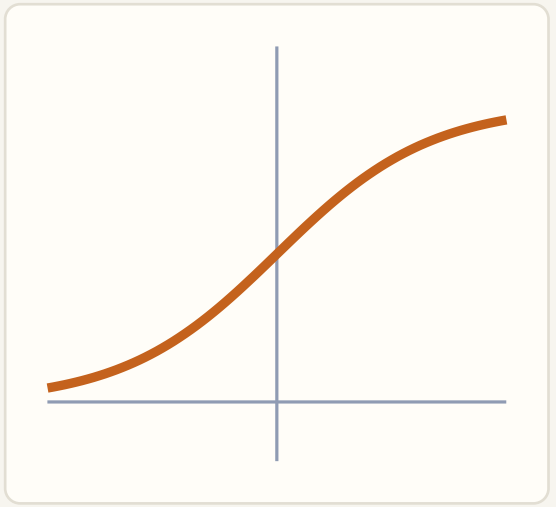
Funciones de activación



Lineal

$f(h) = h$

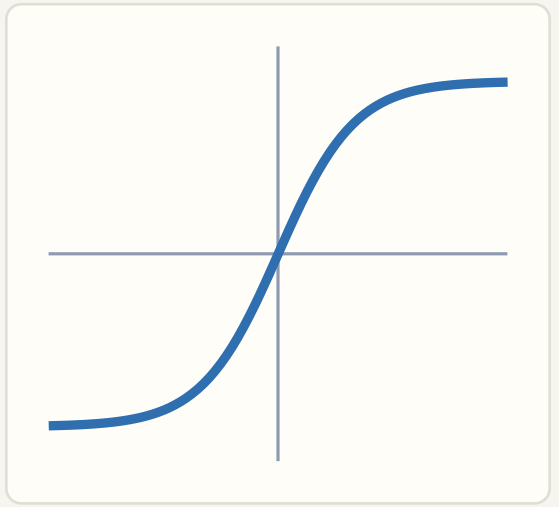
cualquier número



Sigmoide

$\frac{1}{(1 + e^{-h})}$

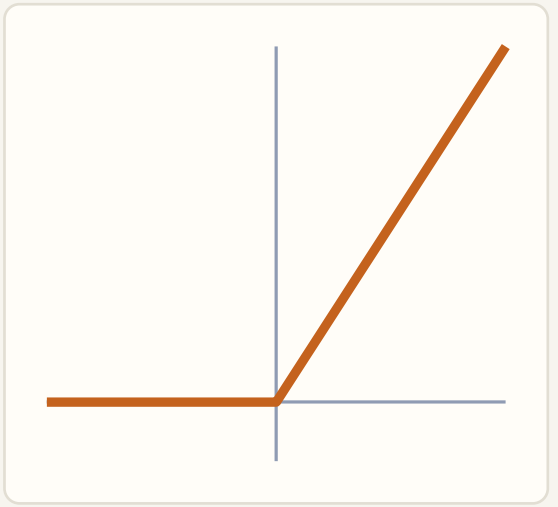
entre 0 y 1



tanh

$\frac{(e^{h} - e^{-h})}{(e^{h} + e^{-h})}$

entre −1 y 1



ReLU

$máx(0, h)$

0 o positivo

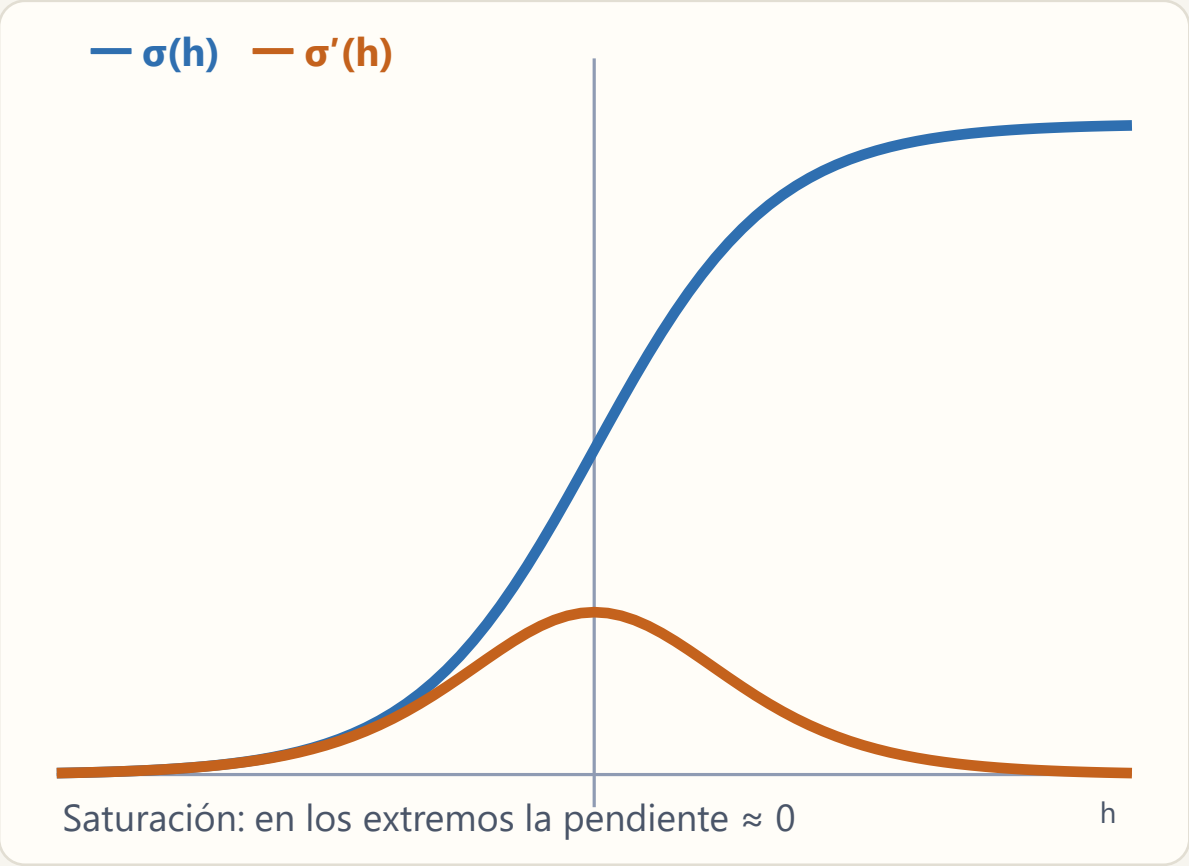
e ≈ 2.718 (como π ≈ 3.14). e^{-h} = “e elevado a menos h”.

Tabla de valores

h	lineal	sigmoide	tanh	ReLU
−3	−3	0.0474	−0.9951	0
−1	−1	0.2689	−0.7616	0
0	0	0.5	0	0
1	1	0.7311	0.7616	1
3	3	0.9526	0.9951	3

La sigmoide nunca llega a 0 ni a 1, y vale exactamente 0.5 cuando h = 0.

La sigmoide en detalle



$$\sigma'(h) = \sigma(h) \cdot (1 - \sigma(h))$$

Máxima en h = 0: σ′(0) = 0.25.

Saturación: con h muy grande o muy negativo aprende lento.

No lineal: si todo fuera lineal, la red entera seguiría siendo una recta.

Ejemplo a mano

`X = [0.7, −0.3]  ·  W = [0.1, 0.8]  ·  b = −0.1`

Paso	Cálculo
1. Combinación lineal	h = 0.07 − 0.24 − 0.1 = −0.27
2. Sigmoide	e^0.27 ≈ 1.3100 → σ(−0.27) = 1 / 2.3100 ≈ 0.4329
3. Con tanh	tanh(−0.27) ≈ −0.2636
Con el escalón	h < 0 → ŷ = 0 (sin más información)

La sigmoide dice algo más rico: “0.43, casi en la frontera”.

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_04_red_neuronal_simple

TODO 1 pt sigmoide(h) con np.exp(-h)

TODO 1 pt Neurona: combinacion_lineal y forward

TODO 1 pt tanh y comparación con la sigmoide

Pistas: np.dot(self.W, X) hace la suma de productos; dentro de la clase usa self.W y self.b.

```
# comprobaciones
Output: 0.4329        # sigmoide
tanh(-0.27) = -0.2636

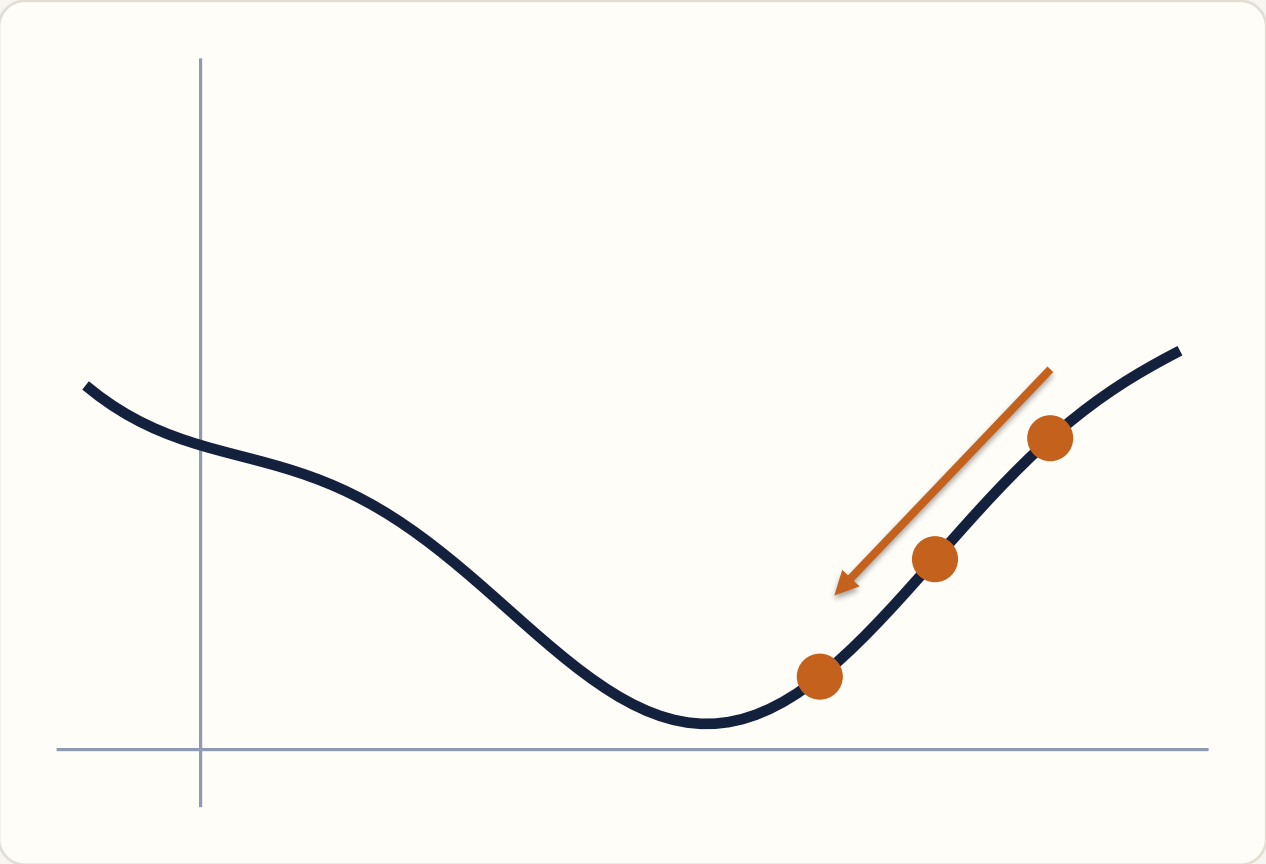
# error típico: np.exp(h)
# sin el signo menos -> 0.5671
```

Sin Python: abre el notebook en Google Colab.

Descanso · 10 min

Después: el descenso por gradiente

Descenso por gradiente



Objetivo: encontrar el x que hace f(x) lo más pequeña posible.

En redes: f es el error y x son los pesos.

De noche, sin linterna: sientes con los pies hacia dónde baja y das un paso.

La regla del descenso

$$x \leftarrow x - \eta \cdot f'(x)$$

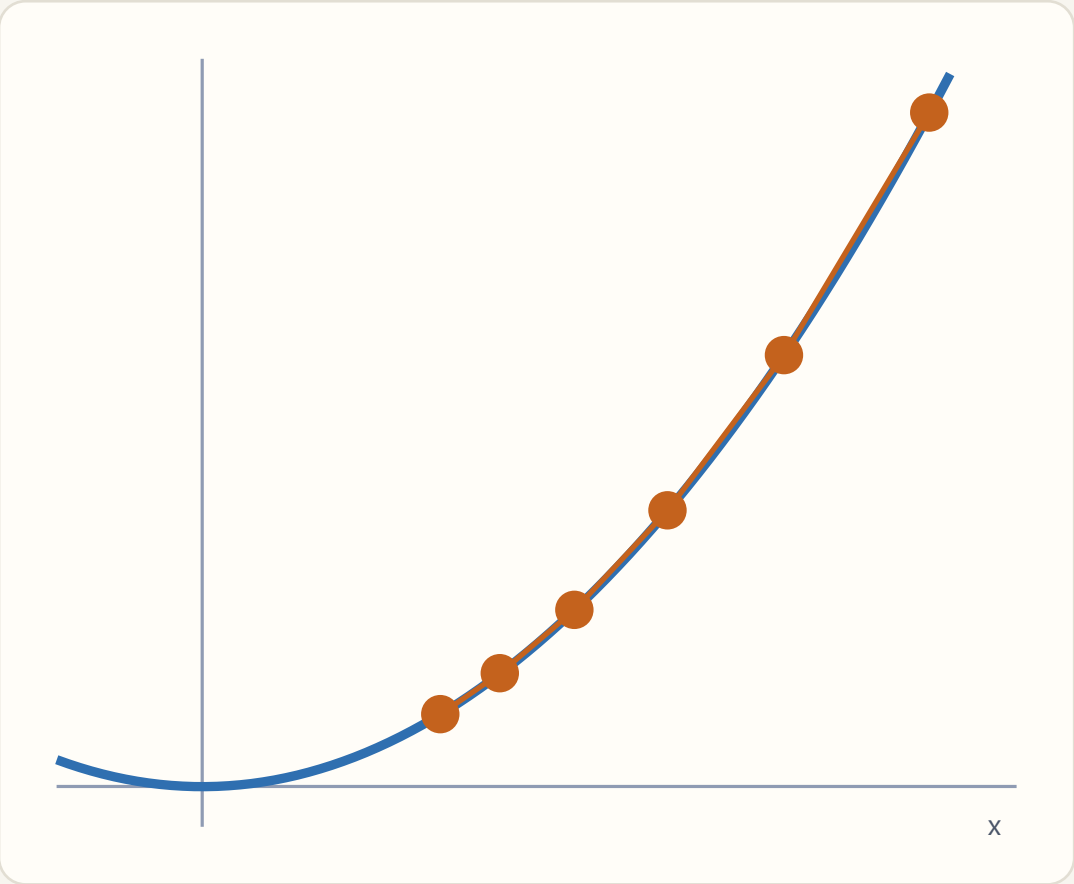
Símbolo	Significa
x	Dónde estás parado
f′(x)	La pendiente: hacia dónde sube
η (eta)	Tasa de aprendizaje: tamaño del paso

¿Por qué el signo menos?

La derivada apunta hacia donde la función sube.

Queremos bajar: vamos en **contra de la pendiente.**

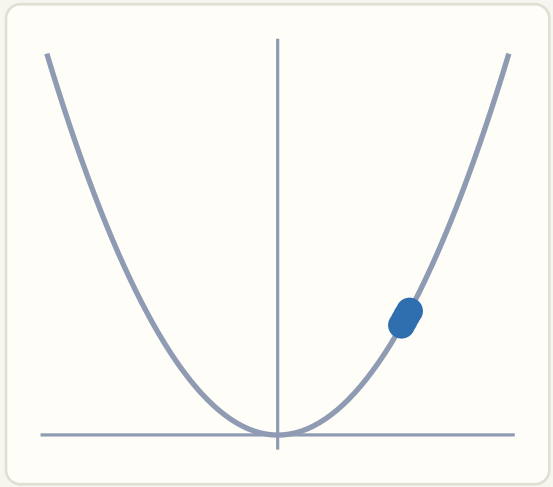
Ejemplo: $f(x) = x^2$, $\eta = 0.1$



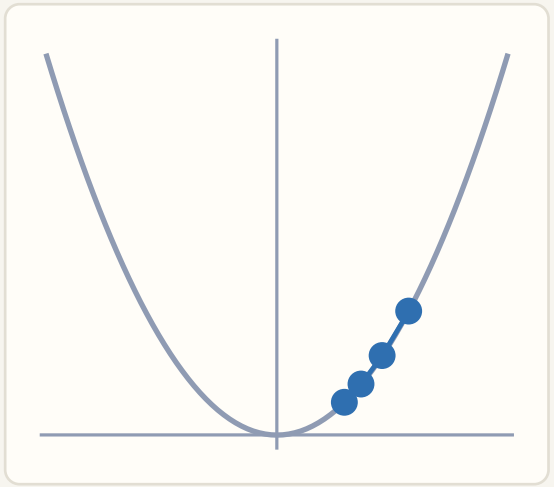
x	f′(x) = 2x	x − 0.1·f′(x)
5	10	4
4	8	3.2
3.2	6.4	2.56
2.56	5.12	2.048
2.048	4.096	1.6384

Los pasos se acortan solos al acercarse al fondo: la pendiente es más pequeña ahí.

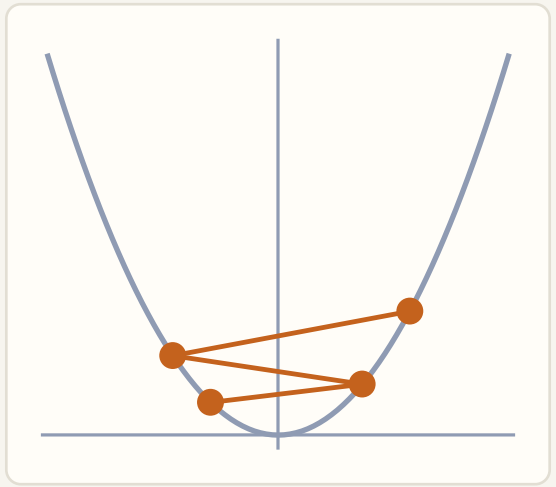
La tasa de aprendizaje η



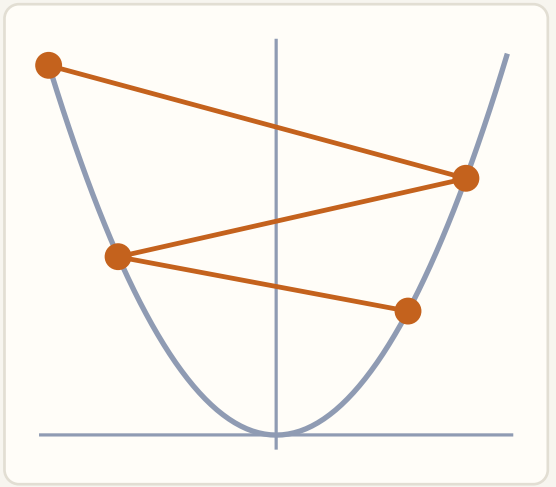
η = 0.01 · muy lento



η = 0.1 · bien



η = 0.9 · oscila



η = 1.1 · diverge

η	0.01	0.1	0.5	0.9	1.1
Desde x = 5, cinco pasos	4.52	1.64	0 (de un salto)	−1.64 (oscila)	−12.44 (diverge)

η es un hiperparámetro: lo elegimos nosotros. Ni muy pequeño ni muy grande.

Ejercicios en papel · 10 min

1

$f(x) = x^2$

$x_0 = 3 \cdot \eta = 0.25$

Calcula dos pasos.

2

$f(x) = 2x + x^2$

$x_0 = 10 \cdot \eta = 0.2$

Pista: $f'(x) = 2 + 2x$

Calcula 6 valores.

3

Sigmoide en $h = 0$

¿Cuánto vale $\sigma(0)$? ¿Y $\sigma'(0)$?

NOTEBOOKS · 15 MIN · 2 PUNTOS · + DOCS/PRACTICA_03.PY

Práctica · 1_05_descenso_por_gradiente_1D

TODO 1 pt gradient_descent_update: la regla del descenso

TODO 1 pt ciclo while hasta error < 1e-3; contar iteraciones

Pistas: la derivada ya existe como df(x). Para detenerse, usa while y un contador.

```
# historial esperado
[5, 4.0, 3.2, 2.56, 2.048, 1.6384]

# iteraciones con 1e-3:
# |x_nuevo - x| -> 32   f(x) -> 23
```

Demo: python docs/practica_03.py --graficar (activaciones y cinco valores de η)

Para llevar

1. Neurona moderna: activación suave y derivable.
2. Derivada = pendiente: dice hacia dónde sube.
3. Descenso: $x \leftarrow x - \eta \cdot f'(x)$, con η ni muy grande ni muy pequeña.

Próxima clase: bajar por el error de la red moviendo los pesos.

Tarea: notebooks 1_04 y 1_05 (5 pts)

Leer: libro, “Descenso por gradiente” (aprendizaje)

Guía del notebook `1_04_red_neuronal_simple`: una neurona con sigmoide

Puntos: 3 · **Tiempo:** 20 min · **Clase:** 3 · **Archivo:** `notebooks/1_04_red_neuronal_simple.ipynb`
Teoría completa: Guía de la Clase 3, secciones B1 a B4.

¿Qué vas a aprender?

- A programar la función **sigmoide**, un "escalón suave".
- A escribir una neurona como una **clase** de Python, con sus métodos.
- A comparar la sigmoide con la **tangente hiperbólica (tanh)**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_04_red_neuronal_simple.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- La neurona moderna hace lo mismo que el perceptrón: h = W·X + b y luego ŷ = f(h). Pero **f es suave**.
- **Sigmoide:** $\sigma(h) = 1 / (1 + e^{-h})$. Devuelve un número entre 0 y 1.
- **tanh:** $(e^{h} - e^{-h}) / (e^{h} + e^{-h})$. Devuelve un número entre −1 y 1.
- En NumPy, e^{x} se escribe `np.exp(x)`, y el producto punto W·X se escribe `np.dot(W, X)`.

Paso a paso

Celda "Cargamos paquetes"

Ejecútala.

Celda `TODO (1 punto)`: `sigmoide(h)`

Escribe la fórmula y devuelve el resultado.
⚠️ Ojo con el **signo menos** del exponente: es `np.exp(-h)`.

Celda `TODO (1 punto)`: clase `Neurona`

Completa los dos métodos:

- **`combinacion_lineal(self, X)`:** devuelve h = W·X + b. Usa `np.dot(self.W, X) + self.b`. Dentro de la clase, los pesos se llaman `self.W` y el sesgo `self.b`.
- **`forward(self, X)`:** calcula h llamando a `self.combinacion_lineal(X)` y devuelve `self.activacion(h)`.

Celda "Definamos unos pesos y sesgo"

Usa entradas [0.7, −0.3], pesos [0.1, 0.8] y sesgo −0.1.

✅ **Comprobación:** debe imprimir `Output: 0.4329`.

- La cuenta: h = 0.07 − 0.24 − 0.1 = −0.27, y σ(−0.27) = 1 / (1 + $e^{0.27}$) = 1 / 2.3100 ≈ 0.4329.
- Si te da **0.5671**, te falta el signo menos en `np.exp(-h)`.

Celda `TODO (1 punto)`: tanh y comparación

1. Escribe una función `tanh(h)` con la fórmula de arriba.
2. Crea otra neurona con los mismos pesos, sesgo y entradas, pero con `tanh` como activación.
3. Imprime las dos salidas.

✅ **Comprobación:**

- con tanh sale **−0.2636**, y con la sigmoide **0.4329**;
- puedes comparar tu tanh con `np.tanh(-0.27)`, que también da −0.2636.

Para comentar: tanh es negativa porque h < 0; la sigmoide nunca es negativa (vale entre 0 y 1).

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported format string passed to NoneType`	Una función o método devuelve `None`	Completa el `TODO` y ejecuta de nuevo esa celda **y** la de prueba
Da 0.5671	Falta el signo: `np.exp(h)`	Debe ser `np.exp(-h)`
`NameError: name 'W' is not defined`	Dentro de la clase escribiste `W`	Usa `self.W` y `self.b`
`ValueError: shapes not aligned`	Pesos y entradas de distinto tamaño	Ambos deben tener 2 elementos

Guía del notebook `1_05_descenso_por_gradiente_1D`: bajar por una parábola

Puntos: 2 · **Tiempo:** 15–20 min · **Clase:** 3 · **Archivo:** `notebooks/1_05_descenso_por_gradiente_1D.ipynb`
Teoría completa: Guía de la Clase 3, sección B5.

¿Qué vas a aprender?

- A programar la regla del **descenso por gradiente**: $x \leftarrow x - \eta \cdot f'(x)$.
- A detener un ciclo cuando el resultado ya casi no cambia (**criterio de parada**).

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_05_descenso_por_gradiente_1D.ipynb` → kernel **nni**.
3. El notebook guarda una gráfica en `notebooks/output/`. Esa carpeta ya viene en el repositorio; si no la tienes, haz `git pull` o créala.

La idea en 2 minutos

- Queremos el x que hace **mínima** a $f(x) = x^2$. La respuesta es $x = 0$.
- La **derivada** $f'(x) = 2x$ es la **pendiente**: indica hacia dónde sube la curva.
- Para bajar, damos un paso **en contra** de la pendiente: **$x \leftarrow x - \eta \cdot f'(x)$**.
- **η** (tasa de aprendizaje) es el tamaño del paso.

Paso a paso

Celda: `f` y `df`

Define la función x^2 y su derivada $2x$. Ejecútala.

Celda `TODO`: `gradient_descent_update(theta, learning_rate)`

Devuelve el valor nuevo de theta con la regla de arriba. La derivada ya existe: es `df(theta)`.

Celda "Prueba del método"

Empieza en $x = 5$ con tasa 0.1 y hace 5 pasos.

✅ **Comprobación:** `Historial de valores de x: [5, 4.0, 3.2, 2.56, 2.048, 1.6384]`. Puede mostrar más decimales, como 1.6384000000000003.

Cada paso multiplica x por 0.8, porque $x - 0.1 \cdot 2x = 0.8x$. Por eso se acerca a 0 cada vez más despacio.

Celda de la gráfica

Dibuja la parábola y los puntos rojos bajando hacia el fondo. También la guarda en `output/gradient_descent_1D.png`.

Celda `TODO (1 punto)`: detenerse con un umbral

Reescribe el ciclo con `while` para que se detenga cuando "el error" sea menor que 0.001, y cuenta las iteraciones.

Pistas:

1. Empieza con `x = 5` y un contador `iteraciones = 0`.
2. Dentro del `while`: calcula el x nuevo, suma 1 al contador, y decide si ya paras.
3. Elige y escribe tu **criterio de parada**.

✅ **Comprobación** (ambos criterios son válidos si los explicas):

Criterio	Iteraciones
El **cambio** $\|x_{nuevo} - x\| < 0.001$	**32** (x final ≈ 0.00396)
El **valor** $f(x) = x^2 < 0.001$	**23**

Para pensar

- ¿Qué pasa con tasa = 1.1? → Cada paso multiplica x por −1.2: oscila y **diverge**.
- ¿Y con 0.5? → Llega a 0 en un solo paso. Pasa solo en esta función tan simple.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported operand ... NoneType`	`gradient_descent_update` devuelve `None`	Completa la función
`FileNotFoundError: output/gradient_descent_1D.png`	No existe la carpeta `output`	`git pull`, o crea la carpeta `notebooks/output`
El historial **crece** en lugar de bajar	Sumaste en vez de restar	Es `theta - learning_rate * df(theta)`
El `while` nunca termina	La condición nunca se cumple, o la tasa es muy grande	Revisa el criterio; detén la celda con el botón ■

Guía paso a paso — Clase 4: Red neuronal simple (parte 2)

Temas: medir el error · regla delta (descenso por gradiente en una neurona) · entrenamiento completo · inicialización de pesos · ADALINE · red con varias salidas
Duración: ~2 h 30 min
Material: diapositivas `Clase4_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clase 3 (funciones de activación, derivada y descenso por gradiente en una variable)
Lecturas base (libro del Dr. Irving Vasquez):

- Descenso por gradiente (aprendizaje)
- Redes neuronales de varias salidas

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Medir el error de una red con el error cuadrático medio (MSE).
2. Aplicar a mano un paso de la **regla delta**: término de error, incremento y actualización.
3. Entrenar una neurona completa con descenso por gradiente y medir su exactitud.
4. Explicar por qué los pesos se inicializan al azar y cómo funciona la inicialización de Xavier.
5. Calcular la salida de una red de una capa con varias salidas usando matrices.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 3 (10 min)	1–4
0:10	B1. Medir el error (15 min)	5–6
0:25	B2. La regla delta (25 min)	7–10
0:50	C1. Práctica: un paso del descenso (20 min)	11
1:10	Descanso (10 min)	12
1:20	B3. Entrenamiento completo e inicialización (15 min)	13–15
1:35	C2. Práctica: entrenar una neurona (25 min)	16

Tiempo	Tema	Diapositivas
2:00	B4. ADALINE (8 min)	17
2:08	B5. Red con varias salidas (12 min)	18–19
2:20	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio y probar la práctica

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_04.py
```

Verificación: imprime cuatro bloques (regla delta, OR y XOR, ADALINE, red de varias salidas) y al final `Todas las comprobaciones pasaron.`

Para ver las gráficas (la pérdida bajando y la recta de ADALINE):

```
python docs/practica_04.py --graficar
```

A.2 Checklist

- [ ] `python docs/practica_04.py` funciona.
- [ ] VS Code abre `notebooks/1_07_descenso_por_gradiente.ipynb` con el kernel **nni** y la **primera celda corre sin error** (carga `data.csv` con `data_prep`).
- [ ] En el pizarrón: una "parábola del error" con el peso w en el eje horizontal y el error E en el vertical.
- [ ] Calculadora con e^x.

PARTE B — Teoría desde cero

Repaso de la Clase 3 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Qué hace la sigmoide?" → convierte h en un número entre 0 y 1, de forma suave.
- "¿Qué es la derivada?" → la pendiente: hacia dónde y cuánto sube la función.
- "¿Cuál es la regla del descenso?" → $x \leftarrow x - \eta \cdot f'(x)$.

Hoy: en lugar de bajar por x^2, vamos a bajar por **el error de la red**, y lo que moveremos serán **los pesos**. Eso es **entrenar**.

B1. Medir el error (15 min) — diapositivas 5–6

Para mejorar, la red primero necesita **un número que diga qué tan mal lo está haciendo**. A ese número se le llama **error**, **pérdida** o **función de costo** (*loss*); son sinónimos en este curso.

Tres formas de medirlo

Nombre	Fórmula	Comentario
Error residual (de un ejemplo)	$e = y - \hat{y}$	Puede ser positivo o negativo
Suma de errores al cuadrado (SSE)	$\Sigma (y - \hat{y})^2$	Elevar al cuadrado evita que errores + y − se cancelen
Error cuadrático medio (MSE)	$(1/m) \Sigma (y - \hat{y})^2$	El promedio: no crece solo por tener más datos

(m = número de ejemplos.)

¿Por qué al cuadrado? Si un ejemplo se equivoca +1 y otro −1, la suma simple daría 0, como si no hubiera error. Al elevar al cuadrado, ambos cuentan. Además, castiga más los errores grandes.

Ejemplo (diapositiva 5): etiquetas y = [1, 0, 1], predicciones $\hat{y}$ = [0.8, 0.3, 0.6].

- Errores: 0.2, −0.3, 0.4 → cuadrados: 0.04, 0.09, 0.16.
- SSE = 0.29 · MSE = 0.29 / 3 ≈ **0.0967**.

La pérdida es una montaña (diapositiva 6)

Si dibujamos el error E en función de **un peso** w, sale una curva con forma de valle. **Entrenar = bajar a ese valle moviendo w.** Es exactamente lo de la Clase 3: el descenso por gradiente, pero ahora la variable es el peso.

B2. La regla delta (25 min) — diapositivas 7–10

La derivada del error respecto a un peso (diapositiva 7)

Para una neurona con un solo ejemplo usamos $E = ½ (y - \hat{y})^2$. El ½ solo está para que las cuentas queden limpias.

Cambiar un peso w_i provoca un **efecto dominó**:

> w_i cambia → **h** cambia → **$\hat{y}$ = f(h)** cambia → **el error E** cambia.

La derivada total es la multiplicación de los efectos de cada eslabón. A esto se le llama **regla de la cadena**:

Eslabón	Derivada
E respecto a $\hat{y}$	$-(y - \hat{y})$

Eslabón	Derivada
ŷ respecto a h	f′(h) (la pendiente de la activación)
h respecto a w_i	x_i (porque h = … + $w_i x_i$ + …)

$$\frac{\partial E}{\partial w_i} = -(y - \hat{y})\, f'(h)\, x_i$$

Aplicando el descenso (w ← w − η·derivada), los dos signos menos se cancelan:

$$w_i \leftarrow w_i + \eta\,(y - \hat{y})\, f'(h)\, x_i$$

La regla delta en tres pasos (diapositiva 8)

Para que sea fácil de calcular a mano, se separa en tres pasos:

Paso	Fórmula	En palabras
1. **Término de error**	δ = (y − ŷ)·f′(h)	Cuánto y hacia dónde corregir
2. **Incremento**	$\Delta w_i = \eta \cdot \delta \cdot x_i$ (y Δb = η·δ)	Cuánto mover cada peso
3. **Actualización**	$w_i \leftarrow w_i + \Delta w_i$ · b ← b + Δb	Mover los pesos

δ se lee "delta"; por eso se llama **regla delta** (Widrow y Hoff, 1960). Con la sigmoide, f′(h) = ŷ·(1 − ŷ), lo que vimos en la Clase 3.

Ejemplo resuelto (diapositiva 9) — los mismos números del notebook `1_06`

Datos: x = [1, 1], y = 1, w = [0.1, 0.2], b = 0, η = 2, activación sigmoide.

1. **Predicción:** h = 0.1·1 + 0.2·1 + 0 = 0.3 → ŷ = σ(0.3) ≈ **0.5744**
2. **Derivada de la sigmoide:** f′(h) = 0.5744·(1 − 0.5744) ≈ **0.2445**
3. **Término de error:** δ = (1 − 0.5744)·0.2445 ≈ **0.1040**
4. **Incremento:** Δw = 2·0.1040·[1, 1] ≈ **[0.2081, 0.2081]** · Δb ≈ **0.2081**
5. **Actualización:** w ≈ **[0.3081, 0.4081]** · b ≈ **0.2081**
6. **Comprobar:** h ≈ 0.924 → ŷ ≈ 0.7159. La pérdida ½(y − ŷ)² baja de **0.0905** a **0.0404** ✓

Comparación con el perceptrón (diapositiva 10)

	Perceptrón (Clase 2)	Regla delta (hoy)
Activación	Escalón	Continua (sigmoide, lineal…)
Corrección	η·(y − ŷ)·x	η·(y − ŷ)·**f′(h)**·x
De dónde sale	Intuición de Rosenblatt	Descenso por gradiente (matemática)
Qué indica el error	Solo si acertó o no	Qué tan lejos estuvo

¡Se parecen mucho! La regla delta añade f′(h), que viene de derivar.

B3. Entrenamiento completo e inicialización (15 min) — diapositivas 13–15

El algoritmo completo (diapositiva 13)

En la práctica, la red se entrena con **muchos ejemplos** y durante **muchas épocas** (pasadas por todo el conjunto de datos):

1. **Inicializar** los pesos (ver abajo).
2. Para cada **época**:
    1. Poner los incrementos acumulados en cero.
    2. Para cada ejemplo (x, y): calcular $\hat{y}$, el término de error δ y **sumar** el incremento $\delta \cdot x$ al acumulado.
    3. Actualizar los pesos con el **promedio** de los incrementos multiplicado por η (dividir entre el número de ejemplos).
    4. Medir el error (MSE) para ver si va bajando.
3. Al final, medir qué tan bien funciona con datos **que no se usaron para entrenar** (prueba).

A esta forma (actualizar una vez por época con el promedio) se le llama descenso por gradiente **por lotes**.

Inicialización de los pesos (diapositiva 14)

- **¿Por qué no empezar en cero?** Todas las predicciones saldrían iguales y, en redes más grandes, todas las neuronas aprenderían lo mismo. Se rompe la "simetría" empezando **al azar**.
- **¿Por qué valores pequeños?** Con pesos grandes, h es grande, la sigmoide **se satura** y su pendiente es casi 0: la red aprende muy lento (Clase 3).
- **Inicialización de Xavier:** valores al azar uniformes entre $-1/\sqrt{n}$ y $+1/\sqrt{n}$, donde n = número de entradas de la neurona.
    - Ejemplo: con n = 6 entradas, los pesos se eligen entre **−0.408 y +0.408**.

Exactitud y el conjunto de datos de la práctica (diapositiva 15)

- **Datos:** admisiones a un posgrado (`notebooks/data.csv`, 400 alumnos).
    - **Entradas:** puntaje GRE, promedio GPA y el prestigio de la universidad de origen (`rank`, convertido en 4 columnas de 0/1). En total, **6 entradas**.
    - **Salida:** `admit`, que vale 1 si fue admitido y 0 si no.
    - El archivo `data_prep.py` ya escala los datos y separa **90 % para entrenar** (360) y **10 % para probar** (40).
- **Exactitud** (*accuracy*) = aciertos / predicciones. La red dice "admitido" si $\hat{y} > 0.5$.

B4. ADALINE (8 min) — diapositiva 17

- Es la neurona de Widrow y Hoff con **activación lineal**: $\hat{y} = w \cdot x + b$. Sirve para **regresión**, es decir, predecir un número.
- Se entrena con el mismo descenso por gradiente. Como f(h) = h, su derivada es f′(h) = 1, y la regla queda: $w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$.
- **Ejemplo** (notebook `1_08`): predecir el **salario** a partir de los **años de experiencia** (30 personas).
- **Resultado con los datos estandarizados** (`docs/practica_04.py`): salario ≈ **9 450 · años + 24 848**. Para 5 años de experiencia → ≈ **72 098**.

- **Lección práctica:** en el notebook los datos no están escalados (los salarios valen decenas de miles). Por eso se usa una tasa diminuta (0.0001) y el sesgo aprende muy lento: la recta se ve razonable, pero no es la óptima. **Escalar los datos** (restar la media y dividir entre la desviación) hace el entrenamiento mucho más fácil.

B5. Red de una capa con varias salidas (12 min) — diapositivas 18–19

La idea (diapositiva 18)

Varias neuronas comparten **las mismas entradas**, y cada una da **su propia salida**. Por ejemplo, con datos del clima predecir a la vez la probabilidad de lluvia, la temperatura y la humedad.

- Cada entrada x_i se conecta con cada salida $\hat{y}_j$ por un peso w_{ij}. A esto se le llama capa **completamente conectada** (*fully connected*).
- Con m entradas y n salidas, los pesos forman una **matriz W de m filas × n columnas**, y hay **un sesgo por salida**: B = [b_1, …, b_n].

$$H = X\,W + B \qquad \hat{Y} = f(H)$$

La activación se aplica a cada elemento de H por separado.

Ejemplo resuelto (diapositiva 19)

X = [8, 4], W = [[−0.1, 0.6], [−0.8, 0.3]], B = [−0.7, 0.3], activación sigmoide.

- **Salida 1:** h_1 = 8·(−0.1) + 4·(−0.8) − 0.7 = −0.8 − 3.2 − 0.7 = **−4.7** → $\hat{y}_1$ = σ(−4.7) ≈ **0.0090**
- **Salida 2:** h_2 = 8·0.6 + 4·0.3 + 0.3 = 4.8 + 1.2 + 0.3 = **6.3** → $\hat{y}_2$ = σ(6.3) ≈ **0.9982**

> ⚠️ **Nota para el docente:** el libro trae este mismo ejemplo con H = [−7.7, 6.3] y ŷ = [$4.52\cdot10^{-4}$, $9.81\cdot10^{-1}$]. Las cuentas correctas son las de arriba. Puedes usarlo como reto: "encuentren el error del libro".

Número de parámetros: (m + 1)·n. Aquí (2 + 1)·2 = **6** (4 pesos + 2 sesgos).

PARTE C — Práctica en la computadora

Preparación (todos):

- VS Code → **Open Folder** → `curso-redes-neuronales`.
- Kernel **nni** en cada notebook.
- Ejecutar las celdas **en orden** con **Shift + Enter**.

C1. Notebook `notebooks/1_06_descenso_unpaso.ipynb` (20 min, 3 puntos) — diapositiva 11

Qué hay que hacer: los tres pasos de la regla delta, cada uno en su función.

1. **TODO (1 pt):** `error_term(y, x, neurona)` → δ = (y − ŷ)·f′(h).

- Pista: obtén h con `neurona.combinacion_lineal(x)`, ŷ con `neurona.forward(x)` y f′ con `sigmoid_prime(h)`.

2. **TODO (1 pt):** `incremento(eta, error_term, X)` → η·δ·X.
3. **TODO (1 pt):** `actualizacion(neurona, incremento_W, incremento_b)` → suma los incrementos a `neurona.W` y `neurona.b`.
4. En la celda final, `salida` es la predicción de la neurona: `neurona.forward(x)`.

Verificación (los números de B2):

Valor	Debe dar
Salida	≈ 0.5744
Término de error	≈ 0.1040
Incremento	≈ [0.2081, 0.2081]
Pesos actualizados	≈ [0.3081, 0.4081]

Después, vuelve a hacer la predicción: debe dar ≈ 0.7159, más cerca de y = 1 (la pérdida bajó).

C2. Notebook `notebooks/1_07_descenso_por_gradiente.ipynb` (25 min, 3 puntos) — diapositiva 16

Qué hay que hacer:

1. **TODO (1 pt):** inicializar `weights` con Xavier.
   - Pista: `np.random.uniform(-1/np.sqrt(n_features), 1/np.sqrt(n_features), size=n_features)`.
2. Ejecuta la celda de exactitud **antes** de entrenar: con pesos al azar sale baja.
3. **TODO (1 pt):** completar el ciclo, siguiendo el algoritmo de B3:
   - `output` → la predicción de `nodo` para `x`.
   - `error_term` → (y − output) por la derivada de la sigmoide. Recuerda que f′ = output·(1 − output).
   - `incremento_w +=` → el término de error por `x`.
   - Al terminar los ejemplos de la época: `weights +=` la tasa por el incremento acumulado, **dividido entre** `n_records`.
4. Grafica la pérdida: la curva debe **bajar**.
5. **(1 pt):** prueba distintas cantidades de épocas hasta superar **0.74** de exactitud.

Resultados de referencia (semilla 42, `learnrate = 0.5`):

Épocas	Pérdida al final	Exactitud
10	≈ 0.225	≈ 0.60
100	≈ 0.205	≈ **0.775** ✓
1000	≈ 0.197	≈ 0.725

Pregunta a la clase: "¿Por qué con 1000 épocas la pérdida baja pero la exactitud empeora?" → El conjunto de prueba es pequeño (40 alumnos) y la red empieza a ajustarse demasiado a los datos de entrenamiento. Es una primera probada de **sobreajuste**, que veremos más adelante.

⚠️ **Detalle importante:** escribe `weights += ...` y **no** `weights = weights + ...`. La neurona `nodo` comparte el mismo arreglo de pesos; con `+=` lo modificas "en el lugar" y `nodo` ve el cambio. Con `weights = weights + ...` creas un arreglo nuevo, `nodo` se queda con los pesos viejos y **la pérdida no baja**.

C3. Notebook `notebooks/1_08_adaline.ipynb` (demostración, sin puntos)

No tiene `TODO`: ejecútalo completo en vivo y comenta cada gráfica:

1. Los datos: más experiencia, más salario.
2. La recta antes de entrenar: al azar, muy mala.
3. La "superficie del error": el valle en 3D que el descenso tiene que bajar.
4. La pérdida durante el entrenamiento: baja.
5. La recta final.

Compárala con el resultado de `docs/practica_04.py`, que escala los datos (B4).

C4. Demo: `docs/practica_04.py --graficar` (5 min)

- OR se aprende: la pérdida cae a casi 0.
- **XOR no:** la pérdida se queda en 0.25. Una sola neurona sigue sin poder con XOR, aunque ahora use sigmoide y descenso por gradiente. Para resolverlo hace falta **otra capa**, y eso es el próximo módulo.

Quiz y cierre (10 min) — diapositivas 20–21

Quiz (en papel, 5 min):

Pregunta	Respuesta
¿Por qué el error se eleva al cuadrado?	Para que los errores + y − no se cancelen, y castigar más los grandes
¿Cuáles son los tres pasos de la regla delta?	Término de error, incremento, actualización
¿Qué añade la regla delta respecto a la del perceptrón?	La derivada de la activación f′(h)
¿Por qué no se inicializan los pesos en cero?	Las salidas serían iguales; hay que romper la simetría
¿Cuántos parámetros tiene una capa con 3 entradas y 2 salidas?	(3 + 1)·2 = 8

Para llevar:

1. Entrenar = bajar por la montaña del error moviendo los pesos.
2. Regla delta: $\delta = (y - \hat{y}) \cdot f'(h)$, $\Delta w = \eta \cdot \delta \cdot x$.
3. Una sola capa sigue sin resolver XOR → **redes multicapa** (próximo módulo).

Tarea:

1. Terminar `1_06` y `1_07` (6 puntos en total).
2. En `docs/practica_04.py`, cambia `eta` de la función `entrenar` a 0.1 y a 10: ¿qué pasa con la pérdida de OR?

3. Leer del libro: Redes neuronales multicapa.

Rúbrica:

Notebook	Puntos
1_06 Descenso, un paso	3
1_07 Descenso completo	3
1_08 ADALINE	Demostración, sin puntos

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'data_prep'` o `'nni'`	El notebook no se abrió desde el repo	Abre en VS Code la carpeta `curso-redes-neuronales` completa; en Colab, clona el repo y haz `%cd curso-redes-neuronales/notebooks`
`FileNotFoundError: data.csv`	Se ejecutó desde otra carpeta	Igual que arriba: el notebook debe correr dentro de `notebooks`
`TypeError: unsupported operand ... NoneType`	Un `TODO` sigue devolviendo `None`	Completa la función y vuelve a ejecutar desde esa celda
En `1_07` la pérdida no baja nada	Usaste `weights = weights + ...`	Usa `weights += ...` (ver C2)
La pérdida sube y baja (`WARNING - La pérdida ha aumentado`)	Tasa de aprendizaje muy grande	Baja `learnrate` (por ejemplo a 0.1)
La exactitud es 0.60 y no sube	Muy pocas épocas	Sube `epochs` a ~100
`ValueError: shapes not aligned`	`weights` con tamaño distinto al número de entradas	`size=n_features` en la inicialización
`1_08` da una recta distinta a la de practica_04	Datos sin escalar y pesos iniciales al azar	Es normal (ver B4)
Resultados distintos cada vez	Números al azar sin semilla	Ejecuta la celda con `np.random.seed(42)` antes

CLASE 4 · MÓDULO 3 (PARTE 2)

Entrenar una neurona

Regla delta, entrenamiento completo, ADALINE y redes de varias salidas

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 3
0:10	Medir el error
0:25	La regla delta
0:50	Práctica: un paso del descenso
1:10	Descanso
1:20	Entrenamiento completo e inicialización
1:35	Práctica: entrenar una neurona
2:00	ADALINE
2:08	Red con varias salidas
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Medir

el error de una red con el MSE.

2

Aplicar

un paso de la regla delta a mano.

3

Entrenar

una neurona completa y medir su exactitud.

4

Inicializar

los pesos al azar con Xavier.

5

Calcular

una capa con varias salidas.

Repaso de la Clase 3

¿Sigmoide?

Convierte h en un número entre 0 y 1, suave.

¿Derivada?

La pendiente: hacia dónde sube.

¿Descenso?

$x \leftarrow x - \eta \cdot f'(x)$

Hoy: bajar por el error de la red moviendo los pesos. Eso es entrenar.

Medir el error

Medida	Fórmula
Error residual	$e = y - \hat{y}$
SSE	$\Sigma\ (y - \hat{y})^2$
MSE (promedio)	$(1/m)\ \Sigma\ (y - \hat{y})^2$

¿Por qué al cuadrado? +1 y −1 no se cancelan, y los errores grandes pesan más.

Ejemplo

`y = [1, 0, 1]`

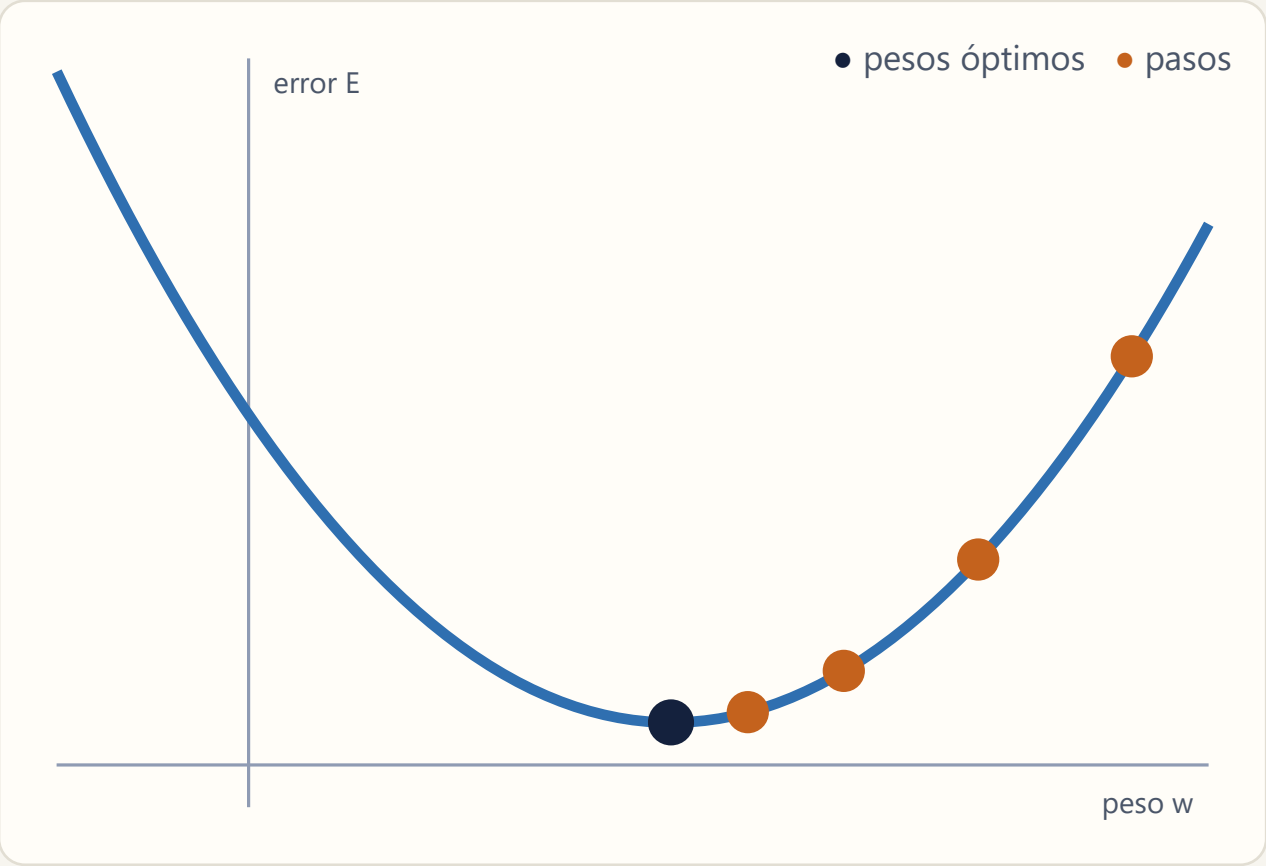
`ŷ = [0.8, 0.3, 0.6]`

errores²: 0.04, 0.09, 0.16

SSE = 0.29

MSE = 0.29 / 3 ≈ **0.0967**

La pérdida es una montaña



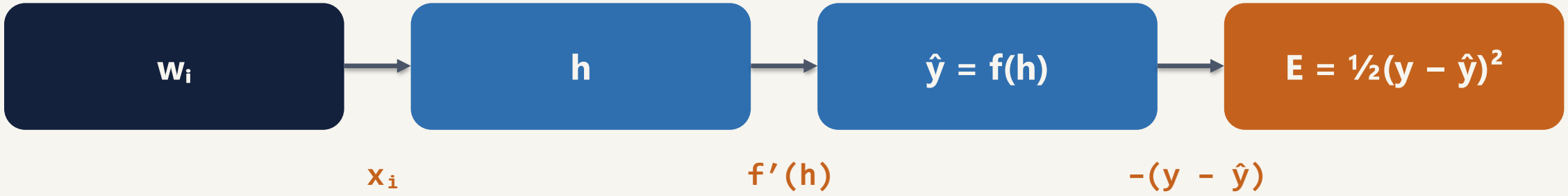
Entrenar = bajar al valle del error moviendo los pesos.

Es lo mismo de la clase 3:

x → **peso w**

f(x) → **error E**

¿Cuánto cambia el error si muevo un peso?



Regla de la cadena: se multiplican los efectos de cada eslabón

$$\partial E/\partial w_i = -(y - \hat{y}) \cdot f'(h) \cdot x_i$$

$$w \leftarrow w - \eta \cdot \partial E/\partial w \quad \Rightarrow \quad w_i \leftarrow w_i + \eta\,(y - \hat{y})\, f'(h)\, x_i$$

La regla delta en tres pasos

1 · Término de error

$$\delta = (y - \hat{y}) \cdot f'(h)$$

Cuánto y hacia dónde corregir

2 · Incremento

$$\Delta w_i = \eta \cdot \delta \cdot x_i$$

$\Delta b = \eta \cdot \delta$

3 · Actualización

$$w_i \leftarrow w_i + \Delta w_i$$

$b \leftarrow b + \Delta b$

Con la sigmoide: $f'(h) = \hat{y} \cdot (1 - \hat{y})$

Ejemplo: un paso de la regla delta

`x = [1, 1] · y = 1 · w = [0.1, 0.2] · b = 0 · η = 2 · sigmoide`

Paso	Cálculo
Predicción	$h = 0.3 \rightarrow \hat{y} = \sigma(0.3) \approx 0.5744$
Derivada	$f'(h) = 0.5744 \cdot 0.4256 \approx 0.2445$
Término de error	$\delta = (1 - 0.5744) \cdot 0.2445 \approx 0.1040$
Incremento	$\Delta w = 2 \cdot 0.1040 \cdot [1, 1] \approx [0.2081, 0.2081]$ · $\Delta b \approx 0.2081$
Actualización	$w \approx [0.3081, 0.4081]$ · $b \approx 0.2081$
Comprobar	$\hat{y} \approx 0.7159$ · pérdida 0.0905 → 0.0404 ✓

Perceptrón vs. regla delta

	Perceptrón (Clase 2)	Regla delta (hoy)
Activación	Escalón	Continua (sigmoide, lineal…)
Corrección	$\eta \cdot (y - \hat{y}) \cdot x$	$\eta \cdot (y - \hat{y}) \cdot \mathbf{f'(h)} \cdot x$
De dónde sale	Intuición de Rosenblatt	Descenso por gradiente
El error indica	Solo si acertó	Qué tan lejos estuvo

¡Casi la misma fórmula! La diferencia es la pendiente de la activación.

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_06_descenso_unpaso

TODO 1 pt error_term: $\delta = (y - \hat{y}) \cdot f'(h)$

TODO 1 pt incremento: $\eta \cdot \delta \cdot X$

TODO 1 pt actualizacion: sumar a W y b

Pistas: h con neurona.combinacion_lineal(x), ŷ con neurona.forward(x), f′ con sigmoid_prime(h).

```
# comprobaciones
Salida: 0.5744
Término de error: 0.1040
Incremento: [0.2081 0.2081]
Pesos actualizados: [0.3081 0.4081]
```

Al final: vuelve a predecir. Debe dar ≈ 0.7159, más cerca de y = 1.

Descanso · 10 min

Después: entrenar con todos los datos

El algoritmo completo

1. Inicializar los pesos al azar (Xavier).
2. Para cada época: incrementos acumulados = 0.
3. Para cada ejemplo: $\hat{y}$, δ y sumar $\delta \cdot x$ al acumulado.
4. Actualizar: $w \mathrel{+}= \eta \cdot \text{acumulado} / n_\text{ejemplos}$.
5. Medir el error (MSE) y repetir.
6. Al final: exactitud con datos de prueba.

Vocabulario

Época: una pasada por todos los datos.

Por lotes: una actualización por época con el promedio.

Prueba: datos que no se usaron para entrenar.

Inicialización de los pesos

¿Por qué no en cero?

Todas las salidas serían iguales: hay que romper la simetría.

¿Por qué pequeños?

Con pesos grandes la sigmoide se satura y aprende lento.

Xavier

$w \sim U(-1/\sqrt{n}, +1/\sqrt{n})$

n = número de entradas

Ejemplo: con n = 6 entradas, cada peso se elige al azar entre **−0.408 y +0.408**

Los datos de la práctica: admisiones

Columna	Qué es
gre	Puntaje del examen GRE (escalado)
gpa	Promedio (escalado)
rank_1 … rank_4	Prestigio de la universidad de origen (0/1)
admit	Salida: 1 admitido, 0 no

400 alumnos · 6 entradas

90 % entrenamiento (360) · 10 % prueba (40)

Exactitud

`aciertos / predicciones`

La red dice “admitido” si $\hat{y} > 0.5$.

Meta del notebook: **más de 0.74**

NOTEBOOKS · 25 MIN · 3 PUNTOS

Práctica · 1_07_descenso_por_gradiente

TODO 1 pt pesos con Xavier (np.random.uniform)

TODO 1 pt completar el ciclo de entrenamiento

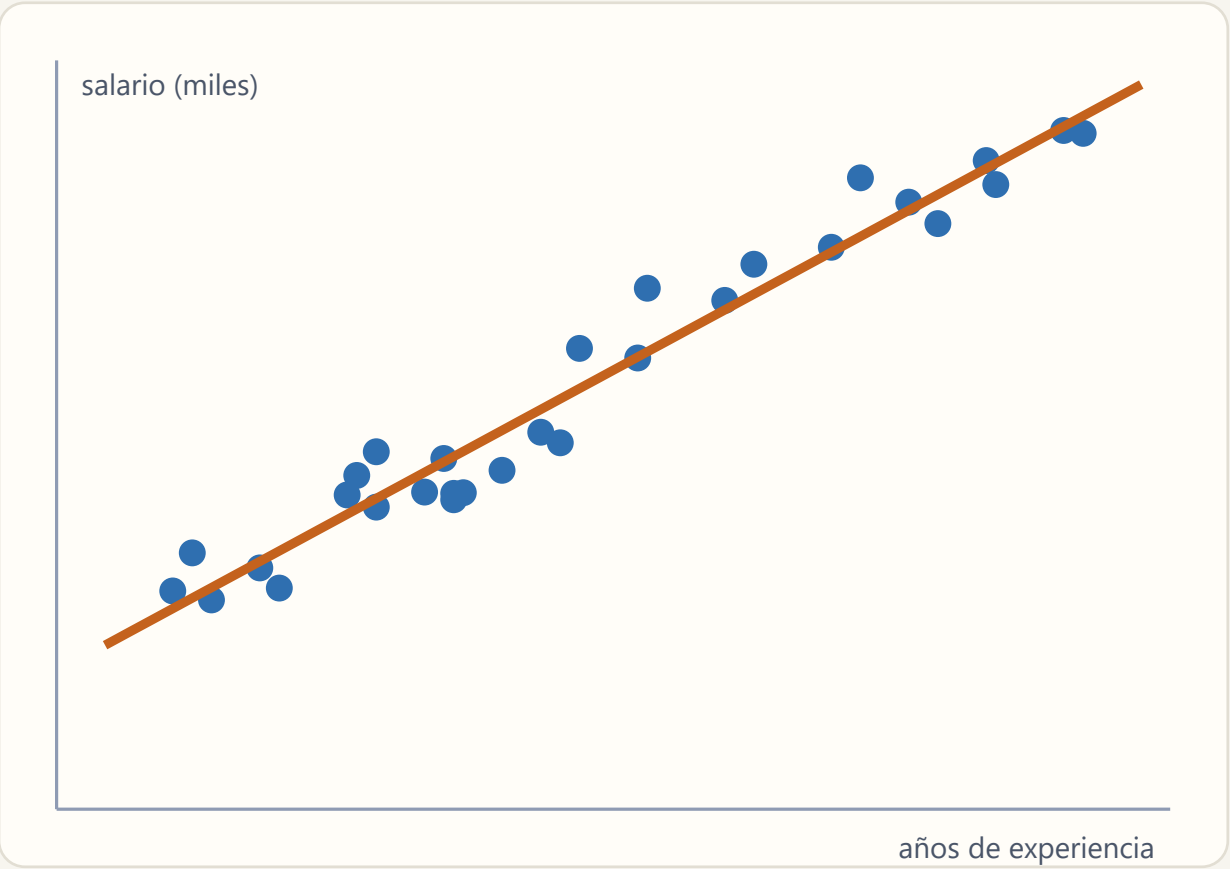
1 pt épocas hasta exactitud > 0.74

Pista: sigue los pasos 3 y 4 del algoritmo (diapositiva 13).

```
# referencia (semilla 42, learnrate 0.5)
# épocas  pérdida  exactitud
#   10     0.225    0.60
#  100     0.205    0.775
# 1000     0.197    0.725
```

Usa `weights += …` y no weights = weights + …: la neurona comparte el arreglo; si creas uno nuevo, la pérdida no baja.

ADALINE: regresión con una neurona



Activación lineal: $\hat{y} = w \cdot x + b$

Derivada: $f'(h) = 1$

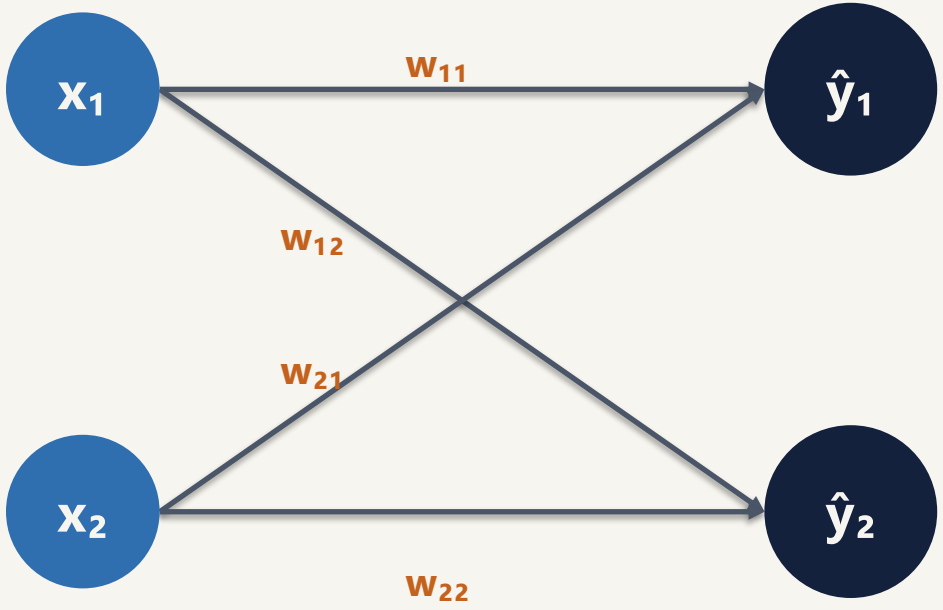
Regla: $w \leftarrow w + \eta(y - \hat{y})x$

salario ≈ 9 450 · años + 24 848

5 años → ≈ 72 098

Lección: escalar los datos acelera mucho el entrenamiento.

Red de una capa con varias salidas



$$H = X \cdot W + B \qquad \hat{Y} = f(H)$$

W: matriz de m entradas × n salidas.

B: un sesgo por salida.

f: se aplica a cada elemento de H.

Parámetros: $(m + 1) \cdot n$

Cada entrada se conecta con cada salida: “completamente conectada”.

Ejemplo: 2 entradas, 2 salidas

`X = [8, 4] · W = [[−0.1, 0.6], [−0.8, 0.3]] · B = [−0.7, 0.3] · sigmoide`

Salida	Combinación lineal	Sigmoide
$\hat{y}_1$	$8 \cdot (-0.1) + 4 \cdot (-0.8) - 0.7 = -4.7$	$\sigma(-4.7) \approx 0.0090$
$\hat{y}_2$	$8 \cdot 0.6 + 4 \cdot 0.3 + 0.3 = 6.3$	$\sigma(6.3) \approx 0.9982$

Parámetros: `(2 + 1) · 2 = 6`

Reto: el libro dice H = [−7.7, 6.3]. ¿Dónde está el error?

Quiz · 5 minutos

1. ¿Por qué el error se eleva al cuadrado?
2. ¿Cuáles son los tres pasos de la regla delta?
3. ¿Qué añade la regla delta respecto a la del perceptrón?
4. ¿Por qué no se inicializan los pesos en cero?
5. ¿Cuántos parámetros tiene una capa con 3 entradas y 2 salidas?

Para llevar

1. Entrenar = bajar por el error moviendo los pesos.
2. Regla delta: $\delta = (y - \hat{y}) \cdot f'(h)$, $\Delta w = \eta \cdot \delta \cdot x$.
3. Una sola capa sigue sin resolver XOR.

Próximo módulo: redes multicapa y retropropagación.

Tarea: notebooks 1_06 y 1_07 (6 pts)

Leer: libro, “Redes neuronales multicapa”

Guía del notebook `1_06_descenso_unpaso`: un paso de la regla delta

Puntos: 3 · **Tiempo:** 20 min · **Clase:** 4 · **Archivo:** `notebooks/1_06_descenso_unpaso.ipynb`
Teoría completa: Guía de la Clase 4, sección B2.

¿Qué vas a aprender?

- Los tres pasos con los que una neurona corrige sus pesos: **término de error**, **incremento** y **actualización**.
- A comprobar que, después de un paso, la neurona se equivoca **menos**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_06_descenso_unpaso.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos (regla delta)

Paso	Fórmula	En palabras
1. Término de error	$\delta = (y - \hat{y}) \cdot f'(h)$	Cuánto y hacia dónde corregir
2. Incremento	$\Delta w = \eta \cdot \delta \cdot x$ ($\Delta b = \eta \cdot \delta$)	Cuánto mover cada peso
3. Actualización	$w \leftarrow w + \Delta w$, $b \leftarrow b + \Delta b$	Mover los pesos

Con la sigmoide, $f'(h) = \sigma(h) \cdot (1 - \sigma(h))$. El notebook ya la trae como `sigmoid_prime`.

Paso a paso

Celda: `sigmoid`, `sigmoid_prime` y la clase `Neurona`

Ya están hechas. Ejecútala.

Celda `TODO (1 punto)`: `error_term(y, x, neurona)`

Pistas:

- h se obtiene con `neurona.combinacion_lineal(x)`;
- ŷ, con `neurona.forward(x)`;
- $f'(h)$, con `sigmoid_prime(h)`;
- multiplica $(y - \hat{y})$ por $f'(h)$.

Celda `TODO (1 punto)`: `incremento(eta, error_term, X)`

Devuelve η·δ·X.

Celda `TODO (1 punto)`: `actualizacion(neurona, incremento_W, incremento_b)`

Suma el incremento a `neurona.W`, y el del sesgo a `neurona.b`. Devuelve la neurona.

Celda "valores de ejemplo"

Usa tasa 2.0, x = [1, 1], y = 1, w = [0.1, 0.2] y b = 0.

Celda "Calcular la salida de la red"

Cambia `salida = None` por la predicción de la neurona, `neurona.forward(x)`. Lo demás ya llama a tus funciones.

✅ **Comprobación:**

Línea	Debe imprimir
Salida	≈ 0.5744
Término de error	≈ 0.1040
Incremento	≈ [0.2081 0.2081]
Incremento del sesgo	≈ 0.2081
Pesos actualizados	≈ [0.3081 0.4081]

Último punto (1 punto): ¿bajó la pérdida?

Agrega una celda que vuelva a predecir con la neurona actualizada y compare la pérdida ½(y − ŷ)² antes y después.

✅ **Comprobación:** la nueva salida es ≈ **0.7159**, más cerca de y = 1. La pérdida baja de **0.0905** a **0.0404**.

Errores frecuentes

Síntoma	Causa	Solución
Término de error ≈ −0.104	Escribiste (ŷ − y) en lugar de (y − ŷ)	El orden es **y − ŷ**
Incremento del sesgo con dos números	Pasaste `x` en lugar de `1`	Para el sesgo, la "entrada" es 1 (la celda ya lo hace)
`TypeError: ... NoneType`	Alguna función devuelve `None`	Completa las tres funciones y vuelve a ejecutarlas
Los pesos no cambian	`actualizacion` no guarda en `neurona.W`	Asigna `neurona.W = neurona.W + incremento_W`

Guía del notebook `1_07_descenso_por_gradiente`: entrenar una neurona completa

Puntos: 3 · **Tiempo:** 25–30 min · **Clase:** 4 · **Archivo:** `notebooks/1_07_descenso_por_gradiente.ipynb`
Teoría completa: Guía de la Clase 4, sección B3.

¿Qué vas a aprender?

- A entrenar una neurona con **todos** los ejemplos, durante muchas **épocas**.
- A inicializar los pesos con **Xavier**.
- A medir la **exactitud** con datos que la red no vio al entrenar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_07_descenso_por_gradiente.ipynb` → kernel **nni**.
   Usa la carpeta del repo: el notebook carga `data.csv`, `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Datos:** 400 alumnos que pidieron entrar a un posgrado.
  - Entradas: puntaje GRE, promedio GPA y el prestigio de su universidad (`rank`, en 4 columnas de 0/1). Son **6 entradas**.
  - Salida: `admit`, que vale 1 si fue admitido.
  - `data_prep.py` ya los escala y separa **360 para entrenar** y **40 para probar**.
- **Algoritmo por época:**
  1. Para cada alumno: predicción $\hat{y}$, término de error $\delta = (y - \hat{y})\cdot\hat{y}(1 - \hat{y})$, y **acumular** $\delta \cdot x$.
  2. Al final de la época: pesos += η · acumulado / número de alumnos.
- **Xavier:** pesos al azar entre $-1/\sqrt{n}$ y $+1/\sqrt{n}$ ($n = 6$ → entre −0.408 y +0.408).
- **Exactitud** = aciertos / predicciones. La red dice "admitido" si $\hat{y} > 0.5$.

Paso a paso

Celda 1: paquetes y datos

Carga todo y fija la semilla 42. Si da error, revisa que estés en la carpeta del repositorio.

Celda `TODO (1 punto)`: inicializar con Xavier

Pista: `np.random.uniform(-1/np.sqrt(n_features), 1/np.sqrt(n_features), size=n_features)`.

Celda: exactitud antes de entrenar

✅ **Comprobación:** con la semilla 42 sale **0.650**. Es casi lo mismo que adivinar "no admitido" siempre.

Celda: hiperparámetros

`epochs = 10` y `learnrate = 0.5`. Más adelante subirás las épocas.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los cuatro huecos siguiendo el algoritmo:

1. `output` → `nodo.forward(x)`.
2. `error_term` → (y − output) · output · (1 − output).
3. `incremento_w +=` → error_term · x.
4. Después del `for` de ejemplos: `weights +=` tasa · incremento acumulado, **dividido entre** `n_records`.

⚠️ Escribe `weights += ...` y no `weights = weights + ...`. La neurona `nodo` comparte el arreglo `weights`: con `+=` lo modificas "en el lugar" y la neurona ve el cambio. Con `=` creas un arreglo nuevo, la neurona se queda con los pesos viejos y **la pérdida no baja**.

Celda: gráfica de la pérdida

Debe bajar época tras época.

Celda: exactitud final (1 punto si supera 0.74)

✅ **Comprobación** (semilla 42, `learnrate = 0.5`):

Épocas	Pérdida al final	Exactitud
10	≈ 0.225	≈ 0.60
100	≈ **0.2055**	**0.775** ✓
1000	≈ 0.197	≈ 0.725

Para pensar

¿Por qué con 1000 épocas la pérdida baja pero la exactitud empeora? El conjunto de prueba es pequeño (40 alumnos) y la red se ajusta demasiado a los datos de entrenamiento: es **sobreajuste**.

Errores frecuentes

Síntoma	Causa	Solución
La pérdida no baja nada	`weights = weights + ...`	Usa `weights += ...`
`ModuleNotFoundError: data_prep` o `nni`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
`ValueError: shapes not aligned`	`size` distinto de `n_features`	`size=n_features`
`WARNING - La pérdida ha aumentado`	Tasa muy grande	Baja `learnrate` a 0.1

Síntoma	Causa	Solución
Exactitud distinta a la tabla	No ejecutaste la celda con `np.random.seed(42)`	Ejecuta todo desde la primera celda

Guía del notebook `1_08_adaline`: regresión con una neurona

Puntos: demostración, sin `TODO` · **Tiempo:** 10–15 min · **Clase:** 4 · **Archivo:** `notebooks/1_08_adaline.ipynb`
Teoría completa: Guía de la Clase 4, sección B4.

¿Qué vas a aprender?

- Que una neurona con **activación lineal** (ADALINE) sirve para **predecir números** (regresión).
- A ver el error como una **superficie** (un valle en 3D) que el descenso por gradiente tiene que bajar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_08_adaline.ipynb` → kernel **nni**. El notebook lee `files/Salary_dataset.csv`, así que hay que abrirlo desde la carpeta del repo.
3. Este notebook **no tiene** `TODO`: ejecútalo completo y lee cada resultado.

La idea en 2 minutos

- **ADALINE** (Widrow y Hoff, 1960): $\hat{y} = w \cdot x + b$, sin función escalón ni sigmoide. Es una **recta**.
- **Error:** el error cuadrático medio (MSE) entre los salarios reales y los predichos.
- **Regla** (descenso por gradiente con $f' = 1$): $w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$ y $b \leftarrow b + \eta \cdot (y - \hat{y})$.

Recorrido por las celdas

1. **Datos:** 30 personas, con sus años de experiencia y su salario. La gráfica muestra que a más experiencia, más salario.
2. **Clase `ADALINE`:** pesos al azar entre 0 y 1; `predict` calcula $w \cdot x + b$.
3. **Recta sin entrenar:** casi plana y pegada a 0, muy lejos de los puntos (los salarios valen decenas de miles).
4. **Superficie del error:** para muchas combinaciones de peso y sesgo calcula el MSE y lo dibuja en 3D. El punto más bajo de ese valle son los mejores parámetros.
5. **Entrenamiento:** 1000 pasadas con tasa 0.0001, ejemplo por ejemplo.
6. **Historial del MSE:** baja rápido y luego se aplana.
7. **Recta final:** pasa por en medio de los puntos.

✅ **Qué esperar:**

- El resultado cambia un poco cada vez, porque los pesos iniciales son al azar (no hay semilla). En una ejecución salió $w \approx 8\,400$ y $b \approx 32\,000$.
- La recta **óptima** (mínimos cuadrados) es **salario ≈ 9 450 · años + 24 848**. `docs/practica_04.py` llega exactamente a ella.

Para pensar

¿Por qué el notebook no llega exactamente a la recta óptima?

- Los datos no están **escalados**: los años van de 1 a 10.6 y los salarios de 37 000 a 122 000.
- Por eso hay que usar una tasa diminuta (0.0001), y con ella el **sesgo** aprende muy lento.
- Si se escalan los datos (restar la media y dividir entre la desviación estándar), el descenso llega al óptimo rápido. Eso hace `docs/practica_04.py`.

Pruébalo:

```
python docs/practica_04.py --graficar
```

Errores frecuentes

Síntoma	Causa	Solución
`FileNotFoundError: files/Salary_dataset.csv`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
La gráfica 3D no aparece	Falta matplotlib, o la celda dio error	`pip install matplotlib` con `(nni)` activo
Recta distinta a la de un compañero	Pesos iniciales al azar	Es normal; agrega `np.random.seed(0)` al principio para repetir resultados

Guía paso a paso — Clase 5: Redes multicapa y retropropagación

Temas: capas ocultas · pesos como matrices · pase hacia adelante · retropropagación · entrenar una red multicapa · ¡XOR resuelto!
Duración: ~2 h 30 min
Material: diapositivas `Clase5_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clases 3 y 4 (sigmoide, descenso por gradiente, regla delta)
Lecturas base (libro del Dr. Irving Vasquez):

- Redes neuronales multicapa
- Retropropagación

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar por qué una capa oculta permite resolver problemas como XOR.
2. Calcular el pase hacia adelante de una red de dos capas con matrices, cuidando los tamaños.
3. Aplicar a mano un paso de la **retropropagación**.
4. Entrenar una red multicapa en Python y leer su curva de aprendizaje.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 4 (10 min)	1–4
0:10	B1. ¿Por qué capas ocultas? (15 min)	5–6
0:25	B2. Pesos como matrices y pase hacia adelante (20 min)	7–9
0:45	C1. Práctica: notebook `1_09` (20 min)	10
1:05	Descanso (10 min)	11
1:15	B3. Retropropagación (30 min)	12–16
1:45	C2. Práctica: notebook `1_10` (15 min)	17

Tiempo	Tema	Diapositivas
2:00	C3. Entrenar la red completa y resolver XOR (20 min)	18–19
2:20	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_05.py
```

Verificación: la red aprende XOR. Deben salir las cuatro predicciones `0, 1, 1, 0`, la pérdida bajando de ≈ 0.26 a ≈ 0.0006, y al final `¡La red multicapa resuelve XOR!`.

Para ver la curva y la frontera que aprendió la red:

```
python docs/practica_05.py --graficar
```

Checklist:

- [ ] `python docs/practica_05.py` funciona.
- [ ] VS Code abre `notebooks/1_09_red_multicapa_ejercicio.ipynb` con el kernel **nni**.
- [ ] Tienes a mano las guías de los notebooks `1_09`, `1_10` y `1_11` (en `docs/guias_notebooks/` o en `entregables/Guias_de_los_notebooks.pdf`).
- [ ] En el pizarrón: el plano con los 4 puntos de XOR. Lo usarás para dibujar dos rectas.

PARTE B — Teoría desde cero

Repaso de la Clase 4 (5 min) — diapositivas 1–4

- "¿Cuáles son los tres pasos de la regla delta?" → término de error, incremento y actualización.
- "¿Qué problema quedó sin resolver?" → **XOR**: una sola neurona solo traza una recta, y XOR no se separa con una recta.

Hoy lo resolvemos.

B1. ¿Por qué capas ocultas? (15 min) — diapositivas 5–6

Dos rectas en lugar de una (diapositiva 5)

En el plano de XOR, los puntos (0, 1) y (1, 0) valen 1; los puntos (0, 0) y (1, 1) valen 0. Dibuja **dos** rectas:

- la recta A deja **abajo** solo el punto (0, 0) (como un OR);
- la recta B deja **arriba** solo el punto (1, 1) (como un AND).

Los puntos que valen 1 están **entre** las dos rectas. Entonces:

1. una neurona calcula la recta A;
2. otra neurona calcula la recta B;
3. una **tercera** neurona combina ambas: "arriba de A **y** debajo de B".

Las dos primeras forman la **capa oculta**; la tercera es la **capa de salida**.

Idea clave: cada neurona oculta traza una recta. La salida combina esas rectas y puede formar regiones con forma de franja, de esquina o de cualquier figura.

La arquitectura (diapositiva 6)

- **Capa de entrada:** los datos (no calcula nada).
- **Capa oculta:** neuronas cuya salida no vemos directamente. De ahí su nombre.
- **Capa de salida:** la predicción.
- Esta red se llama **perceptrón multicapa** (MLP). Si tiene muchas capas ocultas, hablamos de **aprendizaje profundo**.

B2. Pesos como matrices y pase hacia adelante (20 min) — diapositivas 7–9

Pesos como matrices (diapositiva 7)

Red del notebook `1_09`: **4 entradas → 3 ocultas → 2 salidas**.

- Entre la entrada y la oculta hay 4 × 3 = 12 pesos. Van en una matriz $\mathbf{W_1}$ de **4 filas × 3 columnas**: filas = entradas, columnas = neuronas de la capa siguiente.
- Entre la oculta y la salida: $\mathbf{W_2}$ de **3 × 2**.

Regla de los tamaños: (a × b)·(b × c) = (a × c). El número del medio debe coincidir. Por ejemplo, (1 × 4)·(4 × 3) = (1 × 3).

Pase hacia adelante (diapositiva 8)

Es la Clase 4 dos veces:

Paso	Fórmula	Tamaño
Capa oculta	$H_1 = X \cdot W_1 + B_1$ y $A_1 = \sigma(H_1)$	1 × 3
Capa de salida	$H_2 = A_1 \cdot W_2 + B_2$ y $\hat{Y} = \sigma(H_2)$	1 × 2

La salida de una capa es la entrada de la siguiente.

Contar parámetros (diapositiva 9)

- **Pesos:** 4·3 + 3·2 = 18.
- **Sesgos:** 3 + 2 = 5.
- **Total: 23 parámetros**.

Regla general: por cada capa, (entradas + 1) × neuronas. Aquí: (4 + 1)·3 + (3 + 1)·2 = 15 + 8 = 23.

B3. Retropropagación (30 min) — diapositivas 12–16

El problema (diapositiva 12)

Con una neurona sabíamos su error: y − ŷ. Pero **¿cuál es el error de una neurona oculta?** Nadie nos dice qué debió producir.

La solución: calcular el error en la salida y **mandarlo hacia atrás**. Cada neurona oculta recibe una parte proporcional al peso que la une con la salida: si influyó mucho, carga más culpa.

Analogía: un equipo pierde un partido. El entrenador reparte la responsabilidad según cuánto participó cada jugador en la jugada decisiva.

Las fórmulas (diapositiva 13)

Con la sigmoide, f′ = salida·(1 − salida).

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y})\cdot\hat{y}(1 - \hat{y})$	$\Delta w_ho = \eta\cdot\delta_o\cdot a_h$
Oculta	$\delta_h = \delta_o\cdot w_ho\cdot a_h(1 - a_h)$	$\Delta w_ih = \eta\cdot\delta_h\cdot x_i$

- a_h es la salida de la neurona oculta.
- La fórmula de la oculta es la misma regla delta, pero su "error" es **δ_o multiplicado por el peso que la une con la salida**.

Viene de la regla de la cadena (Clase 4): el error depende de w_ih a través de h_h, luego de a_h, luego de la salida. Solo hay un eslabón más.

Ejemplo resuelto: pase hacia adelante (diapositiva 14)

Red 3 → 2 → 1 del notebook `1_10`:

- x = [0.5, 0.1, −0.2], objetivo = 0.6, η = 0.5;
- W_ih = [[0.5, −0.6], [0.1, −0.2], [0.1, 0.7]] y W_ho = [0.1, −0.3].

Valor	Cálculo	Resultado
Entrada a la oculta	x·W_ih	[0.24, −0.46]
Salida oculta a_h	σ(…)	[0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3)	−0.0601
Salida ŷ	σ(−0.0601)	**0.4850**

Ejemplo resuelto: hacia atrás (diapositiva 15)

Valor	Cálculo	Resultado
Error	0.6 − 0.4850	0.1150
δ_o	0.1150 · 0.4850 · 0.5150	**0.02873**
δ_h	0.02873 · [0.1, −0.3] · a_h(1 − a_h)	**[0.000708, −0.002045]**
Δw_ho	0.5 · 0.02873 · [0.5597, 0.3870]	**[0.00804, 0.00556]**
Δw_ih	0.5 · δ_h · x (matriz 3 × 2)	≈ [[0.000177, −0.000511], …]

Los incrementos tienen **el mismo tamaño** que los pesos: cada peso necesita su propio incremento.

El gradiente que se desvanece (diapositiva 16)

Los incrementos de la capa oculta son **mucho más pequeños** que los de la salida. El error llega multiplicado por pesos pequeños y por derivadas de la sigmoide, que valen como mucho 0.25.

Con muchas capas, esto se repite y las primeras capas casi no aprenden. Es el problema del **gradiente que se desvanece**, y una razón por la que las redes profundas usan ReLU.

El algoritmo completo (diapositiva 18)

Para cada época:

1. Poner en cero los incrementos acumulados.
2. Para cada ejemplo: pase hacia adelante → errores δ_o y δ_h → acumular los incrementos.
3. Actualizar las dos matrices de pesos con η · acumulado (o con el promedio).
4. Medir la pérdida.

PARTE C — Práctica en la computadora

Para cada notebook, la guía detallada (celda por celda, con pistas y valores) está en `docs/guias_notebooks/`.

C1. `notebooks/1_09_red_multicapa_ejercicio.ipynb` (20 min, 3 puntos) — diapositiva 10

- Tamaños de las capas: 4, 3 y 2.
- W_1 de (4, 3) y W_2 de (3, 2), con `np.random.normal`, media 0 y desviación 1/√4.
- Pase hacia adelante con `np.dot`. Ojo: la segunda capa usa **A_1**, no H_1.
- Celda extra con sesgos B_1 (1 × 3) y B_2 (1 × 2).

✅ **Comprobación:**

- la capa oculta da 3 números entre 0 y 1, y la salida 2;
- en total hay 23 parámetros;
- con la semilla 42 la salida es ≈ [0.3210, 0.3627].

Guía: NB_1_09.

C2. `notebooks/1_10_retropropagacion_base.ipynb` (15 min, 3 puntos) — diapositiva 17

Los valores de B3 son exactamente los que deben salir con η = 0.5.

Error típico: en Δw_ih hay que convertir x en columna con `x[:, None]` para obtener la matriz 3 × 2.

Guía: NB_1_10.

C3. `notebooks/1_11_retropropagacion_completo.ipynb` y XOR (20 min, 3 puntos) — diapositivas 18–19

Datos de admisiones (los de `1_07`), red 6 → 2 → 1, 900 épocas, semilla 21:

Actualización	Pérdida	Exactitud
0.001 · acumulado (plantilla)	0.2646 → 0.2013	0.725
0.5 · acumulado / n_records	0.2641 → 0.1999	**0.750**

⚠️ El notebook original imprime la pérdida usando solo el último ejemplo (`x`). Cámbialo por `features.values`.

Cierre en vivo:

```
python docs/practica_05.py --graficar
```

Una red 2 → 3 → 1 aprende XOR: predice 0.025, 0.981, 0.971 y 0.026, que redondeados dan 0, 1, 1 y 0. La frontera que aprendió **ya no es una recta**.

Guía: NB_1_11.

Quiz y cierre (10 min) — diapositivas 20–21

Pregunta	Respuesta
¿Por qué una capa oculta permite resolver XOR?	Cada neurona oculta traza una recta y la salida las combina
¿Qué tamaño tiene W entre una capa de 4 y una de 3?	4 × 3
¿Cuántos parámetros tiene una red 2 → 3 → 1?	(2+1)·3 + (3+1)·1 = 13
¿Cómo se calcula el error de una neurona oculta?	$\delta_h = \delta_o \cdot$ peso hacia la salida · su derivada
¿Qué es el gradiente que se desvanece?	El error se hace muy pequeño al ir hacia atrás por muchas capas

Para llevar:

1. Las capas ocultas combinan rectas y resuelven problemas no lineales.
2. Pase hacia adelante = la Clase 4 repetida capa por capa.
3. Retropropagación = mandar el error hacia atrás, repartido según los pesos.

Tarea:

1. Terminar `1_09`, `1_10` y `1_11` (9 puntos).
2. En `docs/practica_05.py`, cambia `n_ocultas` a 2 y a 8: ¿sigue aprendiendo XOR? Cambia también la `semilla`.
3. Leer del libro: Retropropagación.

Rúbrica:

Notebook	Puntos
1_09	3
1_10	3
1_11	3

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`ValueError: shapes ... not aligned`	Orden de las matrices o tamaños mal puestos	Sigue la regla (a × b)·(b × c); usa `.shape` para revisar
`operands could not be broadcast together` en Δw_ih	x no está como columna	`x[:, None]`
Los signos de los δ salen al revés	Usaste ŷ − y	Es **y − ŷ**
`1_11` no aprende (exactitud ≈ 0.42)	Dividiste entre n_records con una tasa muy pequeña (0.005)	Tasa 0.5 si divides, o 0.001 sin dividir
La pérdida impresa en `1_11` salta	El original usa solo el último ejemplo	Usa `features.values`
`practica_05.py` no aprende XOR con otra semilla	Algunas inicializaciones se atascan	Prueba otra semilla o más neuronas ocultas (pasa también en redes reales)

CLASE 5 · MÓDULO 4

Redes multicapa y retropropagación

Capas ocultas, pase hacia adelante, retropropagación… ¡y XOR resuelto!

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 4
0:10	¿Por qué capas ocultas?
0:25	Pesos como matrices y pase hacia adelante
0:45	Práctica: 1_09
1:05	Descanso
1:15	Retropropagación
1:45	Práctica: 1_10
2:00	Entrenar la red completa y resolver XOR
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Explicar

por qué una capa oculta resuelve XOR.

2

Calcular

el pase hacia adelante con matrices.

3

Aplicar

un paso de retropropagación a mano.

4

Entrenar

una red multicapa en Python.

Repaso de la Clase 4

¿Regla delta?

Término de error, incremento y actualización.

¿Varias salidas?

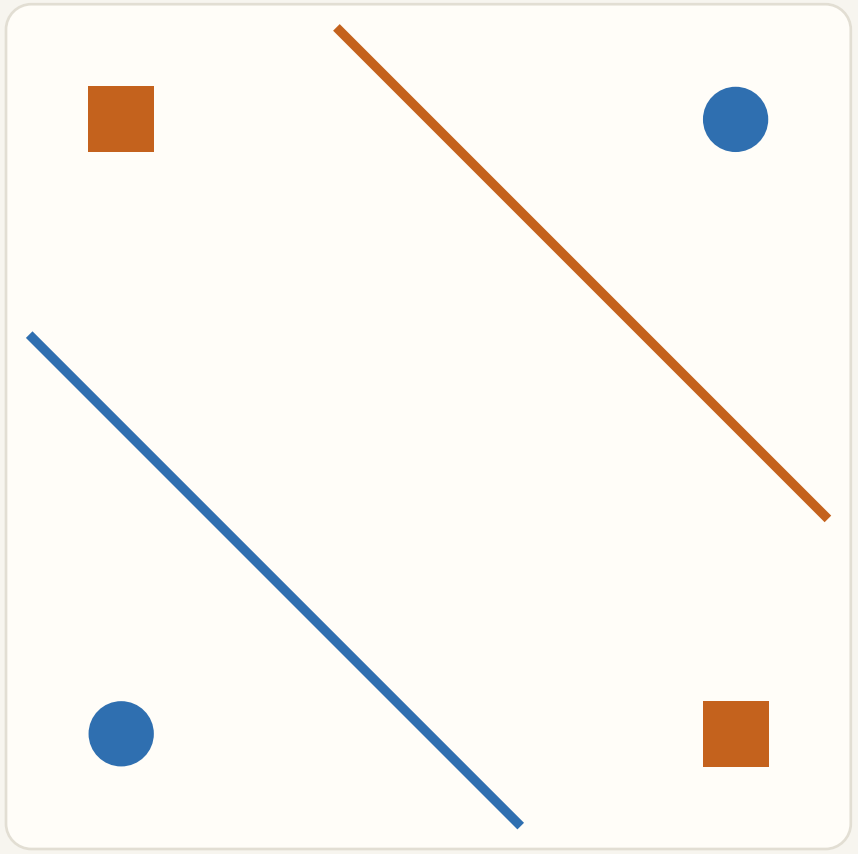
H = X·W + B: una matriz de pesos.

¿Qué quedó pendiente?

XOR: una neurona solo traza una recta.

Hoy: juntar neuronas en capas para resolver XOR.

XOR con dos rectas



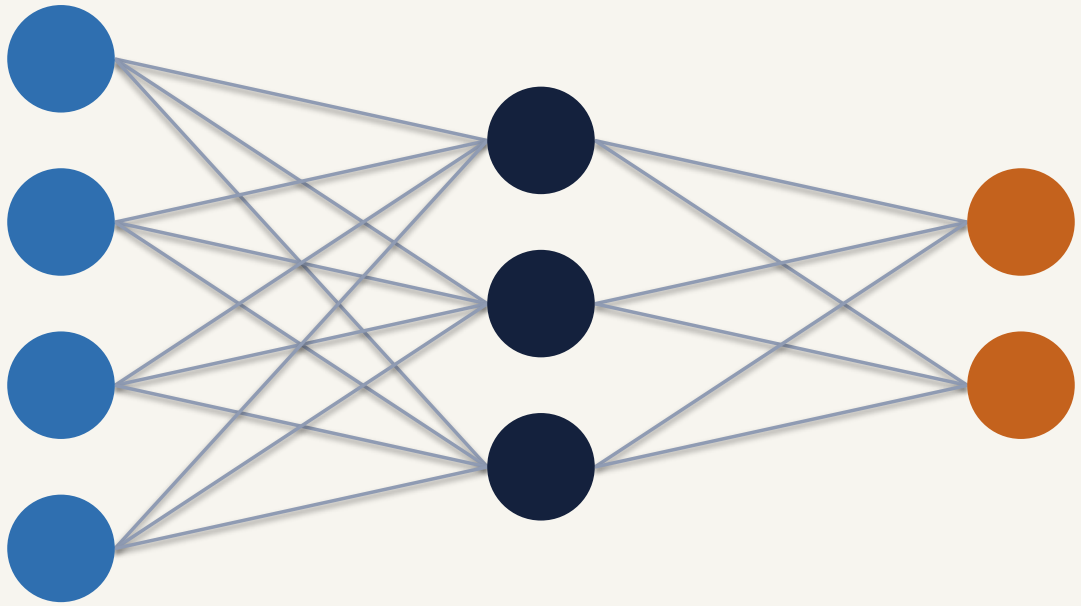
Neurona oculta 1: recta A (deja abajo solo el (0, 0)).

Neurona oculta 2: recta B (deja arriba solo el (1, 1)).

Neurona de salida: “arriba de A y debajo de B”.

Cada neurona oculta traza una recta; la salida las combina en una franja.

Perceptrón multicapa (MLP)



Entrada (4) **Oculta (3)** **Salida (2)**

Entrada: los datos; no calcula nada.

Oculta: su salida no se ve desde afuera.

Salida: la predicción.

Muchas capas ocultas = aprendizaje profundo.

Los pesos son matrices

Matriz	Conecta	Tamaño
W_1	entrada (4) → oculta (3)	4 × 3
W_2	oculta (3) → salida (2)	3 × 2
B_1	un sesgo por neurona oculta	1 × 3
B_2	un sesgo por neurona de salida	1 × 2

$$(a \times b) \cdot (b \times c) = (a \times c)$$

el número del medio debe coincidir

Ejemplo:

$$(1 \times 4) \cdot (4 \times 3) = (1 \times 3)$$

una fila de 3 números: las 3 neuronas ocultas.

Pase hacia adelante

Capa oculta

$$H_1 = X \cdot W_1 + B_1$$

$$A_1 = \sigma(H_1)$$

tamaño: 1 × 3

Capa de salida

$$H_2 = A_1 \cdot W_2 + B_2$$

$$\hat{Y} = \sigma(H_2)$$

tamaño: 1 × 2

La salida de una capa (A_1) es la entrada de la siguiente.

Contar parámetros

Capa	Pesos	Sesgos	Total
4 → 3	4·3 = 12	3	15
3 → 2	3·2 = 6	2	8
Red 4 → 3 → 2	18	5	**23**

Regla por capa:

`(entradas + 1) × neuronas`

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_09_red_multicapa_ejercicio

TODO 1 pt tamaños 4, 3, 2 y matrices W_1, W_2 con np.random.normal

TODO 1 pt pase hacia adelante con np.dot

1 pt agregar sesgos B_1 (1×3) y B_2 (1×2)

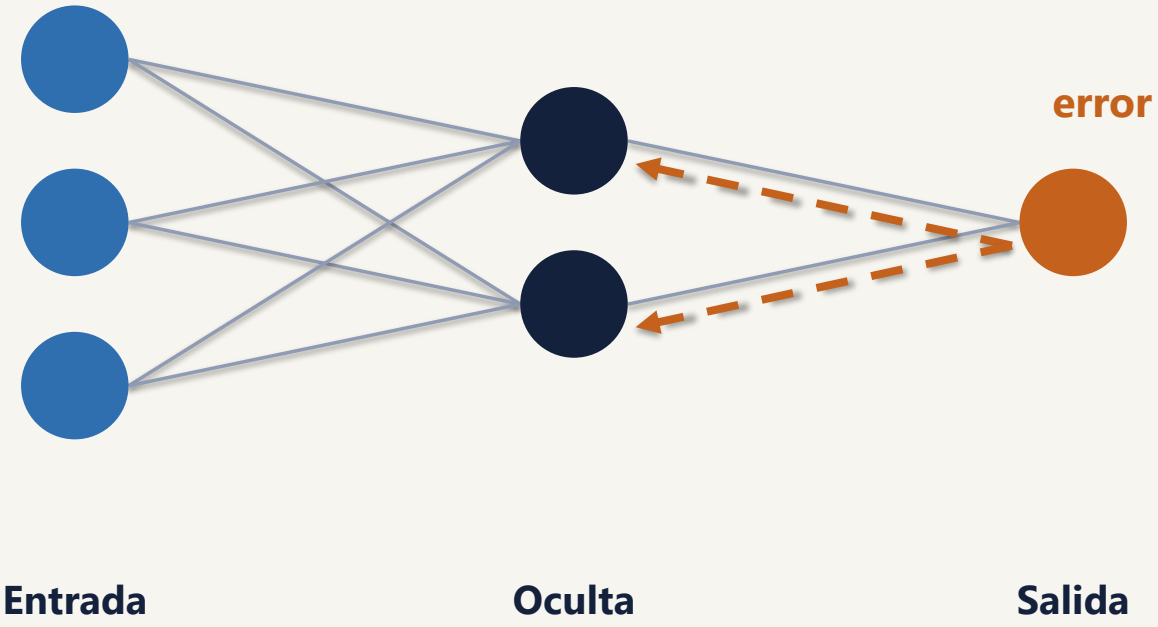
Pista: la segunda capa usa A_1 (después de la sigmoide), no H_1. Desviación sugerida: 1/√4.

```
# comprobaciones
W_1: (4, 3)  W_2: (3, 2)  X: (1, 4)
oculta: 3 números entre 0 y 1
salida: 2 números entre 0 y 1
parámetros con sesgos: 23
```

Descanso · 10 min

Después: la retropropagación

¿Cuál es el error de una neurona oculta?



Nadie nos dice qué debió producir una neurona oculta.

Solución: calcular el error en la salida y **mandarlo hacia atrás**.

Cada neurona oculta recibe una parte según el peso que la une con la salida.

Las fórmulas

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$	$\Delta w_ho = \eta \cdot \delta_o \cdot a_h$
Oculta	$\delta_h = \delta_o \cdot w_ho \cdot a_h(1 - a_h)$	$\Delta w_ih = \eta \cdot \delta_h \cdot x_i$

a_h = salida de la neurona oculta.

La oculta usa la regla delta, pero su "error" es **δ_o × el peso que la une con la salida**.

Ejemplo (notebook 1_10): hacia adelante

`x = [0.5, 0.1, -0.2] · y = 0.6 · η = 0.5 · W_ih = [[0.5, -0.6], [0.1, -0.2], [0.1, 0.7]] · W_ho = [0.1, -0.3]`

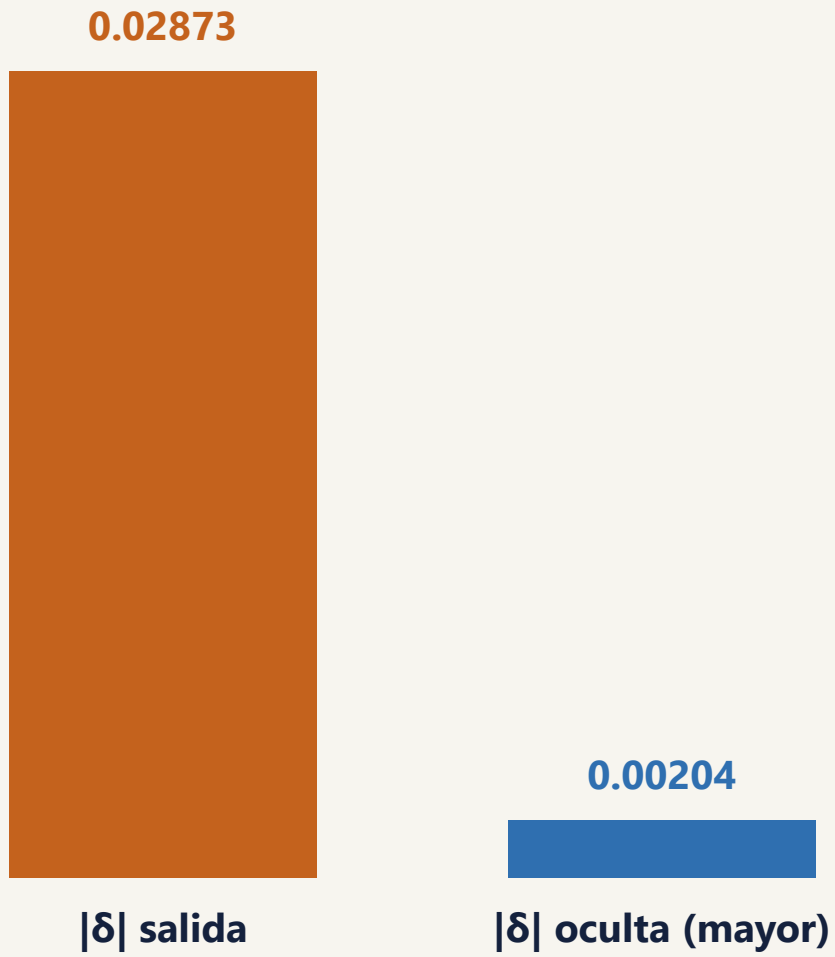
Valor	Cálculo	Resultado
Entrada a la oculta	x · W_ih	[0.24, −0.46]
Salida oculta a_h	σ(…)	[0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3)	−0.0601
Salida ŷ	σ(−0.0601)	**0.4850**

Ejemplo (notebook 1_10): hacia atrás

Valor	Cálculo	Resultado
Error	$0.6 - 0.4850$	0.1150
δ_o	$0.1150 \cdot 0.4850 \cdot 0.5150$	0.02873
δ_h	$0.02873 \cdot [0.1, -0.3] \cdot a_h(1 - a_h)$	$[0.000708, -0.002045]$
Δw_ho	$0.5 \cdot 0.02873 \cdot [0.5597, 0.3870]$	$[0.00804, 0.00556]$
Δw_ih	$0.5 \cdot \delta_h \cdot x$ (matriz 3×2)	$[[0.000177, -0.000511], \ldots]$

Cada peso tiene su incremento: las matrices de incrementos miden lo mismo que las de pesos.

El gradiente que se desvanece



La capa oculta recibe un error ~14 veces más pequeño.

Se multiplica por pesos pequeños y por derivadas de la sigmoide (≤ 0.25).

Con muchas capas, las primeras casi no aprenden.

Por eso las redes profundas usan ReLU.

NOTEBOOKS · 15 MIN · 3 PUNTOS

Práctica · 1_10_retropropagacion_base

TODO 1 pt error, δ de salida y δ de la oculta

TODO 1 pt incrementos y actualización

1 pt ¿mismo tamaño que los pesos?

Pista: para Δw_ih convierte x en columna con x[:, None]: da una matriz 3 × 2.

```
# comprobaciones (η = 0.5)
error = 0.1150
delta salida = 0.02873
delta oculta = [0.000708 -0.002045]
Δw_ho = [0.00804 0.00556]
```

Entrenar la red completa (notebook 1_11)

1. Incrementos acumulados = 0
2. Por ejemplo: adelante → δ_o y δ_h → acumular
3. Actualizar W_ih y W_ho
4. Medir la pérdida y repetir

Actualización	Pérdida	Exactitud
0.001 · acumulado	0.265 → 0.201	0.725
0.5 · acum. / n	0.264 → 0.200	**0.750**

Ojo: el notebook original imprime la pérdida con solo el último ejemplo (x). Usa features.values.

NOTEBOOKS · 20 MIN · 3 PUNTOS · + DOCS/PRACTICA_05.PY

Práctica · 1_11 + ¡XOR resuelto!

TODO 1 pt ciclo de entrenamiento

TODO 1 pt curva de aprendizaje

1 pt preguntas

Demo: python docs/practica_05.py --graficar. La frontera aprendida ya no es una recta.

```
# python docs/practica_05.py  (red 2-3-1)
[0 0] -> 0.025 -> 0
[0 1] -> 0.981 -> 1
[1 0] -> 0.971 -> 1
[1 1] -> 0.026 -> 0
pérdida: 0.2596 -> 0.0006
```

Quiz · 5 minutos

1. ¿Por qué una capa oculta permite resolver XOR?
2. ¿Qué tamaño tiene W entre una capa de 4 y una de 3?
3. ¿Cuántos parámetros tiene una red 2 → 3 → 1?
4. ¿Cómo se calcula el error de una neurona oculta?
5. ¿Qué es el gradiente que se desvanece?

Para llevar

1. Las capas ocultas combinan rectas: resuelven problemas no lineales.
2. Pase hacia adelante: la clase 4 repetida capa por capa.
3. Retropropagación: el error viaja hacia atrás, repartido según los pesos.

Próxima clase: optimizadores (cómo caminar mejor por la montaña del error).

Tarea: notebooks 1_09, 1_10 y 1_11 (9 pts)

Leer: libro, “Retropropagación”

Guía del notebook `1_09_red_multicapa_ejercicio`: una red con capa oculta

Puntos: 3 · **Tiempo:** 20–25 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_09_red_multicapa_ejercicio.ipynb`
Lectura: libro, Redes neuronales multicapa.

¿Qué vas a aprender?

- Qué es una **capa oculta** y por qué hace falta.
- A representar los pesos de cada capa como una **matriz**.
- A calcular la salida de una red de dos capas (**pase hacia adelante**) con productos de matrices.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_09_red_multicapa_ejercicio.ipynb` → kernel **nni**.
   El notebook usa el paquete `nni` y la imagen `files/test1.png` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

¿Por qué capas ocultas?

Una sola neurona solo traza **una recta**, y por eso no puede con XOR (Clases 2 y 4). Si ponemos una **capa de neuronas en medio** (la capa oculta), cada una traza su recta y la neurona de salida las combina. Así la red puede separar regiones con formas más complejas. Esta red también se llama **perceptrón multicapa** (MLP).

La red de este notebook

4 entradas → 3 neuronas ocultas → 2 salidas, como en la figura del notebook.

Pesos como matrices

- Entre la entrada (4) y la oculta (3) hay 4 × 3 = 12 pesos. Van en una matriz **W_1 de 4 filas y 3 columnas**: las filas son las entradas y las columnas, las neuronas de la capa siguiente.
- Entre la oculta (3) y la salida (2) hay una matriz **W_2 de 3 × 2**.

Pase hacia adelante (dos veces lo mismo de la Clase 4)

Paso	Fórmula	Tamaño del resultado
Capa oculta	$H_1 = X \cdot W_1 \ (+ B_1)$ y $A_1 = \sigma(H_1)$	1 × 3
Salida	$H_2 = A_1 \cdot W_2 \ (+ B_2)$ y $\hat{Y} = \sigma(H_2)$	1 × 2

La salida de la capa oculta, $\mathbf{A_1}$, es la entrada de la siguiente.

Regla de los tamaños: para multiplicar (a × b)·(b × c), el número del medio debe coincidir, y el resultado es (a × c). Por ejemplo, (1 × 4)·(4 × 3) = (1 × 3).

Paso a paso

Celda 1: paquetes

Importa NumPy y `nni`, y fija la semilla 42.

Celda: tamaños de las capas

Completa `N_input`, `N_hidden` y `N_output` según la figura: **4, 3 y 2**.

Celda `TODO (1 punto)`: pesos y entrada

1. **Media y desviación:** media 0 y una desviación pequeña. Una buena elección es **1/√(número de entradas)**, la inicialización normal escalada que vimos con Xavier.
2. **Matrices:** `np.random.normal(mean, stdev, size=(N_input, N_hidden))` para W_1, y de tamaño `(N_hidden, N_output)` para W_2. Usa las variables de tamaño, no números fijos.
3. **Entrada:** un vector aleatorio de tamaño `(1, N_input)`, por ejemplo con `np.random.normal(size=(1, N_input))`.

✅ **Comprobación:** imprime las formas. Deben ser W_1 **(4, 3)**, W_2 **(3, 2)** y X **(1, 4)**.

Celda `TODO (1 punto)`: calcular la salida

- `H_1` = producto de X por W_1 (`np.dot`).
- `H_2` = producto de A_1 por W_2. **Ojo:** se usa A_1 (después de la sigmoide), no H_1.

✅ **Comprobación:**

- la salida de la capa oculta tiene **3 números** entre 0 y 1;
- la salida de la red tiene **2 números** entre 0 y 1.

Con la semilla 42, y generando W_1, W_2 y X en ese orden con desviación 1/√4, sale:

- capa oculta ≈ [0.4489, 0.6991, 0.3965];
- red ≈ [0.3210, 0.3627].

Si generas en otro orden, los números cambian, pero los tamaños no.

Último punto (1 punto): sesgos

Agrega una celda nueva con un vector de sesgos para cada capa: **B_1 de tamaño (1, 3)** y **B_2 de tamaño (1, 2)**. Súmalos antes de cada sigmoide.

✅ **Comprobación:**

- con sesgos en cero, la salida es igual a la de antes;
- con otros valores, cambia;
- en total la red tiene 4·3 + 3 + 3·2 + 2 = **23 parámetros**.

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: shapes (1,4) and (3,2) not aligned`	Multiplicaste X por W_2, o H_1 en lugar de A_1	Sigue el orden: X·W_1 → σ → A_1·W_2 → σ
`TypeError: ... NoneType`	Algún tamaño o matriz sigue en `None`	Completa todas las variables
`AttributeError: module 'nni' has no attribute 'sigmoid'`	Notebook abierto fuera del repo, o `nni` viejo	Abre la carpeta `curso-redes-neuronales` y haz `git pull`
Salidas iguales a 0.5	Pesos o entradas todos en cero	Usa `np.random.normal`

Guía del notebook `1_10_retropropagacion_base`: retropropagación, un paso

Puntos: 3 · **Tiempo:** 25–30 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_10_retropropagacion_base.ipynb`
Lectura: libro, Retropropagación.

¿Qué vas a aprender?

- Cómo se **reparte la culpa del error** entre las capas de una red: la **retropropagación** (*backpropagation*).
- A calcular el término de error de la capa de salida y el de la capa oculta.
- A calcular los incrementos de **todas** las matrices de pesos.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_10_retropropagacion_base.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

En la Clase 4 aprendimos a corregir los pesos de **una** neurona, porque conocíamos su error (y − ŷ). Pero en una red con capa oculta, **¿cuál es el "error" de una neurona oculta?** Nadie nos dice cuál debió ser su salida.

La solución: mandar el error hacia atrás

1. Calcula el error en la **salida**, como siempre.
2. **Repártelo** hacia la capa oculta: cada neurona oculta recibe una parte proporcional al **peso** que la conecta con la salida (si influyó mucho, carga más culpa).
3. Con esos términos de error, cada capa se actualiza con la misma regla delta.

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y}) \cdot f'(h_o)$	$\Delta w_ho = \eta \cdot \delta_o \cdot a_h$ (a_h = salida de la oculta)
Oculta	$\delta_h = \delta_o \cdot w_ho \cdot f'(h_h)$	$\Delta w_ih = \eta \cdot \delta_h \cdot x_i$

Con la sigmoide, f′ = salida·(1 − salida).

La red de este notebook

3 entradas → 2 neuronas ocultas → 1 salida, con pesos ya dados.

Paso a paso

Celdas 1–3: funciones y red

Definen la sigmoide, su derivada y la red:

- x = [0.5, 0.1, −0.2], objetivo = 0.6, tasa 0.5;
- pesos de entrada a oculta (3 × 2) y de oculta a salida (2).

Celda: pase frontal (ya está hecha)

Calcula, en este orden:

Valor	Resultado
Entrada a la capa oculta	x·W_ih = [0.24, −0.46]
Salida de la capa oculta	σ(...) ≈ [0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3) ≈ −0.0601
Salida	σ(−0.0601) ≈ 0.4850

Celda `TODO (1 punto)`: términos de error

- `error` = objetivo − salida.
- `del_err_output` = error · salida · (1 − salida).
- `del_err_hidden` = del_err_output · `weights_hidden_output` · salida_oculta · (1 − salida_oculta). Es un vector de 2 números, uno por neurona oculta.

✅ **Comprobación:**

Variable	Valor
error	≈ 0.1150
del_err_output	≈ 0.02873
del_err_hidden	≈ [0.000708, −0.002045]

Celda `TODO (1 punto)`: incrementos y actualización

1. **Tasa:** el notebook pide proponer otra. Para compararte con estos valores, usa primero **0.5**.
2. **Oculta → salida:** `delta_w_h_o` = tasa · del_err_output · salida_oculta.
3. **Entrada → oculta:** `delta_w_i_h` = tasa · del_err_hidden · x, ordenado como **matriz 3 × 2**.
   Pista: `x[:, None]` convierte x en una columna (3 × 1); multiplicada por un vector de 2 da una matriz 3 × 2.
4. **Actualizar:** suma cada incremento a su matriz de pesos.

✅ **Comprobación** (tasa 0.5):

- **Incremento oculta → salida** ≈ [0.00804, 0.00556].
- **Incremento entrada → oculta** ≈

```
[[ 0.000177 -0.000511]
 [ 0.000035 -0.000102]
 [-0.000071  0.000204]]
```

- **Nuevos pesos oculta → salida** ≈ [0.10804, −0.29444].
- Con otra tasa, todos los incrementos se escalan en la misma proporción. Por ejemplo, con 1.0 son el doble.

Pregunta (1 punto)

¿Son del mismo tamaño las matrices de incrementos que las de pesos? Sí: cada peso necesita su propio incremento.

- La de entrada → oculta es **3 × 2**.
- La de oculta → salida tiene **2** elementos.

Puedes comprobarlo con `.shape`.

Para pensar

¿Por qué los incrementos de la capa oculta son tan pequeños? El error llega multiplicado por pesos pequeños (0.1 y −0.3) y por derivadas de la sigmoide (≤ 0.25). En redes con muchas capas, esto se repite y el error casi desaparece: es el problema del **gradiente que se desvanece**.

Errores frecuentes

Síntoma	Causa	Solución
`del_err_hidden` es un solo número	Olvidaste multiplicar por el vector `weights_hidden_output`	Debe dar 2 valores
`ValueError: operands could not be broadcast`	Multiplicaste vectores 3 y 2 sin `x[:, None]`	Convierte x en columna
Valores con el signo contrario	Escribiste salida − objetivo	Es **objetivo − salida**
`TypeError: ... NoneType`	`learnrate` sigue en `None`	Asígnale un número

Guía del notebook `1_11_retropropagacion_completo`: entrenar una red multicapa

Puntos: 3 · **Tiempo:** 30–40 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_11_retropropagacion_completo.ipynb`
Requisito: haber hecho `1_07` (entrenar una neurona) y `1_10` (un paso de retropropagación).

¿Qué vas a aprender?

- A juntar todo: pase hacia adelante, retropropagación y descenso por gradiente, durante muchas épocas.
- A dibujar la **curva de aprendizaje** e interpretarla.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_11_retropropagacion_completo.ipynb` → kernel **nni**.
   El notebook usa `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Mismos datos que en `1_07`:** admisiones a un posgrado, 6 entradas, 360 alumnos para entrenar y 40 para probar.
- **Red:** 6 entradas → **2 neuronas ocultas** → 1 salida, todas con sigmoide.
- **Cada época:**
  1. Pone los incrementos acumulados en cero.
  2. Para cada alumno: pase hacia adelante, error, término de error de la salida y de la oculta (igual que en `1_10`), y **acumula** los incrementos.
  3. Al terminar los alumnos, **actualiza** las dos matrices de pesos.

Paso a paso

Celdas 1–2: datos

- Estadísticas de las entradas: GRE y GPA ya están escalados (media ≈ 0, desviación ≈ 1).
- Clases: **243 no admitidos y 117 admitidos**.

Celda 3: tamaños e hiperparámetros

- `n_hidden = 2`, `epochs = 900` y `learnrate = 0.001`.
- Pesos iniciales al azar, normales con desviación 1/√6, y semilla 21.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los huecos con lo que ya hiciste en `1_10`:

Hueco	Qué escribir
`hidden_input`	x·W_ih (`np.dot`)
`hidden_output`	sigmoide de lo anterior (`functions.sigmoid`)
`output`	sigmoide de hidden_output·W_ho
`error`	y − output
`output_error`	error · output · (1 − output)
`hidden_error`	output_error · `weights_hidden_output` · hidden_output · (1 − hidden_output)
`del_w_hidden_output +=`	output_error · hidden_output
`del_w_input_hidden +=`	hidden_error · x en forma de matriz (`x[:, None]`, como en `1_10`)
Actualizar `weights_... +=`	tasa · incremento acumulado

Dos opciones válidas para la actualización final:

- **A, sin dividir:** pesos += `learnrate` · acumulado, con `learnrate = 0.001`, como viene la plantilla.
- **B, promediado:** pesos += `learnrate` · acumulado / `n_records`, igual que en `1_07`. Así la tasa **no** depende del número de alumnos, pero hay que subirla a ≈ **0.5**.

✅ **Comprobación** (semilla 21, 900 épocas):

Opción	Pérdida inicial → final	Exactitud de prueba
A: 0.001 sin dividir	0.2646 → 0.2013	**0.725**
B: 0.5 con / n_records	0.2641 → 0.1999	**0.750**
⚠️ 0.005 con / n_records	0.2659 → 0.2516	0.425 (casi no aprende)

La última fila es una tasa **demasiado pequeña** para la opción B: con ella la red casi no aprende.

⚠️ **Detalle del notebook original:** el bloque que imprime `Train loss` calcula la pérdida usando **solo el último alumno** (`x`), no todos. Para una curva correcta, usa `features.values` en lugar de `x` en esas dos líneas.

Celda `TODO (1 punto)`: curva de aprendizaje

1. Antes del ciclo, crea una lista vacía, por ejemplo `historial = []`.
2. Al final de **cada** época, calcula la pérdida con todos los alumnos y agrégala con `.append`.
3. Grafica con `plt.plot(historial)`, poniendo nombres a los ejes (época y pérdida).

✅ **Comprobación:** la curva **baja rápido al principio y luego se aplana**.

Preguntas (1 punto)

- **¿La pérdida de entrenamiento disminuye?** Sí, de ≈ 0.26 a ≈ 0.20.

- **¿Cuál es la mejor pérdida que alcanzaste?** Anota el valor, las épocas y la tasa que usaste. Por ejemplo: "0.1999 con 900 épocas y tasa 0.5 promediada".

Para pensar

- **¿Por qué la exactitud (0.75) es casi igual a la de una sola neurona en `1_07` (0.775)?** Estos datos son casi linealmente separables, así que una capa oculta no ayuda mucho. Las capas ocultas brillan con problemas **no lineales**, como XOR o imágenes.
- **Reto:** sube `n_hidden` a 4 u 8. ¿Mejora la exactitud o aparece sobreajuste?

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: operands could not be broadcast together with shapes (6,2) (6,)`	Sumaste `hidden_error * x` sin convertir x en columna	Usa `hidden_error * x[:, None]`
La pérdida no baja	Tasa muy pequeña para la opción B, o te faltó `+=`	Revisa la tabla de opciones
La pérdida impresa sube y baja raro	El cálculo original usa solo el último alumno	Usa `features.values`
`ModuleNotFoundError: data_prep`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
Tarda mucho	Son 900 épocas × 360 alumnos	Es normal: unos segundos. Ten paciencia o baja las épocas

Guía paso a paso — Clase 6: Optimizadores

Temas: descenso por gradiente en 2D · mapas de contorno · momentum · decaimiento de la tasa de aprendizaje
Duración: ~2 h 30 min
Material: diapositivas `Clase6_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clase 3 (descenso por gradiente en una variable) y Clase 5
Lectura base: libro del Dr. Irving Vasquez, Optimización

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Aplicar el descenso por gradiente con dos parámetros e interpretar un mapa de contorno.
2. Explicar qué hace el **momentum** y cuándo ayuda.
3. Programar reglas de **decaimiento** de la tasa de aprendizaje.
4. Comparar optimizadores con experimentos y sacar conclusiones.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría desde cero.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

> **¿Por qué importa?** Entrenar una red es bajar por la "montaña del error" con miles de parámetros. El optimizador es la forma de caminar. Los que veremos hoy (y sus parientes, como Adam) son los que usan las librerías profesionales.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso (10 min)	1–4
0:10	B1. Descenso por gradiente en 2D (20 min)	5–7
0:30	C1. Práctica: notebook `21` (15 min)	8
0:45	B2. Valles estrechos y momentum (25 min)	9–12
1:10	C2. Práctica: notebook `22` (15 min)	13
1:25	Descanso (10 min)	14
1:35	B3. Decaimiento de la tasa de aprendizaje (25 min)	15–18
2:00	C3. Práctica: notebook `23` (20 min)	19

Tiempo	Tema	Diapositivas
2:20	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_06.py
```

Verificación: imprime tres bloques (esfera, momentum, decaimiento) y al final `Todas las comprobaciones pasaron.`

Con `--graficar` dibuja los cuatro caminos sobre la función de Beale. Úsalo en la clase.

Checklist:

- [ ] `python docs/practica_06.py --graficar` funciona.
- [ ] Ejecutaste `git pull`: el notebook `23_opt_lr_decay` es nuevo, porque el original estaba vacío.
- [ ] Sabes qué hacer si no hay ffmpeg: las animaciones de `21` y `22` se pueden guardar como GIF (ver Parte D).

PARTE B — Teoría desde cero

Repaso (5 min) — diapositivas 1–4

- "¿Cuál es la regla del descenso?" → $x \leftarrow x - \eta \cdot f'(x)$.
- "¿Qué pasa si η es muy grande?" → oscila o diverge.
- "¿Y muy pequeña?" → avanza muy lento.

Hoy: qué hacer cuando hay **muchos** parámetros y la "montaña" tiene formas difíciles.

B1. Descenso por gradiente en 2D (20 min) — diapositivas 5–7

Derivadas parciales y gradiente (diapositiva 5)

Con dos variables, f(x, y), hay **una derivada por variable**. Se calcula como si la otra fuera un número fijo.

Ejemplo, la **esfera** $f(x, y) = x^2 + y^2$:

- $\partial f/\partial x = 2x$;
- $\partial f/\partial y = 2y$.

Juntas forman el **gradiente** $\nabla f = (2x, 2y)$: una flecha que apunta hacia donde la función **sube más rápido**.

Regla: cada parámetro baja en contra de su derivada:

- $x \leftarrow x - \eta \cdot \partial f/\partial x$;
- $y \leftarrow y - \eta \cdot \partial f/\partial y$.

Mapa de contorno (diapositiva 6)

Es como un **mapa topográfico**: cada línea (o color) une puntos de la misma "altura". En la esfera son círculos alrededor del mínimo (0, 0).

Un camino de descenso cruza las líneas **en ángulo recto**, porque el gradiente siempre es perpendicular a ellas.

El efecto de η en 2D (diapositiva 7)

Esfera, desde $(-1, -1)$, máximo 300 iteraciones. Son resultados del notebook `21`:

η	Iteraciones	Qué pasa
0.01	300	Lento: termina en $(-0.0023, -0.0023)$, sin llegar
0.1	56	Bien
0.5	2	De un salto (solo en esta función)
0.9	66	Llega zigzagueando
1.0	300	Rebota para siempre entre $(-1, -1)$ y $(1, 1)$

¿Por qué? Cada paso multiplica x por $(1 - 2\eta)$:

- con $\eta = 0.9$, ese factor es -0.8: cambia de signo y se achica;
- con $\eta = 1$, es -1: cambia de signo y nunca se achica;
- con $\eta > 1$, crece y diverge.

B2. Valles estrechos y momentum (25 min) — diapositivas 9–12

El problema (diapositiva 9)

La **función de Beale** tiene un valle **largo, estrecho y curvo**, con el mínimo en (3, 0.5). En valles así, el gradiente apunta casi siempre **hacia las paredes**, no a lo largo del valle. Con η grande, el descenso **rebota** de pared a pared y puede salir disparado.

La idea del momentum (diapositiva 10)

En lugar de seguir solo el gradiente de este instante, se usa un **promedio** de los gradientes recientes:

- $m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$
- parámetros ← parámetros $- \eta \cdot m$

β (beta, entre 0 y 1) es la "memoria". Con β = 0.9, el paso de ahora se parece mucho a los anteriores.

Intuición (diapositiva 11)

Analogía: una bola que rueda por el valle. Los empujones hacia los lados (las paredes) cambian de signo en cada rebote y **se cancelan** en el promedio. El empuje a lo largo del valle va siempre en la misma dirección y **se mantiene**.

β	Efecto
0	Sin memoria: es el descenso simple
0.5–0.9	Suaviza los rebotes (lo habitual)
0.99	Demasiada memoria: le cuesta frenar y girar

Resultados (diapositiva 12)

Beale desde (2.5, 1.0), 500 iteraciones. Son resultados del notebook `22` y de `practica_06.py`:

η	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	$f = 6.95 \cdot 10^{-4}$	$f = 7.37 \cdot 10^{-4}$	$f = 1.07 \cdot 10^{-3}$
0.03	**diverge**	$f = 1.9 \cdot 10^{-6}$	**$f = 8.0 \cdot 10^{-7}$**

Conclusión honesta:

- Con η pequeña, los tres llegan casi igual. Esta versión de momentum **promedia**: no acelera los pasos.
- Su ventaja es que **permite usar una η más grande** sin que el entrenamiento se descontrole, y así llega mucho más cerca del mínimo.

B3. Decaimiento de la tasa de aprendizaje (25 min) — diapositivas 15–18

El dilema (diapositiva 15)

Con η **fija**:

- si es grande, avanza rápido pero, cerca del mínimo, rebota;
- si es pequeña, afina bien pero tarda muchísimo.

Solución: empezar grande y **reducirla** con el tiempo.

Tres reglas (diapositiva 16)

t es el número de iteración.

Regla	Fórmula	Idea
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$	Cada s iteraciones multiplica por γ (por ejemplo, a la mitad)
Inversa	$\eta_0 / (1 + k \cdot t)$	Baja rápido al inicio, luego despacio
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$	Baja suave y continua

Calcular a mano (diapositiva 17)

Con $\eta_0 = 0.05$, $\gamma = 0.5$, $s = 100$ y $k = 0.01$:

t	Escalonada	Inversa	Exponencial
0	0.05	0.05	0.05
100	0.025	0.025	0.0184
250	0.0125	0.0143	0.0041

Haz en el pizarrón la escalonada en t = 250: 250 // 100 = 2, y $0.05 \cdot 0.5^2 = 0.0125$.

Resultados (diapositiva 18)

Beale desde (1.0, 1.0), $\eta_0 = 0.05$, 500 iteraciones. Son resultados del notebook `23`:

Configuración	f final
Tasa fija 0.05	$5.8 \cdot 10^{-2}$ (rebota, no llega)
Escalonada	**$6.3 \cdot 10^{-5}$**
Inversa	$1.0 \cdot 10^{-4}$
Exponencial	$1.5 \cdot 10^{-3}$ (decae demasiado rápido y se "congela")
Escalonada con $\gamma = 0.8$	**$7.8 \cdot 10^{-7}$**

Mensaje:

- El decaimiento escalonado da casi **mil veces** menos error que la tasa fija.
- Si la tasa decae **demasiado** rápido, el optimizador se queda quieto antes de llegar.

PARTE C — Práctica en la computadora

Guías celda por celda: `docs/guias_notebooks/` (NB_21, NB_22 y NB_23).

C1. `notebooks/21_opt_gradiente_2d.ipynb` (15 min, experimentos) — diapositiva 8

- Ejecuta todo con los valores originales.
  ✅ **Comprobación:** termina en ≈ (−0.00233, −0.00233), con f ≈ $1.09 \cdot 10^{-5}$.
- Cambia `learning_rate` a 0.1, 0.5, 0.9 y 1.0, y compara con la tabla de B1.
- La animación necesita ffmpeg. Si no está, usa `.save('gradient_descent.gif', writer='pillow')`.

C2. `notebooks/22_opt_momentum.ipynb` (15 min, experimentos) — diapositiva 13

- Ejecuta con los valores originales.
  ✅ **Comprobación:** termina en ≈ (2.922, 0.480), con f ≈ $1.07 \cdot 10^{-3}$.
- Sube `learning_rate` a 0.03 y compara con el descenso simple: quita el `#` de la línea comentada.
  ✅ **Comprobación:** el descenso simple diverge (`OverflowError`) y el momentum llega a ≈ (2.998,

0.499).

- **Crea un optimizador nuevo** en cada prueba: el objeto recuerda su momentum.

C3. `notebooks/23_opt_lr_decay.ipynb` (20 min, 3 puntos) — diapositiva 19

- **TODO 1:** las tres reglas.
  ✅ **Comprobación:** los valores de la tabla de B3.
- **TODO 2:** `next_params` con la tasa de la iteración actual.
  ✅ **Comprobación:** tasa fija f = $5.76 \cdot 10^{-2}$, contra decaimiento f = $6.34 \cdot 10^{-5}$.
- **TODO 3:** experimentos de la tabla de resultados.

Demo final:

```
python docs/practica_06.py --graficar
```

Muestra los cuatro caminos: el descenso lento, el momentum con η grande, la tasa fija rebotando y el decaimiento llegando al mínimo.

Quiz y cierre (10 min) — diapositivas 20–21

Pregunta	Respuesta
¿Qué es el gradiente de f(x, y)?	El vector de derivadas parciales; apunta hacia donde f sube más rápido
¿Por qué el descenso rebota en un valle estrecho?	El gradiente apunta a las paredes, no a lo largo del valle
¿Qué controla β en el momentum?	Cuánta memoria de los gradientes anteriores tiene
¿Qué ventaja dio el momentum en Beale?	Permitió una η grande (0.03) sin diverger
¿Qué pasa si la tasa decae demasiado rápido?	Se "congela" antes de llegar al mínimo

Para llevar:

1. Más parámetros = mismo descenso, una derivada por parámetro.
2. El momentum suaviza los rebotes y permite tasas más grandes.
3. El decaimiento empieza rápido y termina fino.

Tarea:

1. Terminar el notebook `23` (3 puntos) y entregar los experimentos de `21` y `22` con una conclusión.
2. En `docs/practica_06.py`, prueba el momentum con β = 0.99 y η = 0.03: ¿qué pasa?
3. Investigar en una frase qué es el optimizador **Adam**: combina momentum con una tasa que se adapta a cada parámetro.

Rúbrica:

Notebook	Puntos
21 (experimentos y conclusión)	1

Notebook	Puntos
22 (experimentos y conclusión)	1
23	3

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`RuntimeError: ... MovieWriter (ffmpeg) not available`	La animación necesita ffmpeg	Cambia a `.save('archivo.gif', writer='pillow')`, o salta esa celda
`OverflowError: (34, 'Result too large')`	El descenso diverge	Baja η, o usa momentum
Aviso `Locator attempting to generate 1001 ticks`	`levels=1000` en el contorno	Solo es un aviso; puedes bajar a 100
Resultados raros al repetir una prueba	Reusaste el optimizador (guarda su momentum o su contador t)	Crea uno nuevo
La tasa en `23` no cambia	Falta `self.t += 1`	Incrementa el contador en `next_params`
`AttributeError: module 'nni' has no attribute ...`	Notebook fuera del repo, o `nni` viejo	Abre la carpeta del repo y haz `git pull`

CLASE 6 · OPTIMIZACIÓN

Optimizadores: caminar mejor

Descenso por gradiente en 2D, momentum y decaimiento de la tasa de aprendizaje

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso
0:10	Descenso por gradiente en 2D
0:30	Práctica: notebook 21
0:45	Valles estrechos y momentum
1:10	Práctica: notebook 22
1:25	Descanso
1:35	Decaimiento de la tasa
2:00	Práctica: notebook 23
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Aplicar

el descenso con dos parámetros.

2

Leer

un mapa de contorno.

3

Explicar

el momentum y cuándo ayuda.

4

Programar

el decaimiento de la tasa.

Repaso

¿La regla?

$x \leftarrow x - \eta \cdot f'(x)$

¿η muy grande?

Oscila o diverge.

¿η muy pequeña?

Avanza muy lento.

Hoy: muchos parámetros y montañas con formas difíciles.

Descenso por gradiente en 2D

Esfera: $f(x, y) = x^2 + y^2$

$\partial f/\partial x = 2x$

$\partial f/\partial y = 2y$

Una derivada por variable (la otra queda fija).

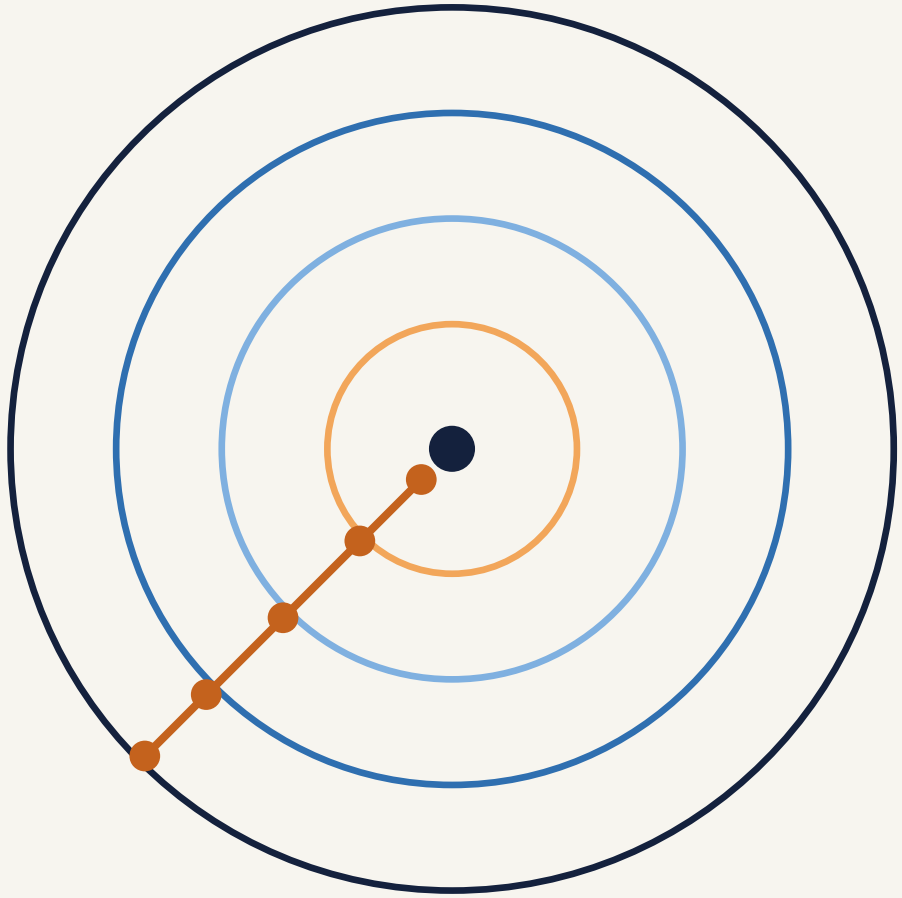
Gradiente $\nabla f = (2x, 2y)$

Una flecha que apunta hacia donde f sube más rápido.

Bajamos en contra.

$$x \leftarrow x - \eta \cdot \partial f/\partial x$$
$$y \leftarrow y - \eta \cdot \partial f/\partial y$$

El mapa de contorno



Como un mapa topográfico: cada línea une puntos de la misma “altura”.

En la esfera son círculos alrededor del mínimo (0, 0).

El camino cruza las líneas en ángulo recto: el gradiente es perpendicular a ellas.

El efecto de η en 2D (notebook 21)

η	Iteraciones	Qué pasa
0.01	300	Lento: termina en (−0.0023, −0.0023)
0.1	56	Bien
0.5	2	De un salto (solo en esta función)
0.9	66	Llega zigzagueando
1.0	300	Rebota para siempre entre (−1, −1) y (1, 1)

Cada paso multiplica x por $(1 - 2\eta)$: con η > 1 crece y diverge.

NOTEBOOKS · 15 MIN · EXPERIMENTOS

Práctica · 21_opt_gradiente_2d

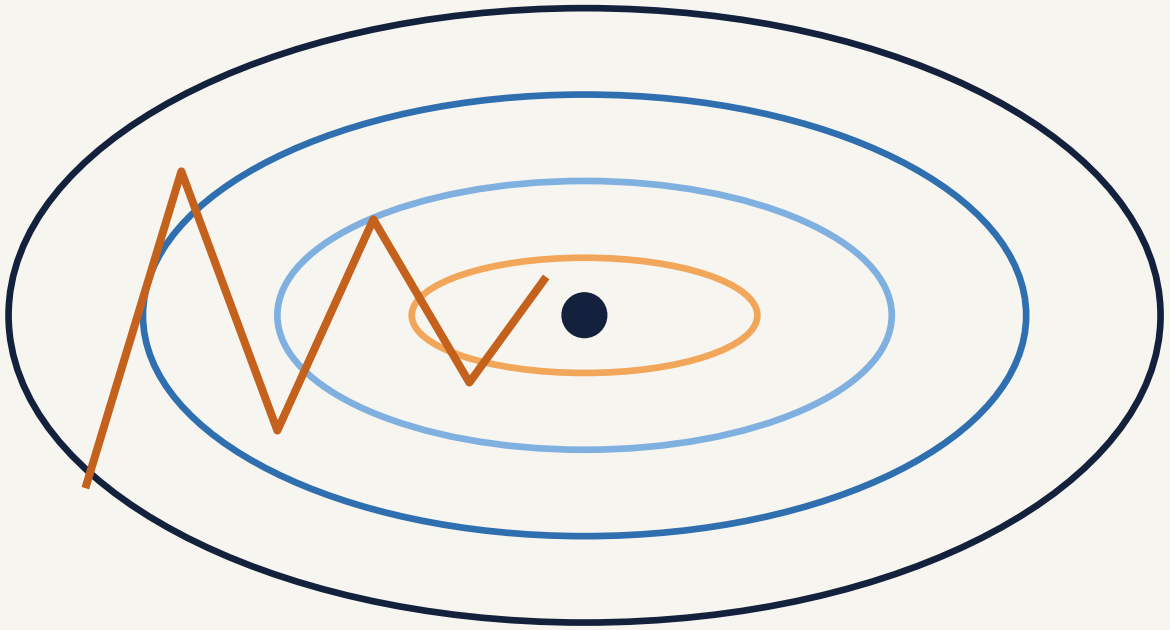
1. Ejecuta con los valores originales
2. Cambia learning_rate: 0.1, 0.5, 0.9, 1.0
3. Anota iteraciones y compáralas con la tabla

La clase GradientDescentOptimizer guarda cada paso en history: así se dibuja el camino.

```
# valores originales
Parámetros: [-0.00233, -0.00233]
Valor: 1.09e-05

# sin ffmpeg, la animación:
.save('x.gif', writer='pillow')
```

Valles estrechos: la función de Beale



Valle largo, estrecho y curvo. Mínimo en (3, 0.5).

El gradiente apunta casi siempre hacia las paredes.

Con η grande, el descenso rebota de pared a pared y puede salir disparado.

La idea del momentum

$$m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$$
$$\text{parámetros} \leftarrow \text{parámetros} - \eta \cdot m$$

Promedio de gradientes

En vez del gradiente de este instante, un promedio de los recientes.

β = memoria

Con β = 0.9 el paso de ahora se parece mucho a los anteriores.

Una bola rodando por el valle

Los empujones hacia las paredes cambian de signo en cada rebote:

en el promedio se cancelan.

El empuje a lo largo del valle va siempre en la misma dirección:

se mantiene.

β	Efecto
0	Sin memoria: descenso simple
0.5 – 0.9	Suaviza los rebotes
0.99	Demasiada memoria: le cuesta girar

Resultados en Beale (notebook 22)

Desde (2.5, 1.0), 500 iteraciones

η	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	$f = 6.95 \cdot 10^{-4}$	$f = 7.37 \cdot 10^{-4}$	$f = 1.07 \cdot 10^{-3}$
0.03	**diverge**	$f = 1.9 \cdot 10^{-6}$	**$f = 8.0 \cdot 10^{-7}$**

El momentum permite usar una η más grande sin descontrolarse… y llega mucho más cerca del mínimo.

NOTEBOOKS · 15 MIN · EXPERIMENTOS

Práctica · 22_opt_momentum

1. Ejecuta con los valores originales
2. Sube learning_rate a 0.03
3. Compara con el descenso simple (línea comentada)

Crea un optimizador nuevo en cada prueba: el objeto recuerda su momentum.

```
# originales (η 0.01, β 0.9)
final ≈ (2.922, 0.480)  f ≈ 1.07e-03

# η 0.03
descenso simple: OverflowError
momentum: ≈ (2.998, 0.499)
```

Descanso · 10 min

Después: el decaimiento de la tasa

El dilema de la tasa fija

η grande

Avanza rápido… pero cerca del mínimo rebota.

η pequeña

Afina bien… pero tarda muchísimo.

Decaimiento

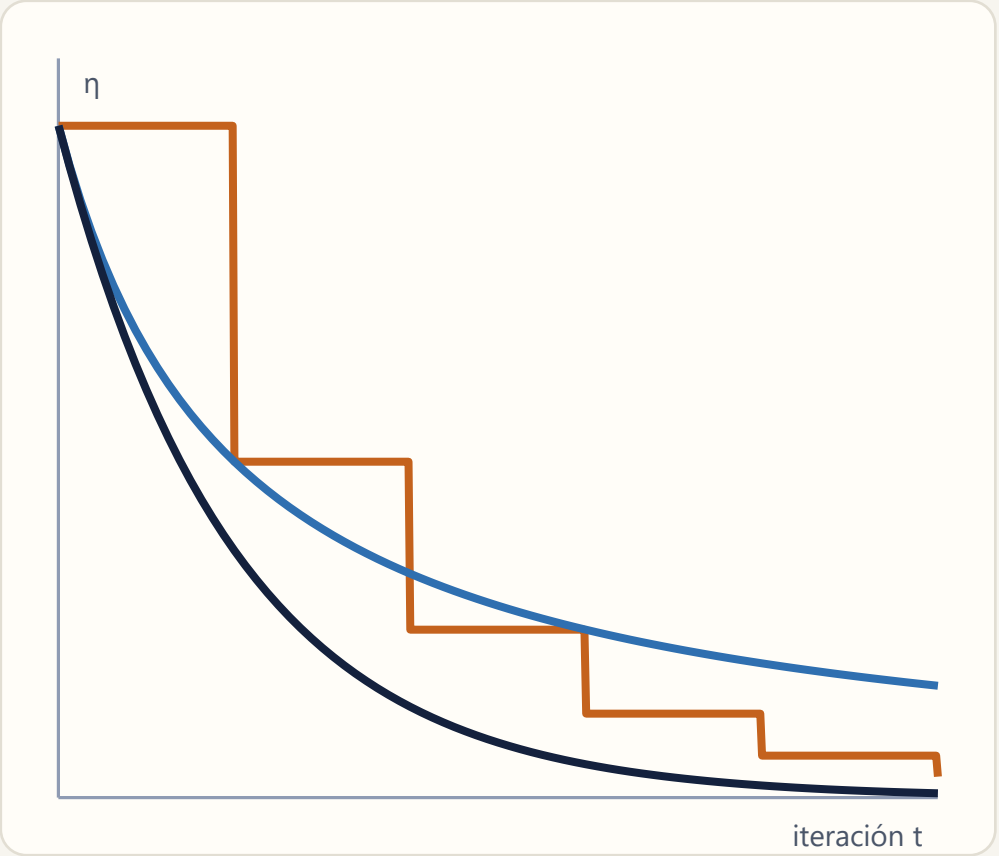
Empieza grande y la reduce con el tiempo.

Lo mejor de los dos mundos: rápido al principio, fino al final.

Tres reglas de decaimiento

Regla	Fórmula
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$
Inversa	$\eta_0 / (1 + k \cdot t)$
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$

— escalonada — inversa — exponencial



Calcular a mano

$\eta_0 = 0.05 \cdot \gamma = 0.5$ cada $s = 100 \cdot k = 0.01$

t	Escalonada	Inversa	Exponencial
0	0.05	0.05	0.05
100	0.025	0.025	0.0184
250	0.0125	0.0143	0.0041

Escalonada en t = 250: 250 // 100 = 2 → $0.05 \cdot 0.5^2 = 0.0125$

Resultados en Beale (notebook 23)

Desde (1.0, 1.0), $\eta_0 = 0.05$, 500 iteraciones

Configuración	f final	Qué pasa
Tasa fija 0.05	$5.8 \cdot 10^{-2}$	Rebota, no llega
Escalonada	$6.3 \cdot 10^{-5}$	Casi mil veces mejor
Inversa	$1.0 \cdot 10^{-4}$	Muy bien
Exponencial	$1.5 \cdot 10^{-3}$	Decae demasiado rápido: se congela
Escalonada con $\gamma = 0.8$	**$7.8 \cdot 10^{-7}$**	La mejor

NOTEBOOKS · 20 MIN · 3 PUNTOS · NUEVO (EL ORIGINAL ESTABA VACÍO)

Práctica · 23_opt_lr_decay

TODO 1 pt las tres reglas de decaimiento

TODO 1 pt next_params con la tasa de cada iteración

TODO 1 pt experimentar y comparar

Pista: guarda la tasa en self.tasas, súmale 1 a self.t y devuelve param − tasa · gradiente.

```
# comprobaciones
t=100: 0.02500  0.02500  0.01839
t=250: 0.01250  0.01429  0.00410

fija 0.05:     f = 5.76e-02
decaimiento:   f = 6.34e-05
```

Quiz · 5 minutos

1. ¿Qué es el gradiente de f(x, y)?
2. ¿Por qué el descenso rebota en un valle estrecho?
3. ¿Qué controla β en el momentum?
4. ¿Qué ventaja dio el momentum en Beale?
5. ¿Qué pasa si la tasa decae demasiado rápido?

Para llevar

1. Más parámetros: mismo descenso, una derivada por parámetro.
2. Momentum: suaviza rebotes y permite tasas más grandes.
3. Decaimiento: empezar rápido y terminar fino.

Próxima clase: redes convolucionales, el puente al aprendizaje profundo.

Tarea: notebook 23 (3 pts) + conclusiones de 21 y 22

Leer: Goodfellow et al., Deep Learning, secciones 8.3 (momentum) y 8.5 (Adam)

Guía del notebook `21_opt_gradiente_2d`: descenso por gradiente en dos dimensiones

Puntos: experimentos, sin `TODO` · **Tiempo:** 20 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/21_opt_gradiente_2d.ipynb`
Requisito: `1_05` (descenso por gradiente en una variable).

¿Qué vas a aprender?

- A extender el descenso por gradiente a **dos parámetros** a la vez.
- A leer un **mapa de contorno**, que es como un mapa topográfico de la función.
- A ver con tus propios ojos el efecto de la tasa de aprendizaje.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/21_opt_gradiente_2d.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Función esférica:** $f(x, y) = x^2 + y^2$. Es un "tazón" cuyo fondo está en (0, 0).
- **Gradiente:** con dos variables hay una derivada por cada una (**derivadas parciales**):
  - $\partial f/\partial x = 2x$, que se calcula como si y fuera un número fijo;
  - $\partial f/\partial y = 2y$.
  - Juntas forman el **gradiente** $\nabla f = (2x, 2y)$: una flecha que apunta hacia donde la función sube más rápido.
- **Regla:** igual que en 1D, pero para cada parámetro: $x \leftarrow x - \eta \cdot \partial f/\partial x$ y $y \leftarrow y - \eta \cdot \partial f/\partial y$.
- **Contorno:** cada color es una "altura". El punto rojo es el mínimo; la línea blanca, el camino del descenso.

Recorrido por las celdas

1. **Paquetes y el problema:** `nni.SphereProblem()` trae la función, sus derivadas y el área a dibujar (de −2 a 2).
2. **Gráficas:** el mapa de contorno (izquierda) y la superficie 3D (derecha).
3. **Clase** `GradientDescentOptimizer` (léela con calma):
   - `next_params`: aplica la regla a cada parámetro;
   - `optimize`: repite hasta llegar al máximo de iteraciones o hasta que los parámetros casi no cambien (`tolerance`);
   - `history`: guarda cada paso para dibujarlo.
4. **Hiperparámetros:** inicio (−1, −1), tasa 0.01 y 300 iteraciones.

5. **Optimizar.**
   ✅ **Comprobación:** con los valores originales, `Parámetros optimizados` ≈ **[−0.00233, −0.00233]** y `Mejor valor encontrado` ≈ **$1.09 \cdot 10^{-5}$**. Usa las 300 iteraciones: con 0.01 avanza lento.
6. **Camino en el contorno:** una línea recta desde (−1, −1) hacia el centro.
7. **Animación:** guarda un video `gradient_descent.mp4`.

> ⚠️ El video **necesita ffmpeg**. Si sale `RuntimeError: Requested MovieWriter (ffmpeg) not available`, cambia el final de esa línea por `.save('gradient_descent.gif', writer='pillow')`. Así guarda un GIF sin instalar nada más.

Experimentos sugeridos

Cambia `learning_rate` en la celda de hiperparámetros y vuelve a ejecutar desde ahí. Resultados reales desde (−1, −1), con un máximo de 300 iteraciones:

Tasa	Iteraciones	Dónde termina	Qué pasa
0.01	300 (el máximo)	(−0.0023, −0.0023)	Lento: no termina de llegar
0.1	56	(0, 0)	Bien
0.5	2	(0, 0)	Llega de un salto (solo en esta función)
0.9	66	(0, 0)	Llega, pero **zigzagueando** de un lado al otro
1.0	300	(−1, −1)	**Rebota para siempre** entre (−1, −1) y (1, 1)
> 1.0	—	se aleja	**Diverge**

¿Por qué? Cada paso multiplica x por $(1 - 2\eta)$:

- con $\eta = 0.9$, ese factor es −0.8: cambia de signo, pero se achica;
- con $\eta = 1$, es −1: cambia de signo sin achicarse nunca;
- con $\eta > 1$, crece en cada paso.

Para pensar

- Cambia el punto inicial a (2, −1.5). ¿El camino sigue siendo una recta? (En la esfera, sí.)
- ¿Qué pasaría en una función con forma de valle alargado, en lugar de un tazón redondo? Lo verás en el notebook 22.

Errores frecuentes

Síntoma	Causa	Solución
`RuntimeError: ... MovieWriter (ffmpeg) not available`	No hay ffmpeg	Guarda como GIF con `writer='pillow'`, o salta esa celda
Aviso `Locator attempting to generate 1001 ticks`	`levels=1000` en el contorno	Solo es un aviso; puedes bajar a `levels=100`

Síntoma	Causa	Solución
`AttributeError: module 'nni' has no attribute 'SphereProblem'`	Notebook abierto fuera del repo, o `nni` viejo	Abre la carpeta `curso-redes-neuronales` y haz `git pull`
`OverflowError`	Tasa > 1: los números crecen sin límite	Usa una tasa menor que 1

Guía del notebook `22_opt_momentum`: el optimizador con momentum

Puntos: experimentos, sin `TODO` obligatorios · **Tiempo:** 20–25 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/22_opt_momentum.ipynb`
Requisito: `21_opt_gradiente_2d`.

¿Qué vas a aprender?

- Qué es el **momentum** y por qué ayuda en valles estrechos.
- A comparar dos optimizadores sobre una función difícil, la de **Beale**.
- Que el momentum permite usar tasas de aprendizaje con las que el descenso simple **se descontrola**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/22_opt_momentum.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

La función de Beale tiene un **valle largo, estrecho y curvo**. Con el descenso simple, si la tasa es grande el algoritmo **rebota** contra las paredes del valle y se puede salir disparado; si es pequeña, avanza muy lento.

Momentum = recordar los pasos anteriores

En lugar de seguir solo el gradiente de este momento, el optimizador lleva un **promedio** de los gradientes recientes:

$m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$ y parámetros $\leftarrow$ parámetros $- \eta \cdot m$

- **β** (beta, entre 0 y 1) es cuánta "memoria" tiene. Con $\beta = 0.9$, el paso de ahora se parece mucho a los anteriores.
- Los rebotes de un lado a otro **se cancelan** en el promedio, y la dirección constante, a lo largo del valle, **se mantiene**.

Analogía: una bola rodando por un valle no cambia de dirección bruscamente con cada piedrita; sigue su camino.

Función de Beale: su mínimo está en (3, 0.5) y vale 0. El notebook dibuja el **logaritmo** de la función para que se vea el valle.

Recorrido por las celdas

1. **Paquetes y problema:** `nni.ReducedBealeProblem()`, que dibuja x entre 0 y 4.5 e y entre −2 y 2.
2. **Gráficas:** contorno (con el valle curvo) y superficie 3D.
3. **Clase `MomentumOptimizer`:** hereda del descenso por gradiente y solo cambia `next_params` para usar la fórmula de arriba.
   El comentario `TODO: initialize dinamically` señala que `self.momentum = np.zeros(2)` supone 2 parámetros. Una mejora opcional: crear el vector del tamaño de `params` la primera vez que se llama a `next_params`.
4. **Hiperparámetros:** inicio (2.5, 1.0), tasa 0.01, 500 iteraciones y β = 0.9.
5. **Optimizar.**
   ✅ **Comprobación** (valores originales): termina ≈ **(2.922, 0.480)** con f ≈ **$1.07 \cdot 10^{-3}$**.
6. **Camino en el contorno** y **animación**, que necesita ffmpeg; ver el aviso de abajo.

Experimentos (resultados reales, desde (2.5, 1.0), 500 iteraciones)

Para comparar, el notebook trae comentada la línea del descenso simple: quita el `#` de esa línea y pon `#` en la del momentum.

Tasa	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	f ≈ $6.95 \cdot 10^{-4}$	f ≈ $7.37 \cdot 10^{-4}$	f ≈ $1.07 \cdot 10^{-3}$
0.03	**diverge** (`OverflowError`)	f ≈ $1.9 \cdot 10^{-6}$ ✓	**f ≈ $8.0 \cdot 10^{-7}$** ✓

Cómo leer la tabla:

- Con tasa pequeña (0.01) los tres llegan casi igual. El momentum no acelera aquí, porque esta versión **promedia**: no amplifica los pasos.
- Con tasa grande (0.03) el descenso simple rebota hasta "explotar", mientras que el momentum **suaviza los rebotes** y llega prácticamente al mínimo (3, 0.5).
- **Conclusión:** el momentum permite usar tasas más grandes con seguridad.

⚠️ **Crea un optimizador nuevo cada vez que pruebes otros valores.** El objeto guarda su `momentum` de la ejecución anterior.

Para pensar

- ¿Qué pasa con β = 0.99? → Tiene demasiada memoria: tarda en frenar y en girar con la curva del valle (con tasa 0.01 queda en f ≈ 0.12).
- ¿Y con β = 0? → La fórmula se vuelve el descenso simple.

Errores frecuentes

Síntoma	Causa	Solución
`OverflowError: (34, 'Result too large')`	El descenso simple divergió con tasa grande	Es justamente lo que se quiere mostrar; usa momentum o baja la tasa

Síntoma	Causa	Solución
`RuntimeError: ... ffmpeg not available`	La animación necesita ffmpeg	Cambia a `.save('momentum_beta.gif', writer='pillow')`
Resultados raros al repetir	Reusaste el mismo optimizador	Crea uno nuevo en cada prueba
`AttributeError: ... 'GradientDescentOptimizer'`	`nni` viejo o notebook fuera del repo	`git pull` y abre la carpeta del repo

Guía del notebook `23_opt_lr_decay`: decaimiento de la tasa de aprendizaje

Puntos: 3 · **Tiempo:** 25 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/23_opt_lr_decay.ipynb`
Requisito: `21` y `22`.

> En el repositorio original del curso este notebook está **vacío**. El de este repositorio es nuevo, hecho con el mismo estilo y la misma función de Beale que el 22.

¿Qué vas a aprender?

- A **reducir la tasa de aprendizaje** mientras se entrena: grande al principio para avanzar rápido, pequeña al final para afinar.
- Tres reglas de decaimiento y cómo programarlas.
- A crear tu propio optimizador **heredando** del de `nni`.

Antes de empezar

1. `conda activate nni`, y `git pull` para tener este notebook.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/23_opt_lr_decay.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

Con una tasa **fija** hay un dilema: si es grande, rebota cerca del mínimo; si es pequeña, tarda muchísimo. Con **decaimiento**, empezamos grande y la vamos bajando.

Regla	Fórmula	En Python
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$	cada `cada` iteraciones multiplica por `gamma`
Inversa	$\eta_0 / (1 + k \cdot t)$	baja rápido al inicio y luego despacio
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$	baja suave y continuo (`math.exp`)

- t es el número de iteración (0, 1, 2…).
- `t // s` es la división entera: 250 // 100 = 2.

Paso a paso

Celdas 1–3: paquetes y función de Beale

Igual que en el notebook 22. El mínimo está en (3, 0.5).

Celda `TODO (1 punto)`: las tres reglas

Escribe las tres funciones con las fórmulas de la tabla.

✅ **Comprobación:** la celda siguiente debe imprimir:

```
t =   0:  escalonada = 0.05000   inversa = 0.05000   exponencial = 0.05000
t = 100:  escalonada = 0.02500   inversa = 0.02500   exponencial = 0.01839
t = 250:  escalonada = 0.01250   inversa = 0.01429   exponencial = 0.00410
```

Celda `TODO (1 punto)`: `DecayOptimizer.next_params`

Sigue los 4 pasos escritos en los comentarios:

1. Calcula la tasa de esta iteración: `self.regla(self.lr0, self.t)`.
2. Guárdala en `self.tasas` (con `.append`) y en `self.learning_rate`.
3. Suma 1 a `self.t`.
4. Devuelve los parámetros nuevos: para cada par (parámetro, gradiente), parámetro − tasa · gradiente. Pista: es igual a la clase del notebook 21, pero con la tasa que calculaste.

Celda: comparar tasa fija y decaimiento

Desde (1.0, 1.0), con $\eta_0 = 0.05$ y 500 iteraciones.

✅ **Comprobación:**

```
Tasa fija 0.05:       termina en (2.866, 0.409),  f = 5.76e-02
Con decaimiento:      termina en (2.980, 0.495),  f = 6.34e-05
```

Con decaimiento, el error es casi **mil veces menor**. La gráfica muestra:

- con tasa fija, el camino **rebotando** dentro del valle;
- con decaimiento, el camino asentándose en el mínimo;
- la tasa de aprendizaje bajando "en escalones".

Celda `TODO (1 punto)`: experimenta

✅ **Resultados de referencia** (desde (1.0, 1.0), 500 iteraciones):

Configuración	Termina en	f
Escalonada (γ = 0.5 cada 100)	(2.980, 0.495)	$6.3 \cdot 10^{-5}$
Inversa (k = 0.01)	(2.975, 0.494)	$1.0 \cdot 10^{-4}$
Exponencial (k = 0.01)	(2.908, 0.476)	$1.5 \cdot 10^{-3}$
Tasa fija 0.01	(2.910, 0.477)	$1.4 \cdot 10^{-3}$
Escalonada con γ = 0.8	**(2.998, 0.499)**	$\mathbf{7.8 \cdot 10^{-7}}$

La exponencial con k = 0.01 **baja demasiado rápido**: en la iteración 250 la tasa ya es 0.004, y se "congela" antes de llegar. Quedó casi igual que la tasa fija pequeña.

Para usar γ = 0.8 sin cambiar la función, puedes pasar una regla "envuelta": `regla=lambda lr0, t: tasa_escalonada(lr0, t, gamma=0.8)`.

Preguntas (orientación)

- **¿Por qué con 0.05 fijo no llega al mínimo?** El paso es demasiado grande para el valle estrecho: salta de una pared a la otra.
- **¿Qué ventaja tiene empezar grande y reducir?** Avanza rápido cuando está lejos y afina cuando está cerca.
- **¿Y si la tasa decae demasiado rápido?** Se queda "congelado" antes de llegar, como la exponencial de la tabla.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... NoneType`	Alguna regla o `next_params` devuelve `None`	Completa todos los `return`
La tasa nunca cambia	No incrementas `self.t`	Agrega `self.t += 1`
La gráfica de la tasa sale vacía	No guardas en `self.tasas`	`self.tasas.append(tasa)`
Resultados distintos al repetir	Reusaste el mismo optimizador (su `t` sigue contando)	Crea uno nuevo en cada prueba

Guía paso a paso — Clase 7: Redes convolucionales (puente al aprendizaje profundo)

Temas: imágenes como matrices · correlación cruzada · kernels y detección de bordes · ReLU · aprender un kernel con descenso por gradiente
Duración: ~2 h 35 min
Material: diapositivas `Clase7_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clases 4, 5 y 6 (descenso por gradiente, regla de la cadena, momentum)

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar por qué para imágenes se usan **convoluciones** en lugar de capas densas.
2. Calcular a mano la correlación cruzada de una imagen pequeña con un kernel.
3. Programar la operación de una capa convolucional.
4. Entrenar los números de un kernel con descenso por gradiente.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría desde cero.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso (10 min)	1–4
0:10	B1. Imágenes y por qué convolución (15 min)	5–6
0:25	B2. Correlación cruzada (25 min)	7–9
0:50	B3. Detector de bordes y ReLU (10 min)	10–11
1:00	C1. Práctica: notebook `3_1` (30 min)	12
1:30	Descanso (10 min)	13
1:40	B4. Aprender el kernel (25 min)	14–18
2:05	C2. Reto: notebook `3_2` y `practica_07` (20 min)	19

Tiempo	Tema	Diapositivas
2:25	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_07.py
```

Verificación:

- el ejemplo del pizarrón da `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`;
- el kernel aprendido es `[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]`, el filtro de Sobel;
- al final dice `Todas las comprobaciones pasaron`.

Con `--graficar` muestra la foto original, sus bordes verticales, sus bordes horizontales y la curva del error.

Checklist:

- [ ] `python docs/practica_07.py --graficar` funciona.
- [ ] En el pizarrón: una cuadrícula de 5 × 5 con números (la imagen del ejemplo de B2) y otra de 3 × 3 (el kernel).
- [ ] Mira de antemano la estrategia del reto en la guía de `3_2`: es la parte más difícil del curso.

PARTE B — Teoría desde cero

Repaso (5 min) — diapositivas 1–4

- "¿Qué hace una capa de una red?" → H = X·W + B y luego la activación.
- "¿Cómo se entrenan los pesos?" → descenso por gradiente, con la regla de la cadena.

Hoy aplicamos lo mismo a **imágenes**.

B1. Imágenes y por qué convolución (15 min) — diapositivas 5–6

Una imagen es una matriz (diapositiva 5)

- Una imagen en escala de grises es una **matriz de números**: 0 es negro y 255 es blanco.
- Una imagen a color son **3 matrices**: rojo, verde y azul.
- La foto de los notebooks mide **256 × 256**: 65 536 números por canal.

¿Por qué no una capa densa? (diapositiva 6)

Con una capa densa (Clase 5), cada píxel se conecta con cada neurona. Con solo 100 neuronas ya serían 65 536 × 100 = **6.5 millones de pesos**. Además:

- trata cada píxel por separado e **ignora que los vecinos están relacionados**;
- un gato en la esquina y un gato en el centro serían problemas distintos.

La convolución usa un filtro pequeño (por ejemplo 3 × 3 = **9 pesos**) que **recorre toda la imagen**. Los mismos 9 pesos se usan en cada posición:

- buscan el **mismo patrón** (un borde, una textura) en todas partes;
- hay muchísimos menos parámetros.

B2. Correlación cruzada (25 min) — diapositivas 7–9

Cómo funciona (diapositiva 7)

1. Coloca el **kernel** (una matriz pequeña) sobre una zona de la imagen.
2. Multiplica cada número del kernel por el píxel que tiene debajo.
3. **Suma** todo: ese es un píxel de la imagen de salida.
4. Desliza el kernel una posición y repite.

En fórmula: $G[i, j] = \Sigma_u \Sigma_v H[u, v] \cdot I[i + u, j + v]$. Los índices u y v van de $-k$ a k; con un kernel de 3 × 3, k = 1.

> Las librerías llaman "convolución" a esta operación. Estrictamente es una correlación cruzada: la convolución matemática voltea el kernel. Para las redes da igual, porque el kernel se aprende.

Ejemplo en el pizarrón (diapositiva 8)

Imagen 5 × 5: oscuro (0) a la izquierda y claro (9) a la derecha. Cada fila es `0 0 9 9 9`.

Kernel 3 × 3: cada fila es `-1 0 1`. Resta la izquierda y suma la derecha.

Píxel de salida en la posición (1, 1), con la ventana en las columnas 0 a 2:

- cada fila aporta $(-1)\cdot 0 + 0\cdot 0 + 1\cdot 9 = 9$;
- son 3 filas: **27**.

Las tres posiciones de la fila:

Columnas de la ventana	Valores	Resultado
0–2	`0 0 9`	27
1–3	`0 9 9`	27

Columnas de la ventana	Valores	Resultado
2–4	`9 9 9`	0 (zona plana: sin borde)

Salida 3 × 3: `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`.

El kernel **se enciende donde hay un borde vertical** y vale 0 donde la imagen es plana.

Tamaño de la salida (diapositiva 9)

- Sin relleno (**padding 0**), el kernel no puede centrarse en las orillas. La salida pierde k píxeles de cada lado: n − 2k.
  - imagen 5 × 5, kernel 3 × 3 → salida **3 × 3**;
  - imagen 256 × 256 → salida **254 × 254**.
- Con **padding** (rellenar la orilla con ceros), la salida puede medir lo mismo que la entrada.

B3. Detector de bordes y ReLU (10 min) — diapositivas 10–11

El kernel del notebook `3_1` (diapositiva 10)

Cada fila es `-1/3 0 1/3`. Es nuestro ejemplo dividido entre 3:

- da valores **positivos** donde la imagen pasa de **oscuro a claro** (de izquierda a derecha);
- da valores **negativos** donde pasa de claro a oscuro;
- da **≈ 0** en zonas planas.

Después de la ReLU (diapositiva 11)

La ReLU convierte los negativos en 0. Por eso, en la imagen final solo se ven **algunos** bordes verticales: los de oscuro a claro. Tampoco aparecen bordes horizontales, porque este kernel solo compara izquierda contra derecha.

Esa es la respuesta a la pregunta del notebook `3_1`.

B4. Aprender el kernel (25 min) — diapositivas 14–18

La idea (diapositiva 14)

En una red convolucional **nadie elige** los números del kernel: **se aprenden**, igual que los pesos de la Clase 4. Los 9 (o 25) números del kernel son los **pesos** de la capa convolucional.

El gradiente de un peso del kernel (diapositiva 15)

Llamemos:

- G = correlación(K, I);
- P = ReLU(G), la predicción;
- T, la imagen objetivo;
- ECM = promedio de $(P - T)^2$.

El peso K[u, v] se usa en **todas** las posiciones. Por la regla de la cadena, su gradiente **suma la contribución de cada posición**:

$\partial ECM/\partial K[u, v] = (2/N) \cdot \Sigma\ (P - T)[i, j] \cdot [G[i, j] > 0] \cdot I[i + u, j + v]$

Se lee: el error en cada píxel, por si la ReLU dejó pasar (1 o 0), por el píxel que se multiplicó por ese peso. Luego: **K ← K − η·gradiente**.

Las trampas (diapositiva 16)

1. **ReLU muerta:** si al inicio todos los G son negativos, la ReLU da 0 en todos lados y ningún peso recibe gradiente. Con fotos (todos los píxeles positivos) y un kernel aleatorio pasa muy seguido.
2. **Escala:** con píxeles de 0 a 255 los gradientes son enormes. Hay que normalizar a 0–1.
3. **Lentitud:** calcular píxel por píxel con `for` tarda minutos por época.

Los trucos que funcionan:

- normalizar (dividir entre 255);
- **centrar** la entrada (restarle su promedio);
- **vectorizar**;
- usar **momentum** (Clase 6).

`practica_07.py` muestra el efecto de centrar: con la imagen sin centrar, el entrenamiento **diverge**.

Vectorizar (diapositiva 17)

En lugar de recorrer píxeles, se preparan las versiones **desplazadas** de la imagen: 9 para un kernel de 3 × 3, o 25 para uno de 5 × 5.

- La correlación es la suma de cada versión por su peso.
- El gradiente de cada peso es un promedio sobre una matriz completa.
- Todo con NumPy: milisegundos por época.

Resultados (diapositiva 18)

- **`practica_07.py`:** sobre una imagen de ruido centrada, el descenso **recupera exactamente** el filtro de Sobel `[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]`, con ECM de 0.97 a ≈ 0.
- **Reto** `3_2` (foto real, kernel de 5 × 5, ReLU, momentum β = 0.9, η = 2, 2000 épocas):
  - el ECM baja de ≈ 31 600 a ≈ **29**;
  - el kernel aprendido es un Sobel en el centro (fila −0.86, −1.77, −0.89 arriba y 0.93, 1.79, 0.84 abajo);
  - la imagen predicha es casi idéntica al objetivo.

Mensaje final: una red convolucional profunda hace esto mismo con **muchos** kernels y **muchas** capas. Las primeras capas aprenden bordes; las siguientes, formas; las últimas, objetos.

PARTE C — Práctica en la computadora

Guías celda por celda: `docs/guias_notebooks/` (NB_3_1 y NB_3_2).

C1. `notebooks/3_1_capa_convolucional.ipynb` (30 min, 4 puntos) — diapositiva 12

- **`correlacionPixel`:** dos `for` en `range(-k, k + 1)`, acumulando `H[u+k, v+k] * I[i+u, j+v]`. `resultado` debe ser un número de NumPy, por ejemplo `np.array(sumatoria)`.

- `G`: ceros del mismo tamaño que I; al final, recórtalo a `G[k:m-k, k:n-k]`.
- **Pregunta:** ver B3.

✅ **Comprobación:**

- la salida mide **254 × 254**;
- tras la ReLU se ven solo algunos bordes verticales, como en `files/ccnout.png`.

C2. Reto `notebooks/3_2_entrenar_capa_conv.ipynb` y `practica_07.py` (20 min) — diapositiva 19

- Sigue la estrategia de B4. El detalle está en la guía NB_3_2.
- Si el tiempo no alcanza, deja el reto de tarea y muestra en vivo:

```
python docs/practica_07.py --graficar
```

Quiz y cierre (10 min) — diapositivas 20–21

Pregunta	Respuesta
¿Por qué no usar capas densas para imágenes?	Demasiados pesos, e ignoran que los píxeles vecinos se relacionan
¿Cuánto mide la salida de una imagen 7 × 7 con un kernel 3 × 3, sin padding?	5 × 5
¿Qué detecta el kernel con filas −1 0 1?	Bordes verticales (de oscuro a claro, hacia la derecha)
¿Por qué se ven solo algunos bordes tras la ReLU?	La ReLU elimina los valores negativos (bordes de claro a oscuro)
¿Qué son los "pesos" de una capa convolucional?	Los números del kernel

Para llevar:

1. Convolución = un filtro pequeño que recorre la imagen con los mismos pesos.
2. Los kernels detectan patrones (bordes, texturas), y **se aprenden** con descenso por gradiente.
3. Las redes profundas apilan muchas capas convolucionales: es el siguiente curso.

Tarea:

1. Terminar `3_1` (4 puntos).
2. Intentar el reto `3_2`, opcional y con puntos extra.
3. En `practica_07.py`, cambia el kernel objetivo por el detector vertical `[[-1, 0, 1]] * 3` y comprueba que también lo aprende.

Rúbrica:

Notebook	Puntos
3_1	4

Notebook	Puntos
3_2 (reto)	Extra

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`AttributeError: 'float' object has no attribute 'astype'`	`resultado` es un número normal de Python	`np.array(sumatoria)`
La imagen sale con un marco negro	No recortaste G	`G[k:m-k, k:n-k]`
La predicción del reto es toda negra	ReLU muerta	Centra la entrada; ver B4
El ECM explota o sale `nan`	Tasa grande o datos sin normalizar	Divide entre 255 y baja η
Cada época tarda minutos	Cálculo píxel por píxel	Vectoriza con versiones desplazadas
`FileNotFoundError: files/Lenna.png`	Notebook fuera de la carpeta del repo	Abre la carpeta `curso-redes-neuronales`

CLASE 7 · PUENTE AL APRENDIZAJE PROFUNDO

Redes convolucionales

Imágenes, kernels, detección de bordes y cómo se aprende un filtro

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso
0:10	Imágenes y por qué convolución
0:25	Correlación cruzada
0:50	Detector de bordes y ReLU
1:00	Práctica: 3_1
1:30	Descanso
1:40	Aprender el kernel
2:05	Reto: 3_2 y practica_07
2:25	Quiz y cierre

Al terminar la clase podrás…

1

Explicar

por qué se usan convoluciones en imágenes.

2

Calcular

una correlación cruzada a mano.

3

Programar

la operación de una capa convolucional.

4

Entrenar

los números de un kernel con descenso.

Repaso

¿Una capa?

$H = X \cdot W + B$ y una activación.

¿Cómo se entrena?

Descenso por gradiente.

¿Momentum?

Suaviza rebotes y permite η grande.

Hoy: lo mismo, pero sobre imágenes.

Una imagen es una matriz de números

30	60	120	200	240	250
25	55	110	190	235	245
20	50	100	180	230	240
20	45	95	170	225	235
15	40	90	160	220	230

Escala de grises: 0 = negro, 255 = blanco.

A color: 3 matrices (rojo, verde, azul).

La foto de los notebooks: 256 × 256 = 65 536 números por canal.

¿Por qué no una capa densa?

Capa densa

65 536 píxeles × 100 neuronas

= 6.5 millones de pesos

Ignora que los píxeles vecinos se relacionan.

Convolución

Un filtro de 3 × 3

= 9 pesos

Los mismos pesos recorren toda la imagen: buscan el mismo patrón en todas partes.

Un gato en la esquina o en el centro: el mismo filtro lo detecta.

El kernel se desliza sobre la imagen



1. Coloca el kernel.
2. Multiplica cada número por el píxel de abajo.
3. Suma todo: un píxel de salida.
4. Desliza y repite.

Ejemplo en el pizarrón

Cada fila de la imagen: `0 0 9 9 9` · cada fila del kernel: `−1 0 1`

Ventana (columnas)	Valores	Cálculo por fila	× 3 filas
0 – 2	0 0 9	−0 + 0 + 9 = 9	27
1 – 3	0 9 9	−0 + 0 + 9 = 9	27
2 – 4	9 9 9	−9 + 0 + 9 = 0	0 (zona plana)

Salida: `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`

El kernel se enciende donde hay un borde vertical y vale 0 donde la imagen es plana.

Tamaño de la salida

$$\text{sin padding:}\quad \text{salida} = n - 2k$$

Imagen	Kernel	k	Salida
5 × 5	3 × 3	1	3 × 3
256 × 256	3 × 3	1	254 × 254
256 × 256	5 × 5	2	252 × 252

Con padding (rellenar la orilla con ceros) la salida puede medir lo mismo que la entrada.

El kernel del notebook 3_1

−1/3	0	1/3
−1/3	0	1/3
−1/3	0	1/3

Positivo donde la imagen pasa de oscuro a claro (→).

Negativo donde pasa de claro a oscuro.

≈ **0** en zonas planas.

¿Por qué solo se ven algunos bordes?

La ReLU borra negativos

Los bordes de claro a oscuro dan valores negativos: la ReLU los convierte en 0.

Solo compara izquierda y derecha

Los bordes horizontales no cambian de izquierda a derecha: este kernel no los ve.

Es la respuesta a la pregunta del notebook 3_1.

NOTEBOOKS · 30 MIN · 4 PUNTOS

Práctica · 3_1_capa_convolucional

TODO 1 pt correlacionPixel: dos for de −k a k

TODO 1 pt G de ceros del tamaño de I

TODO 1 pt recortar las orillas: G[k:m-k, k:n-k]

1 pt pregunta de la ReLU

Tras la ReLU se ven solo algunos bordes verticales, como en files/ccnout.png.

```
# comprobaciones
imagen: (256, 256) uint8
salida: (254, 254)

# resultado debe ser de NumPy:
resultado = np.array(sumatoria)
```

Descanso · 10 min

Después: aprender el kernel

Los números del kernel se aprenden

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

En una red convolucional nadie elige los números del kernel.

Son los pesos de la capa: se aprenden con descenso por gradiente.

Igual que en la clase 4, pero ahora la “predicción” es una imagen entera.

El gradiente de un peso del kernel

$$\partial ECM/\partial K[u,v] = (2/N) \cdot \Sigma\ (P - T) \cdot [G > 0] \cdot I[i+u,\ j+v]$$

Pieza	Significado
P − T	error en cada píxel (predicción − objetivo)
[G > 0]	1 si la ReLU dejó pasar, 0 si no (derivada de la ReLU)
I[i+u, j+v]	el píxel que se multiplicó por ese peso

Luego: $K \leftarrow K - \eta \cdot \text{gradiente}$

Trampas y trucos

Trampas

- **ReLU muerta:** todo G negativo → sin gradiente.
- **Escala:** píxeles 0–255 → gradientes enormes.
- **Lentitud:** píxel por píxel tarda minutos.

Trucos

- **Normalizar:** dividir entre 255.
- **Centrar:** restar el promedio.
- **Vectorizar** y usar **momentum**.

Vectorizar: versiones desplazadas

```
# una versión desplazada por cada peso (u, v)
d = {(u, v): I[k+u:m-k+u, k+v:n-k+v]
     for u in range(-k, k+1) for v in range(-k, k+1)}

G = sum(K[u+k, v+k] * d[(u, v)] for (u, v) in d)
```

La correlación es la suma de cada versión por su peso.

El gradiente de cada peso es un promedio sobre una matriz completa.

Milisegundos por época.

Resultado: el descenso descubre el filtro de Sobel

Sobel (objetivo)

-1	-2	-1
0	0	0
1	2	1

Aprendido (reto 3_2)

−0.86	−1.77	−0.89
−0.03	−0.01	0.04
0.93	1.79	0.84

practica_07: recupera Sobel exacto (ECM 0.97 → 0).

Reto 3_2: ECM 31 600 → 29; la imagen predicha es casi idéntica.

Una CNN profunda hace esto con muchos kernels y muchas capas: bordes → formas → objetos.

NOTEBOOKS · 20 MIN · RETO (PUNTOS EXTRA)

Práctica · 3_2 · reto + practica_07

1. Normaliza y centra la entrada
2. Vectoriza con versiones desplazadas
3. Descenso con momentum β 0.9, η 2, 2000 épocas

Guía con la estrategia completa:
docs/guias_notebooks/NB_3_2.

```
# referencia del reto
ECM inicial ≈ 31600
ECM final   ≈ 29
kernel ≈ Sobel en el centro

# demo:
# python docs/practica_07.py --graficar
```

Quiz · 5 minutos

1. ¿Por qué no usar capas densas para imágenes?
2. ¿Cuánto mide la salida de una imagen 7×7 con un kernel 3×3?
3. ¿Qué detecta el kernel con filas $-1\ 0\ 1$?
4. ¿Por qué se ven solo algunos bordes tras la ReLU?
5. ¿Qué son los “pesos” de una capa convolucional?

Para llevar

1. Convolución: un filtro pequeño recorre la imagen con los mismos pesos.
2. Los kernels detectan patrones y se aprenden con descenso por gradiente.
3. Las redes profundas apilan muchas capas convolucionales.

Fin del curso introductorio. Siguiente: aprendizaje profundo.

Tarea: notebook 3_1 (4 pts) + reto 3_2 (extra)

Leer: Goodfellow et al., Deep Learning, cap. 9 (redes convolucionales)

Guía del notebook `3_1_capa_convolucional`: la operación de una capa convolucional

Puntos: 4 · **Tiempo:** 30 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_1_capa_convolucional.ipynb`

¿Qué vas a aprender?

- Qué es una **imagen** para la computadora: una matriz de números.
- Qué es un **kernel** (filtro) y cómo se **desliza** sobre la imagen: la correlación cruzada.
- A programar a mano la operación central de las **redes convolucionales** (CNN), que se usan para reconocer imágenes.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_1_capa_convolucional.ipynb` → kernel **nni**.
   El notebook abre `files/Lenna.png`, así que debe abrirse desde la carpeta del repo.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

La imagen

Una imagen en color son 3 matrices: rojo, verde y azul. Aquí se usa solo el **canal rojo**, una matriz de **256 × 256** con valores de 0 (negro) a 255 (blanco).

El kernel y la correlación cruzada

Un **kernel** es una matriz pequeña, aquí de 3 × 3. Se coloca sobre cada píxel de la imagen y se calcula:

$G[i, j] = \Sigma_u \Sigma_v H[u, v] \cdot I[i + u, j + v]$

En palabras:

1. Pon el kernel centrado en el píxel (i, j).
2. Multiplica cada número del kernel por el píxel que tiene debajo.
3. Suma los 9 productos.
4. Ese total es el píxel (i, j) de la imagen de salida G.

- Los índices u y v van de $-k$ a k. Con un kernel de 3 × 3, k = 1, y van de −1 a 1.
- En la matriz de Python, el elemento H[u, v] de la fórmula está en `H[u + k, v + k]`, porque los índices de Python empiezan en 0.

El kernel de este notebook: detector de bordes verticales

```
[[-1/3  0  1/3]
 [-1/3  0  1/3]
 [-1/3  0  1/3]]
```

Resta la columna de la izquierda y suma la de la derecha. Por eso da un número **grande y positivo** donde la imagen pasa de **oscuro a claro** de izquierda a derecha (un borde vertical), y **negativo** en el caso contrario.

Bordes de la imagen

En las orillas el kernel "se sale" de la imagen. Con **padding 0** (sin relleno), esos píxeles no se calculan. La salida es más pequeña: 256 − 2k = **254 × 254**.

Paso a paso

Celdas 1–2: paquetes y la imagen

✅ **Comprobación:** `Dimensiones de la imagen: (256, 256)` y `Tipo de datos de la matriz: uint8`.

Celda del kernel

Construye el detector de bordes y lo muestra.

Celda `TODO`: las dos funciones (3 puntos)

`correlacionPixel(H, I, i, j)` (1 punto)

- Con dos `for` anidados: `u` y `v` en `range(-k, k + 1)`. Acumula en `sumatoria` el producto `H[u + k, v + k] * I[i + u, j + v]`.
- Al final, `resultado` debe ser un número de NumPy, porque la función hace `resultado.astype(int)`. Por ejemplo, `resultado = np.array(sumatoria)`.
- Atajo opcional: `np.sum(H * I[i-k:i+k+1, j-k:j+k+1])` hace lo mismo en una línea.

`correlacionCruzadaValida(H, I)`

- *(1 punto)* `G` debe ser una matriz de ceros **del mismo tamaño** que I: `np.zeros((m, n))`.
- *(1 punto)* Al final, quita las orillas que no se llenaron. Pista: recorta `k` filas y `k` columnas de cada lado, `G[k:m-k, k:n-k]`.

Celda "Prueba"

Aplica tu función, luego la ReLU (que deja en 0 los negativos), y dibuja tres imágenes.

✅ **Comprobación:**

- la imagen "Correlación cruzada" se ve gris, con los contornos marcados en claro y oscuro;
- "ReLU aplicada" se ve negra, con **solo algunos bordes verticales** en blanco;
- debe parecerse a la figura `files/ccnout.png` del notebook;
- el resultado mide **254 × 254**: compruébalo con `resultado.shape`.

Pregunta (1 punto)

¿Por qué tras la ReLU solo se ven algunos bordes verticales?

- El kernel da valores **positivos** en los bordes donde se pasa de **oscuro (izquierda) a claro (derecha)**, y **negativos** en los bordes de claro a oscuro.
- La ReLU convierte todos los negativos en 0, así que esos bordes desaparecen.
- Tampoco se ven los bordes **horizontales**, porque este kernel solo compara izquierda contra derecha.

Para pensar

- Si **volteas** el kernel (los +1/3 a la izquierda), ¿qué bordes aparecen? → Los otros, los que antes eran negativos.
- En una CNN, los números del kernel **no se eligen a mano**: la red los aprende. Eso es el notebook `3_2`.

Errores frecuentes

Síntoma	Causa	Solución
`AttributeError: 'float' object has no attribute 'astype'`	`resultado` es un número normal de Python	`resultado = np.array(sumatoria)`
`IndexError: index 256 is out of bounds`	Recorriste hasta el borde	Los `for` deben ir de `k` a `m-k` (ya vienen así)
La imagen sale con un marco negro	No recortaste G	`return G[k:m-k, k:n-k]`
Valores raros, enormes o negativos de golpe	Operaste con enteros `uint8`, que "dan la vuelta" al pasar de 255	Multiplica por el kernel, que es float, o convierte con `I.astype(float)`
`FileNotFoundError: files/Lenna.png`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
Tarda unos segundos	Son 254 × 254 píxeles × 9 productos con `for`	Es normal

Guía del notebook `3_2_entrenar_capa_conv`: reto, entrenar un kernel convolucional

Puntos: reto, abierto · **Tiempo:** 45–90 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_2_entrenar_capa_conv.ipynb`
Requisito: `3_1` (correlación cruzada) y `1_07` (descenso por gradiente).

¿Qué vas a aprender?

- Que los números de un kernel se pueden **aprender** con descenso por gradiente, igual que los pesos de una neurona.
- A calcular el gradiente de una capa convolucional.
- Trucos prácticos que hacen que un entrenamiento real funcione: **normalizar**, **centrar**, **vectorizar** y usar **momentum**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_2_entrenar_capa_conv.ipynb` → kernel **nni**.
   Usa `files/Lenna.png`, `files/sobel_horizontal.png` y el paquete `nni`.

El reto

Tenemos:

- la **entrada**: el canal rojo de la foto (256 × 256);
- el **objetivo**: la misma foto procesada con un detector de **bordes horizontales** (filtro de Sobel), también 256 × 256.

Hay que encontrar los **25 números de un kernel de 5 × 5** tales que ReLU(correlación(kernel, entrada)) ≈ objetivo.

Es el problema de la Clase 4, con dos cambios:

- los "pesos" son los 25 números del kernel;
- la "predicción" es una imagen entera.

La idea: el gradiente de un peso del kernel

Llamemos:

- G = correlación(K, I);
- P = ReLU(G), la predicción;
- T, el objetivo;
- ECM = promedio de $(P - T)^2$.

Un número del kernel, K[u, v], participa en **todos** los píxeles: en cada uno se multiplica por I[i + u, j + v]. Por la regla de la cadena:

∂ECM/∂K[u, v] = (2/N) · Σ sobre todos los píxeles (i, j) de: (P − T)[i, j] · [G[i, j] > 0] · I[i + u, j + v]

- (P − T) es el error en ese píxel.
- [G > 0] vale 1 si la ReLU dejó pasar el valor y 0 si no: la derivada de la ReLU.
- I[i + u, j + v] es el píxel que se multiplicó por ese peso.

Luego se aplica el descenso por gradiente: **K ← K − η·gradiente**.

Paso a paso

Celdas 1–3: imágenes

✅ **Comprobación:** ambas imágenes miden **(256, 256)**, son `uint8`, y el objetivo va de 0 a 255.

Celda: kernel aleatorio y pase frontal

El kernel es aleatorio y la celda no fija semilla, así que el resultado cambia en cada ejecución. Muy a menudo la predicción sale **toda en negro** (mínimo 0.0 y máximo 0.0): como todos los píxeles de la foto son positivos, si los 25 números del kernel suman un valor negativo, casi todos los G son negativos y la ReLU los deja en 0.

Así **ningún peso recibe gradiente** (es una "ReLU muerta"), y el descenso no puede empezar. Es la primera trampa del reto.

✅ **Comprobación:** cuando la predicción sale toda negra, el `Error cuadrático medio` es ≈ **1185.46**, que es exactamente el error de predecir todo en 0. Si te sale otro número, tu kernel aleatorio dio valores positivos: es normal.

Celdas con las funciones sugeridas

El notebook propone calcular el gradiente **píxel por píxel** con tres funciones auxiliares. Eso es correcto, pero son 252 × 252 píxeles × 25 pesos ≈ 1.6 millones de operaciones de Python **por época**: tardaría minutos por época.

Estrategia que funciona (probada)

1. **Normaliza:** divide la imagen y el objetivo entre 255, para que queden entre 0 y 1. Así los gradientes no son gigantes.
2. **Centra la entrada:** réstale su promedio. Así G ya no es "todo negativo" al inicio y la ReLU no muere. Como el kernel buscado suma 0, centrar no cambia la solución.
3. **Vectoriza:** en lugar de recorrer píxel por píxel, prepara las 25 versiones **desplazadas** de la imagen, una por cada (u, v). Por ejemplo, `I[k+u : m-k+u, k+v : n-k+v]`.
   - La correlación completa es la suma de cada versión desplazada por su peso.
   - El gradiente de cada peso es el promedio de (error · máscara · su versión desplazada) por 2.
   - Todo con operaciones de NumPy sobre matrices completas: cada época tarda milisegundos.
4. **Usa momentum** (notebook 22): V ← β·V + gradiente y K ← K − η·V, con β = 0.9. La imagen tiene píxeles vecinos muy parecidos, lo que hace el valle del error muy alargado: justo donde el momentum ayuda.
5. **Hiperparámetros** que funcionaron:
   - kernel inicial `np.random.normal(size=(5, 5))` con semilla 42;

- tasa η = 2.0;
- β = 0.9;
- **2000 épocas**, unos 5 segundos.

✅ **Resultado de referencia:**

- **ECM:** de ≈ 31 600 a ≈ **29**, en unidades de píxel².
- **Kernel aprendido:** prácticamente el **filtro de Sobel** (bordes horizontales), en el centro del 5 × 5:

```
[[ 0.05 -0.01 -0.09 -0.02  0.07]
 [-0.03 -0.86 -1.77 -0.89 -0.11]
 [-0.05 -0.03 -0.01  0.04  0.06]
 [ 0.05  0.93  1.79  0.84  0.06]
 [-0.05 -0.01  0.08  0.03 -0.08]]
```

  Compáralo con Sobel: fila de arriba −1, −2, −1; centro 0; fila de abajo +1, +2, +1.
- **Imagen:** la predicción es casi idéntica al objetivo.

Para pensar

- **¿Por qué la red "descubrió" el filtro de Sobel?** Porque el objetivo se hizo con él. Una CNN real aprende sus filtros del mismo modo, pero el objetivo es una tarea (reconocer gatos, por ejemplo) y no una imagen dada.
- **¿Por qué funciona mejor con momentum?** Sin él, con tasas grandes el entrenamiento rebota y "mata" la ReLU; con tasas pequeñas avanza muy lento.

Errores frecuentes

Síntoma	Causa	Solución
La predicción siempre es negra y el ECM no cambia	ReLU muerta: todos los G son negativos	Centra la entrada; inicializa con valores pequeños
Cada época tarda minutos	Cálculo píxel por píxel con `for`	Vectoriza con las 25 versiones desplazadas
El ECM sale `nan` o explota	Tasa demasiado grande	Baja η, o normaliza entre 0 y 1 si no lo hiciste
ECM gigantesco al inicio	Trabajas con valores 0–255	Divide imagen y objetivo entre 255
La forma no coincide con el objetivo	La correlación válida mide 252 × 252	Recorta el objetivo igual: `T[k:m-k, k:n-k]`